

Kubernetes en Production

Kubernetes Production · Lesson 0

One path: **build** clusters with **Kubespray**, **operate** them with admin skills (**CKA**)

Welcome

This formation **merges** the CKA administration course with the Kubespray deployment course into **one coherent path** — from install to Day-2.

The big idea:

- **Build** the cluster with **Kubespray** (production-grade, Ansible-driven)
- **Operate & administer** it with core admin skills (**CKA scope**)
- **kubeadm** is kept only as the *under-the-hood* mechanism Kubespray drives

Goal: not just pass an exam, but **build and run** production Kubernetes with confidence.

Learning Objectives

By the end of this lesson you will be able to:

- Describe **what Kubernetes is** — the production container platform
- Name the **control plane** and **worker** components at a glance
- Explain this course's promise: **build with Kubespray + operate (CKA scope)**
- Justify **why Kubespray** for installation vs manual `kubeadm`, kops, managed K8s
- Recognise that **Kubespray uses kubeadm under the hood**
- Map the **four phases** and the **24-lesson roadmap**

Reference: *Kubernetes overview* — kubernetes.io/docs · *Kubespray* — kubespray.io

0.1 What Is Kubernetes?

Kubernetes — The Production Container Platform

Kubernetes (K8s) is an open-source platform for **automating the deployment, scaling and management** of containerized applications.

It takes a fleet of machines and makes it behave like **one resilient computer**:

- **Schedules** containers (grouped into **Pods**) across the nodes
- **Self-heals** — restarts failed containers, reschedules off dead nodes
- **Scales** workloads up and down on demand
- **Rolls out** changes and **rolls back** on failure
- **Exposes & load-balances** services, manages config and secrets

It is the de-facto standard for running containers in production.

Control Plane + Workers at a Glance

A cluster has a **control plane** (the brain) and **worker nodes** (the muscle).

Plane	Component	Role
Control plane	kube-apiserver	The front door — the cluster API
	etcd	Key-value store of all cluster state
	kube-scheduler	Places Pods onto nodes
	kube-controller-manager	Reconciles desired vs actual state
Worker node	kubelet	Node agent — runs the Pods
	container runtime	containerd / CRI-O — runs containers
	kube-proxy	Service networking on the node

We dissect each component in **Lesson 1 — Architecture**.

Declarative & Self-Healing

You don't issue step-by-step commands — you **declare the desired state**.

```
# "I want 3 replicas of nginx, always"
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

1. You **apply** YAML → the **API server** stores it in **etcd**.
2. **Controllers** continuously **reconcile** reality to match the spec.
3. A Pod dies? The controller makes a new one. No human in the loop.

This *reconciliation loop* is the heart of how Kubernetes operates.

0.2 The Course Promise

One Path: Build It, Then Run It

This is **one** formation, not two stitched together. The redundancy between a CKA course and a Kubespray course has been removed.

You will...	...with
Build production clusters	Kubespray (Ansible playbooks)
Operate & administer them	core admin skills (CKA scope)
Go from install → Day-2	one continuous numbering (00–23)

The biggest change vs a classic CKA path: cluster **creation is taught with Kubespray** instead of a hand-cranked `kubeadm` walkthrough.

kubeadm isn't dropped — it's the engine Kubespray drives (more on the next section).

Build vs Run — Two Skill Sets, One Cluster

Build the cluster (Kubespray) → Use the cluster (CKA app/admin) → Operate it (Day-2) (Kubespray + CKA) → Advanced (Kubespray)

- **Build (Kubespray / Ansible):** stand up and maintain the **fleet** — nodes, runtime, CNI, HA, upgrades.
- **Run (CKA / `kubect1`):** deploy and administer the **workloads** — apps, storage, scheduling, RBAC, troubleshooting.

Same cluster, two complementary lenses. You leave able to do **both**.

0.3 Why Kubespray for Installation

Why Kubespray (vs the Alternatives)

Approach	Model	Trade-off
Manual <code>kubeadm</code>	Bootstraps one cluster; you hand-wire OS prep, CNI, HA	Educational, not repeatable at scale
<code>kops</code>	Provisions the infra itself (mostly AWS/GCP)	Single-cloud , couples infra + cluster
Managed (EKS/AKS/GKE)	Cloud runs the control plane	You never learn to operate it
Kubespray	Ansible drives kubeadm across the whole fleet + OS prep + CNI + add-ons + Day-2	Production, multi-node, multi-platform, repeatable

Kubespray = a CNCF / `kubernetes-sigs` project that deploys a **production-ready** cluster with Ansible.

Kubespray's Edge

- **Repeatable & idempotent** — the cluster is described in an **inventory** + **group_vars**; re-runs change only what's needed.
- **HA out of the box** — ≥ 3 control-plane / etcd nodes, load balancing, kube-vip.
- **Multi-OS** — Ubuntu, Debian, RHEL/Rocky/Alma, Flatcar, openSUSE, ...
- **Choice of runtime & CNI** — containerd (default) / CRI-O · Calico (default) / Cilium / Flannel / Kube-OVN.
- **Day-2 playbooks** — `scale.yml`, `upgrade-cluster.yml`, backup & recovery, `reset.yml`, air-gapped.

One Ansible codebase covers the **full lifecycle**: install + scale + upgrade + recover.

Bridge: Kubespray Uses kubeadm Under the Hood

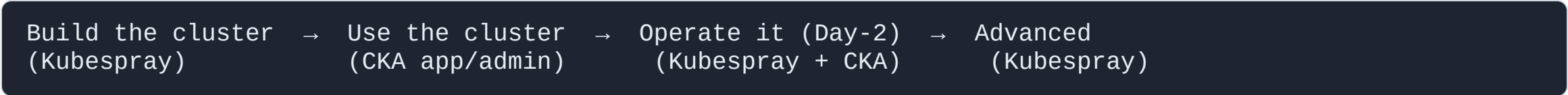
Choosing Kubespray does **not** mean throwing away kubeadm knowledge.

- Since **v2.3**, Kubespray calls **kubeadm** **internally** to create the control plane and join nodes.
- You get **kubeadm's correctness** with **Ansible's fleet management**.
- So every kubeadm concept still maps — **init** / **join**, the control-plane components, the control-plane taint, version-skew rules — and shows up again in later lessons.

Kubespray automates *what you'd otherwise script around kubeadm by hand*.

0.4 How the Course Is Structured

The Four Phases



Phase	What you do	Lessons
Build	requirements → inventory → group_vars → runtime/CNI → <code>cluster.yml</code> → HA	02–07
Use	workloads, storage, access, config, scheduling, networking, security, CRDs	08–15
Operate	maintenance & scaling, upgrades, backup/recovery, add-ons, troubleshooting	16–20
Advanced	hardening & offline, cloud & OpenStack, exam prep & wrap-up	21–23

Course Roadmap (1/2) — Build & Use

#	Lesson	Phase
01	Kubernetes Architecture	Build
02	Requirements & Environment	Build
03	Installation & Inventory (Kubespray)	Build
04	Cluster Configuration (group_vars)	Build
05	Container Runtimes & CNI	Build
06	Deploying the Cluster (Day-1)	Build
07	High Availability	Build
08–11	Apps · Storage · Access · Config & Quotas	Use

Course Roadmap (2/2) — Use, Operate & Advanced

#	Lesson	Phase
12–15	Scheduling · Networking · Security · CRDs & Operators	Use
16	Node Maintenance & Scaling	Operate
17	Upgrades	Operate
18	Backup, Recovery & Reset	Operate
19	Add-ons & Packaging (Helm / Kustomize)	Operate
20	Logging, Monitoring & Troubleshooting	Operate
21	Hardening & Offline	Advanced
22	Cloud & OpenStack	Advanced
23	Exam Preparation & Wrap-up	Advanced

0.5 Who This Is For

Who This Is For

Audience

- Sysadmins, DevOps and platform engineers running or planning to run Kubernetes
- Anyone targeting the **CKA** certification

Prerequisites

- Comfortable on the Linux command line
- Basic containers (containerd/Docker, images, registries) and **YAML**
- Networking fundamentals (IP, DNS, ports); a little **Ansible** helps for the build

No prior Kubernetes experience required — we start from the architecture.

What You'll Be Able to Do

By the end of the formation you can:

- **Build** a production-grade, **HA** cluster with Kubespray — bare metal or cloud (incl. **OpenStack**)
- **Deploy & expose** applications; manage **storage, scheduling, config, RBAC**
- Run **Day-2 operations** — scale, upgrade, back up, recover, troubleshoot
- Operate with the **coverage of the CKA exam** and the **repeatability** of Ansible automation

Build it, run it, keep it running — that's the whole journey.

Key Takeaways

- **Kubernetes** = the production container platform: **control plane** (brain) + **workers** (muscle), driven by **declarative reconciliation**.
- This course is **one path**: **build with Kubespray**, **operate with CKA-scope admin skills** — install to Day-2.
- **Kubespray** wins for production install: **repeatable, HA, multi-OS, CNI/runtime choice, Day-2 playbooks** — and it **uses kubeadm under the hood**.
- The journey runs in **four phases** — **Build** → **Use** → **Operate** → **Advanced** — across **24 lessons**.
- No K8s experience needed — we start at the **architecture**.

End of Lesson 0

Next: Lesson 1 — Kubernetes Architecture

Build with Kubespray · run with `kubectl` · `kubeadm` underneath

Understanding Kubernetes Architecture

Kubernetes Production · Lesson 1

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Describe what Kubernetes is and the problems it solves
- Identify the **control plane** components and their roles
- Identify the **node (worker)** components and their roles
- Explain how a Pod request flows through the cluster
- Distinguish vanilla Kubernetes, distributions and managed offerings

Reference: *Kubernetes Components* — kubernetes.io/docs/concepts/overview/components

1.1 What Is Kubernetes?

What Is Kubernetes?

Kubernetes (K8s) is an open-source platform for **automating deployment, scaling, and management** of containerized applications.

It provides:

- **Service discovery & load balancing** — stable names and IPs for Pods
- **Storage orchestration** — mount local or cloud storage automatically
- **Automated rollouts & rollbacks** — declarative desired state
- **Self-healing** — restarts, reschedules, and replaces failed containers
- **Secret & configuration management** — without rebuilding images

Kubernetes is *declarative*: you describe the **desired state**, controllers continuously reconcile the **actual state** to match it.

The Cluster: Two Planes

A Kubernetes **cluster** is a set of machines (**nodes**) organized into:

Plane	Runs	Hosts user Pods?
Control plane	Cluster "brain": API, scheduling, controllers, etcd	No (by default)
Worker plane	User workloads via kubelet + runtime	Yes

- A production cluster runs **multiple control-plane nodes** for high availability.
- The control plane makes **global decisions**; nodes **run the work**.

1.2 Control Plane Components

Control Plane Components

Component	Role
kube-apiserver	Exposes the Kubernetes HTTP API; front door to the cluster
etcd	Consistent, highly-available key-value store for all cluster data
kube-scheduler	Assigns unscheduled Pods to suitable nodes
kube-controller-manager	Runs controllers that drive actual → desired state
cloud-controller-manager	Integrates with a cloud provider (<i>optional</i>)

Everything talks to the cluster **through the apiserver** — it is the only component that reads/writes etcd directly.

kube-apiserver & etcd

kube-apiserver

- The RESTful front end for the cluster's shared state
- Validates and processes every request (`kubectl`, controllers, kubelets)
- Horizontally scalable — run several instances behind a load balancer

etcd

- Stores the entire cluster state and configuration
- Strongly consistent (Raft); the **single source of truth**
- **Back it up** — losing etcd means losing the cluster

CKA tip: you must know how to **snapshot and restore etcd** (Lesson 7).

Scheduler & Controller Manager

kube-scheduler

- Watches for Pods with no assigned node
- Picks a node based on resource requests, affinity, taints, constraints

kube-controller-manager

- A single binary running many controllers, e.g.:
 - **Node controller** — notices and responds to node failures
 - **ReplicaSet / Deployment controllers** — maintain replica counts
 - **Job controller** — runs Pods to completion
- Each controller watches the API and reconciles toward desired state

1.3 Node Components

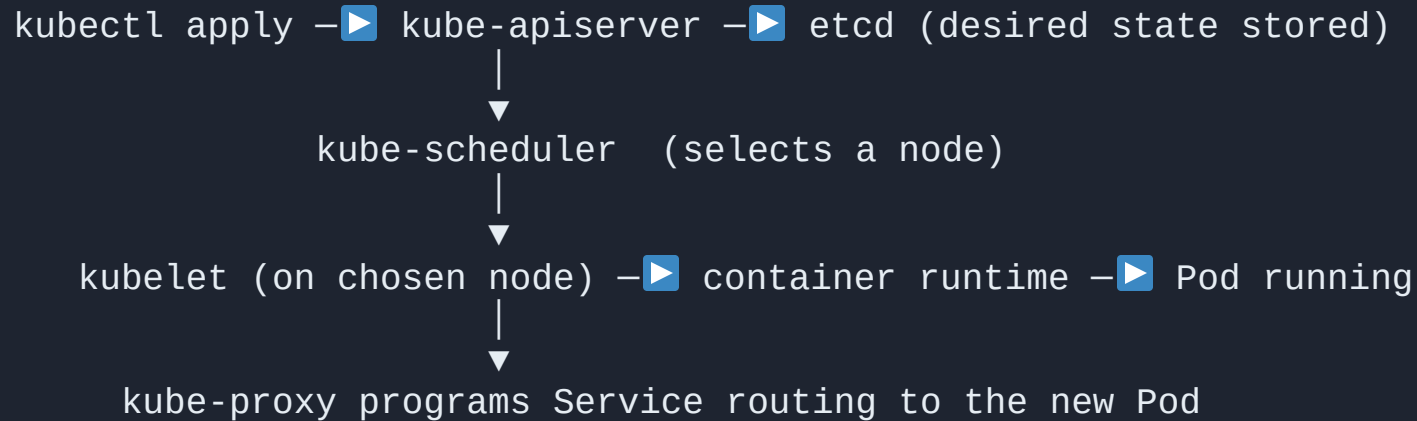
Node (Worker) Components

Component	Role
kubelet	Agent on every node; ensures containers described by PodSpecs are running and healthy
kube-proxy	Maintains network rules so Services reach the right Pods <i>(optional)</i>
Container runtime	Software that actually runs containers (containerd, CRI-O)

Every node also runs a **CNI plugin** (Calico, Cilium, Flannel...) to provide the Pod network.

The kubelet only manages containers created by Kubernetes — not arbitrary ones on the host.

How a Pod Gets Scheduled



1. You submit a manifest → apiserver persists it in etcd
2. Scheduler binds the Pod to a node
3. That node's kubelet tells the runtime to start the containers
4. kube-proxy + CNI wire up networking

Cluster Add-ons

Add-ons extend cluster functionality using Kubernetes resources (often in the `kube-system` namespace):

- **DNS** (CoreDNS) — cluster-wide name resolution (*essentially mandatory*)
- **Web UI (Dashboard)** — browser-based management
- **Container Resource Monitoring** — metrics collection (e.g. metrics-server)
- **Cluster-level Logging** — central log storage

Vanilla Kubernetes ships **no** default CNI or Ingress controller — you choose and install them.

1.4 Vanilla, Distributions & Managed

Vanilla, Distributions & Managed K8s

Flavor	What it is	Examples
Vanilla	Upstream Kubernetes installed directly (e.g. <code>kubeadm</code>)	kubeadm, Kubespray
Distribution	Vanilla + ecosystem add-ons + vendor support, often a version behind	OpenShift, Rancher, Charmed K8s
Managed	Cloud provider runs the control plane for you	EKS, AKS, GKE
Learning	All-in-one, single host	minikube, kind, k3s

- The **CKA exam** uses vanilla, `kubeadm` -built clusters.
- Distributions trade some flexibility for integration and support.

Key Takeaways

- Kubernetes = **declarative** orchestration of containers; controllers reconcile desired vs actual state
- **Control plane**: apiserver, etcd, scheduler, controller-manager (+ optional cloud-controller-manager)
- **Node**: kubelet, kube-proxy, container runtime, CNI
- **All traffic flows through the apiserver**; only it touches etcd
- Know the difference between **vanilla / distribution / managed**

End of Lesson 1

Next: Lesson 2 — Requirements & Environment

```
kubectl get nodes · kubectl get componentstatuses · kubectl get pods -n kube-system
```


Requirements & Environment

Kubernetes Production · Lesson 2

What to prepare **before** running Kubespray

kubespray.io · kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Distinguish the **Ansible control node** from the **target nodes**
- Provision a control node: **Python**, **ansible-core**, **netaddr**, a **venv**
- Meet target-node prerequisites: **OS**, **CPU/RAM**, **swap off**, **IP forwarding**
- Load the right **kernel modules** (**overlay** , **br_netfilter**) and **sysctls**
- Ensure each node has a **unique** hostname, MAC and product_uuid
- Open the **required ports** for the control plane, workers and the **CNI**

Reference: Kubespray *Getting started + port/kernel requirements*, and the *kubeadm host prerequisites* — these are merged here into one prep checklist.

2.1 Control Node vs Target Nodes

Two Sides to Every Deployment

A Kubespray install always has **two distinct sides**:

Side	Role
Control node	Your workstation / bastion that runs Ansible . It is not part of the cluster — it only needs Python + SSH to every target
Target nodes	The future control-plane , etcd and worker machines that become the cluster

- Kubespray drives the targets with **Ansible over SSH** — it performs the same host prep a manual `kubeadm` install needs (swap off, modules, sysctls, ports).
- Most of that prep is **automated** by Kubespray, with one big exception...

Firewalls are not managed by Kubespray — you open the ports yourself.
Everything in this lesson is what you put in place **before running Kubespray**.

2.2 The Ansible Control Node

Ansible Control-Node Requirements

The control node runs Ansible against the cluster; it needs only Python, the right Ansible, and `netaddr`.

Requirement	Value (per docs)
Ansible	v2.14+ (README); current tree pins ansible-core ≥2.18.0, <2.19.0
Python (control node)	3.11–3.13 for Ansible 2.18.x
Jinja	2.11+
Python module	<code>netaddr</code> (<code>python-netaddr</code>) on the machine running Ansible
Install method	Python venv + <code>pip install -r requirements.txt</code>

Using `requirements.txt` pins the **exact** ansible-core Kubespray expects and pulls in Jinja and `netaddr` — don't install Ansible from your distro packages.

Install Ansible in a venv

Create an isolated environment and install exactly what Kubespray pins:

```
VENVDIR=kubespray-venv
KUBESPRAYDIR=kubespray
python3 -m venv $VENVDIR
source $VENVDIR/bin/activate
cd $KUBESPRAYDIR
pip install -r requirements.txt # Ansible 2.18.x, jinja, netaddr
```

Verify before going further:

```
ansible --version # ansible-core within 2.18.x
python3 --version # 3.11 - 3.13 for Ansible 2.18.x
```

A venv keeps Kubespray's pinned Ansible away from system Python — re- **source** the **activate** script in every new shell before running playbooks.

2.3 Target-Node Requirements

Target-Node Requirements

Every future cluster machine — control-plane, etcd, worker — must satisfy:

Requirement	Value (per docs)
Supported OS	Ubuntu, Debian, RHEL/Rocky/Alma, openSUSE, Flatcar, Fedora/CoreOS, Amazon Linux 2, Oracle Linux, Kylin, openEuler...
Memory — control plane	2 GB or more
Memory — worker node	1 GB or more
CPU — control plane	2 CPUs or more
Swap	Disabled
IPv4 forwarding	Enabled (<code>net.ipv4.ip_forward</code>); IPv6 too if used
Network	Full connectivity + Internet/proxy to pull images
Unique IDs	hostname, MAC, product_uuid

Unique Node Identifiers

Every node must have a **unique** hostname, MAC address and **product_uuid** — cloned VMs often share these, which **breaks node registration**.

```
ip link                # MAC address per interface
sudo cat /sys/class/dmi/id/product_uuid # product UUID
hostnamectl            # hostname
```

- Two nodes with the same product_uuid or MAC may be treated as **one** node.
- Always check this **after cloning** a VM template — regenerate as needed.

Kubespray keys the inventory on **inventory_hostname**; duplicate machine identities still cause kubelet registration conflicts at the cluster level.

Disabling Swap & IP Forwarding

Kubernetes requires **swap off**; the kubelet refuses to start otherwise.

Kubespray turns it off, but you can do it by hand:

```
sudo swapoff -a
# Comment any swap line in /etc/fstab so it stays off after reboot
```

Enable IPv4 forwarding on every target (also set in the sysctls below):

```
echo 'net.ipv4.ip_forward = 1' | sudo tee /etc/sysctl.d/99-k8s.conf
sudo sysctl --system
```

Pod and Service traffic is **routed** through each node, so the kernel must be allowed to forward packets — enable IPv6 forwarding too if you run dual-stack.

2.4 Kernel: Version, Modules & sysctls

Kernel Version Requirements

Item	Detail
Kubernetes ≥ 1.32.0	Recommended LTS kernel 4.19 (4.x); any 5.x / 6.x also fine
cgroups v2	Minimum kernel 4.15 , recommended 5.8+
Old EL / Amazon	RHEL 8 / Alma 8 / Rocky 8 / Oracle 8 / Amazon Linux 2 ship kernels below 4.19
Below requirement?	Bypass the preflight check (group_vars):

```
kubeadm_ignore_preflight_errors:  
- SystemVerification
```

Prefer a kernel that **meets** the requirement; only use the ignore-errors escape hatch for stuck-on-old-EL hosts you cannot upgrade.

Kernel Modules & sysctls

Two modules must be loaded so the bridge passes traffic through iptables:

Module	Why
overlay	OverlayFS storage driver for the container runtime
br_netfilter	Bridge traffic visible to iptables (Service/CNI rules)

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay && sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
```

Kubespray's preinstall/runtime roles load these modules and apply these sysctls **for you** — a manual kubeadm host needs them set by hand.

2.5 Required Network Ports

Required Ports — Control Plane

These ports must be **reachable** on each control-plane / etcd node:

Proto	Port	Component
TCP	22	SSH (Ansible)
TCP	6443	Kubernetes API server
TCP	2379	etcd client API
TCP	2380	etcd peer API
TCP	10250	kubelet API
TCP	10257	kube-controller-manager
TCP	10259	kube-scheduler

```
nc 127.0.0.1 6443 -zv -w 2      # test reachability of a port
```

etcd uses **two** ports — **2379** for clients (API server), **2380** for peer-to-peer replication between etcd members.

Required Ports — Workers

These ports must be reachable on each worker node:

Proto	Port	Component
TCP	22	SSH (Ansible)
TCP	10250	kubelet API
TCP	10256	kube-proxy
TCP/UDP	30000–32767	NodePort service range

- **10250** (kubelet) is on **every** node — control-plane and worker alike.
- The **30000–32767** range is where `type: NodePort` Services are exposed.

Workers do **not** open the API/etcd/controller/scheduler ports — those live only on control-plane nodes.

Required Ports — CNI-specific

Open these **only** for the CNI you actually deploy:

CNI	Proto	Port	Purpose
Calico	TCP	179	BGP
Calico	UDP	4789	VXLAN
Calico	TCP	5473	Typha
Calico	UDP	51820 / 51821	WireGuard (v4 / v6)
Cilium	UDP	8472	VXLAN overlay
Cilium	TCP	4240	health checks
Cilium	TCP	4244 / 4245	Hubble server / relay
Cilium	UDP	51871	WireGuard

The CNI is chosen later in the inventory (Lesson 3+). Note its ports now so the firewall is ready before the network add-on rolls out.

2.6 Firewall & Putting It Together

Kubespray Does Not Manage Your Firewall

"The firewalls are not managed — you'll need to implement your own rules."

- Kubespray automates **swap off**, **kernel modules**, **sysctls** and the **bootstrap-os** Python install — but it will **not** open ports for you.
- You must open, before deploying: **6443** (API), **2379/2380** (etcd), **10250** (kubelet), **10257/10259** (controller/scheduler), **10256** + **30000–32767** (kube-proxy / NodePort), plus the **CNI** ports.
- A blocked port shows up later as nodes failing to join or Pods that can't reach the API — far harder to debug than opening them up front.

Everything in this lesson is the **pre-flight checklist**. With the control node, target nodes, kernel and ports ready, the next lessons build the **inventory** and **run the install**.

Key Takeaways

- **Two sides**: the **control node** runs Ansible (not in the cluster); the **target nodes** become control-plane / etcd / workers
- **Control node** = Python **3.11–3.13**, **ansible-core 2.18.x**, **netaddr**, in a **venv** via `pip install -r requirements.txt`
- **Target nodes**: supported OS, **2 GB**+ RAM + **2 CPU** control-plane / **1 GB**+ worker, **swap off**, **IPv4/IPv6 forwarding on**, kernel ≥ 4.19
- Load **overlay** + **br_netfilter**; set **bridge-nf-call** + **ip_forward** sysctls (Kubespray automates this)
- Ensure unique **hostname / MAC / product_uuid** per node
- **Open the firewall yourself**: 6443, 2379/2380, 10250, 10257/10259, 10256, 30000–32767, plus CNI ports — these are the prereqs **before running Kubespray**

End of Lesson 2

Next: Lesson 3 — Installation & Inventory

```
python3 -m venv · pip install -r requirements.txt · swapoff -a · modprobe overlay br_netfilter · ports  
6443 / 2379-2380 / 10250 / 30000-32767
```

Installation & Inventory

Kubernetes Production · Lesson 3

Built from the official documentation

kubespray.io · kubernetes-sigs/kubespray

Learning Objectives

By the end of this lesson you will be able to:

- **Get the Kubespray code** and pin it to a release tag
- Install Ansible deps into a **Python virtualenv** with `requirements.txt`
- Create your own inventory from `inventory/sample`
- Build `hosts.yaml` with the **inventory builder** and read its groups
- **Verify connectivity** before deploying, and know the collection method

Everything happens on the **control node** — no agent runs on the targets.

3.1 Getting the Code

Clone + Check Out a Release Tag

On the control node, work inside a fresh Python virtualenv and **pin a tested release** — never deploy production off a moving `master`.

```
python3 -m venv venv
source venv/bin/activate

git clone https://github.com/kubernetes-sigs/kubespray.git
cd kubespray
git checkout release-2.17      # pin to a release tag/branch
```

- Ansible is a **Python application** — the venv isolates its deps.
- Kubespray ships **different requirements.txt per Ansible version**.

Your Python version limits the usable Ansible/Kubespray version (recent releases: Python 3.11-3.13, Ansible >=2.18,<2.19).

Install the Dependencies

Install into the **active** virtualenv:

```
pip install -r requirements.txt
```

- Pulls the exact Ansible version that release of Kubespray was tested with.
- Keeps system Python untouched — no global package conflicts.

If pip says it *cannot find* `ansible==X`, your Python is too old for this Kubespray tag — use a newer Python or an older release.

3.2 Creating Your Inventory

Copy the Sample Inventory

Copy `inventory/sample` to your own cluster dir (recursive, preserve attrs), then edit **only your copy** so upstream updates never clobber it.

```
cp -rfp inventory/sample inventory/mycluster
```

- Creates `inventory/mycluster/` with the inventory file **plus** the `group_vars/` tree (`group_vars/all/all.yml`, `group_vars/k8s_cluster/k8s-cluster.yml`, ...).
- `group_vars/` is where Lesson 3 configures runtime / CNI / add-ons.

Inventory may be **YAML, JSON, or INI** — pick whichever you prefer.

Defining Hosts in the Inventory

Edit the copied inventory and assign each node to its group(s):

```
<your-editor> inventory/mycluster/inventory.ini
```

```
[kube_control_plane]
```

```
node1  
node2
```

```
[etcd]
```

```
node1  
node2  
node3
```

```
[kube_node]
```

```
node3  
node4
```

The old `contrib/inventory_builder/inventory.py` helper was **removed** upstream — define the inventory **by hand** (INI, YAML or JSON) or with your own tooling.

3.3 Inventory Structure

The Inventory Groups

Group	Role
kube_node	nodes where the pods run (workers)
kube_control_plane	apiserver, scheduler, controller-manager
etcd	etcd members — at least 3 for failover
calico_rr	advanced Calico route-reflector cases
bastion	jump host when nodes have private IPs only

k8s_cluster is **dynamically defined** = kube_node ∪
kube_control_plane ∪ calico_rr — used for cluster-wide group_vars .

Membership Rules

- Server in **both** `kube_control_plane` + `kube_node` → schedulable control plane (control plane *and* worker).
- Server **only** in `kube_control_plane` → standalone, **unschedulable** control plane.
- Server in **both** `kube_node` + `etcd` → etcd schedulable for workloads; keep groups disjoint for **standalone etcd**.

Do **not** hand-edit the children of `k8s_cluster` — it is derived.

hosts.yaml — Per-Host Variables

```
all:
  hosts:
    node1:
      ansible_host: 95.54.0.12    # IP Ansible SSHes to
      ip: 10.3.0.1                # IP k8s services bind to
      access_ip: 10.3.0.1         # IP other nodes use to reach it
  children:
    kube_control_plane:
      hosts: { node1: }
    etcd:
      hosts: { node1: }
    kube_node:
      hosts: { node2: }
    k8s_cluster:
      children: { kube_control_plane:, kube_node: }
```

`ansible_host` = SSH target · `ip` = service bind · `access_ip` = node reach.

3.4 Verifying Connectivity

Ping Every Node First

Confirm Ansible can SSH **and** escalate to root on every host before the long deploy:

```
ansible -i inventory/mycluster/hosts.yaml all -m ping -b
```

- `all` = every host · `-m ping` = connectivity module · `-b` = become root.
- Connection vars: `ansible_user`, `ansible_become`,
`ansible_ssh_private_key_file` — or pass `-u <user>` / `--private-key=...`.

Privilege escalation (`-b` / `--become`) is required here **and** for the real deploy.

Deploying with `cluster.yml`

Same inventory drives the deploy (full coverage in Lesson 6) — `--become` is **mandatory**.

```
# YAML inventory from the builder
ansible-playbook -i inventory/mycluster/hosts.yaml \
  --become --become-user=root cluster.yml

# INI inventory with explicit user + key
ansible-playbook -i inventory/mycluster/inventory.ini \
  -u $USERNAME -b -v --private-key=~/.ssh/id_rsa cluster.yml
```

Without `--become` the playbook fails: it writes to `/etc`, installs packages, and manages systemd daemons.

3.5 Kubespray as a Collection

Consume Kubespray as a Collection

Reference Kubespray from your **own** Ansible repo instead of a clone.

```
# requirements.yml
collections:
  - name: https://github.com/kubernetes-sigs/kubespray
    type: git
    version: master    # use the tag/branch you need
```

```
ansible-galaxy install -r requirements.yml
```

```
# your playbook.yml
- name: Install Kubernetes
  ansible.builtin.import_playbook: kubernetes_sigs.kubespray.cluster
```

Then run `ansible-playbook -i INVENTORY --become --become-user=root PLAYBOOK`
— your inventory/group_vars stay in your repo.

Key Takeaways

- **Clone + pin a release tag**, then `pip install -r requirements.txt` inside a **Python venv** — never run off a moving `master`.
- **Copy** `inventory/sample` → `inventory/mycluster`; edit only your copy.
- Build `hosts.yaml` with the **inventory builder** or edit by hand; groups define topology and **`k8s_cluster` is the derived union**.
- Use **≥ 3 etcd**; `ansible_host` / `ip` / `access_ip` separate SSH, bind, reach.
- **Ping with** `all -m ping -b` before `cluster.yaml`; `--become` is required.

End of Lesson 3

Next: Lesson 4 — Cluster Configuration

`group_vars/all/all.yml` · `group_vars/k8s_cluster/k8s-cluster.yml`

Configuring a Cluster with group_vars

Kubernetes Production · Lesson 4

Built from the official documentation

`kubernetes-sigs/kubespray` · `docs/ansible/vars.md`

Learning Objectives

By the end of this lesson you will be able to:

- Locate cluster configuration in the **group_vars** directory layout
- Set the **key variables** that define a cluster (version, CNI, runtime, subnets)
- Reason about **Ansible variable precedence** — which value wins
- Configure the **DNS stack** (CoreDNS + nodelocaldns) and upstream DNS
- Configure **NTP**, an **HTTP(S) proxy**, and the **download / pre-pull** mechanism

Topology lives in the **inventory**; everything else lives in **group_vars**.

4.1 The group_vars Layout

Where Configuration Lives

Under `inventory/mycluster/group_vars/`, config is split by Ansible group:

Path	Applies to
<code>all/all.yml</code>	All hosts — proxy, NTP, downloads, LB, cloud
<code>all/*.yml</code>	All hosts — OS, etcd, OpenStack, offline (split files)
<code>k8s_cluster/k8s-cluster.yml</code>	<code>k8s_cluster</code> — version, CNI, runtime, subnets, DNS
<code>k8s_cluster/addons.yml</code>	<code>k8s_cluster</code> — dashboard, ingress, metrics, CSI

`k8s_cluster` = `kube_control_plane` + `kube_node` , so these files reach **every** cluster node.

The Two Files That Matter Most

- `group_vars/all/all.yml` — host-level, cross-cutting concerns that also apply to non-Kubernetes hosts (e.g. the load balancer): `proxy`, `NTP`, `downloads`, `cloud_provider`, `loadbalancer_apiserver`.
- `group_vars/k8s_cluster/k8s-cluster.yml` — the Kubernetes cluster itself: `kube_version`, `kube_network_plugin`, `container_manager`, the service/pod subnets, DNS and kubelet settings.

Kubespray ships **production defaults** — you override only what you need: often just version, CNI, a proxy, and DNS.

4.2 Key Cluster Variables

The Variables That Define a Cluster

Variable	Meaning	Default
kube_version	Kubernetes release	(per release)
kube_network_plugin	CNI: calico / cilium / flannel ...	calico
container_manager	CRI runtime	containerd
kube_service_addresses	Service ClusterIP CIDR	10.233.0.0/18
kube_pods_subnet	Pod IP CIDR	10.233.64.0/18
cluster_name / dns_domain	Cluster DNS zone	cluster.local
kubelet_max_pods	Max pods per node	(110)

k8s-cluster.yml — Core Settings

```
# inventory/mycluster/group_vars/k8s_cluster/k8s-cluster.yml
kube_version: v1.31.4
kube_network_plugin: calico      # cilium | flannel | kube-ovn ...
container_manager: containerd   # cri-o | docker
kube_proxy_mode: ipvs           # iptables | nftables

# Service ClusterIP and Pod CIDRs — MUST NOT overlap
kube_service_addresses: 10.233.0.0/18
kube_pods_subnet: 10.233.64.0/18
kube_network_node_prefix: 24     # per-node pod subnet size

cluster_name: cluster.local
kubelet_max_pods: 110
```

Subnet & Networking Rules

- `kube_service_addresses` and `kube_pods_subnet` **must not overlap** each other, and must not collide with your **node network**.
- `kube_network_node_prefix` (default `24`) sets the per-node pod block — `prefix × node-count` must fit inside `kube_pods_subnet`.
- Host addressing is **per host** in the inventory, not in `group_vars`:
`ip` = service-binding (public) IP, `access_ip` = inter-host (private) IP.
- Some aggregates (e.g. `kube_service_subnets`) are **computed** — set the base variables, not the aggregate.

Plan CIDRs once: changing them after deploy means rebuilding the cluster.

4.3 Variable Precedence

Which Value Wins?

Kubespray uses standard Ansible precedence — **lowest to highest**:

1. Kubespray `role/defaults` — built-in defaults
2. `group_vars/all/*`
3. `group_vars/k8s_cluster/*` — more specific group beats `all`
4. `host_vars` — a single host beats any group var
5. `-e extra-vars` on the CLI — **always wins**

```
# Override for one run without editing any file
ansible-playbook -i inventory/mycluster/inventory.ini \
  -e kube_version=v1.31.4 cluster.yml
```

More specific scope beats more general scope — set a cluster default in `k8s_cluster`, override per node in `host_vars`.

4.4 The DNS Stack

CoreDNS + nodelocaldns

- **CoreDNS** is the default cluster DNS — authoritative for `dns_domain` (`cluster.local`) and its subdomains.
- **nodelocaldns** is a per-node **caching agent**; pods query it locally, avoiding iptables DNAT to the central service IP.
- Enabled by **default since release 2.10** (`enable_nodelocaldns: true`).

Resolution path: **pod** → **nodelocaldns** (local cache) → **CoreDNS** for cluster names → **upstream DNS** for everything else.

dns_mode & Upstream DNS

```
# group_vars/k8s_cluster/k8s-cluster.yml
dns_mode: coredns          # coredns_dual | manual | none
enable_node_localdns: true  # per-node DNS cache (default since 2.10)
resolvconf_mode: host_resolvconf # default

# Resolvers CoreDNS forwards non-cluster queries to
upstream_dns_servers:
  - 1.1.1.1
  - 9.9.9.9
# Extra host nameservers (only with host_resolvconf), after upstream
nameservers:
  - 192.168.0.1
```

- `dns_mode`: `coredns` (default), `coredns_dual`, `manual`, `none`.
- Limits: max **6** search domains; nameservers capped at **3** (~2 custom).

4.5 NTP

Time Synchronization

"Time synchronization is important to Kubernetes and Etcd" — clock skew breaks

TLS certificate validation and **etcd** consensus.

```
# group_vars/all/all.yml
ntp_enabled: true           # install ntpd / chrony, enable at boot
ntp_manage_config: true    # let Kubespray write the config
ntp_servers:
  - "0.pool.ntp.org iburst"
  - "1.pool.ntp.org iburst"
ntp_timezone: Etc/UTC      # unset = leave unchanged
ntp_tinker_panic: true     # avoid large-drift panics in VMs
```

`ntp_manage_config: true` is required to apply `ntp_servers` — essential for **air-gapped** sites pointing at internal NTP.

4.6 HTTP(S) Proxy

Outbound Proxy & Auto no_proxy

```
# group_vars/all/all.yml
http_proxy: "http://proxy.corp.tld:3128"
https_proxy: "http://proxy.corp.tld:3128"
# no_proxy is AUTO-generated (all nodes + loadbalancer)
additional_no_proxy: ".corp.tld,10.0.0.0/8" # append, keep auto
no_proxy_exclude_workers: true # control-plane only
```

- Set only `http_proxy` / `https_proxy`; Kubespray builds `no_proxy` to exclude **all nodes and the loadbalancer**.
- Use `additional_no_proxy` to **append** — overriding `no_proxy` disables auto-generation.
- `no_proxy_exclude_workers: true` keeps only control-plane nodes, so scaling workers won't restart the runtime / bounce pods.

4.7 Downloads / Pre-Pull

Controlling Where Artifacts Come From

```
# group_vars/all/all.yml (or download.yml)
download_run_once: true      # download once, then push to all nodes
download_localhost: false   # true = Ansible node is the delegate
download_force_cache: false # provision from cache, no re-download
download_keep_remote_cache: false
download_cache_dir: /tmp/kubespray_cache
```

- Default: **each node downloads independently** (`download_run_once: false`).
- `download_run_once: true` → delegate is the **first kube_control_plane**; cache (~800 MB) lands in `download_cache_dir`.
- `download_localhost: true` → use the **Ansible control node** as delegate (nodes can't reach external registries); needs a local container runtime.

Key Takeaways

- **Topology = inventory; configuration = `group_vars`**. Edit YAML, run `cluster.yml`; re-edit + re-run for Day-2.
- Two files matter most: `all/all.yml` (proxy, NTP, downloads, cloud) and `k8s_cluster/k8s-cluster.yml` (version, CNI, runtime, subnets, DNS).
- A few variables define the cluster: `kube_version`, `kube_network_plugin`, `container_manager`, `kube_service_addresses`, `kube_pods_subnet`.
- **Precedence:** defaults < `all` < `k8s_cluster` < host_vars < `-e` extra-vars.
- Cross-cutting concerns are first-class: **DNS** (CoreDNS + nodelocaldns), **NTP**, **proxy** (auto `no_proxy`), **downloads** (`download_run_once`).

End of Lesson 4

Next: Lesson 5 — Container Runtimes & CNI

`container_manager: containerd · crio · docker`

Container Runtimes & CNI

Kubernetes Production · Lesson 5

Provisioning the data plane with Kubespray

kubespray.io · kubernetes-sigs/kubespray

Learning Objectives

By the end of this lesson you will be able to:

- Select the container runtime with `container_manager` (default `containerd`)
- Configure `containerd` — `SystemdCgroup`, registry mirrors — and **CRI-O**
- Layer **sandboxed runtimes** (gVisor, Kata) on containerd via **RuntimeClass**
- Select the CNI with `kube_network_plugin` (default `calico`)
- Set the **pod / service CIDRs** and `kube_proxy_mode`
- Compare **Calico, Cilium and Flannel** as provisioning choices

Reference: *Kubespray docs — CRI/ and CNI/* — kubespray.io

Provisioning vs the Network Model

This lesson is the **install-time** decision: *which* runtime and *which* CNI you wire into `group_vars` **before** deploying the cluster.

This lesson (Lesson 5)	Comes later (Lesson 13)
<code>container_manager</code> — pick the runtime	How the runtime runs your Pods
<code>kube_network_plugin</code> — pick the CNI	The pod-to-pod / service network model
Pod & service CIDRs , <code>kube_proxy_mode</code>	NetworkPolicy enforcement
Encapsulation / BGP backend	DNS (CoreDNS) resolution

Calico and Cilium *can* enforce policy — but **NetworkPolicy and DNS** are a workload concern covered in **Lesson 13**. Here we only choose and wire them.

5.1 Container Runtimes (CRI)

container_manager — the CRI Switch

One variable decides which **Container Runtime Interface** the kubelet talks to.

```
# group_vars/k8s_cluster/k8s-cluster.yml
container_manager: containerd # default; or: criol
```

Value	Runtime
containerd	Default — industry-standard, runs runc by default
criol	Lightweight CRI implementation (extra all.yml keys needed)

gVisor and **Kata** are **not** values here — they are extra runtimes layered on **containerd** and chosen **per-Pod** via RuntimeClass.

containerd — Default + SystemdCgroup

containerd is the default; it runs containers with the `runc` OCI runtime, configured via the `containerd_runc_runtime` dictionary.

```
containerd_runc_runtime:  
  name: runc  
  type: "io.containerd.runc.v2"  
  options:  
    Root: ""  
    SystemdCgroup: "false"  
    BinaryName: /usr/local/bin/my-runc  
  base_runtime_spec: cri-base.json
```

- `SystemdCgroup` controls systemd cgroup integration — it **must match the kubelet's cgroup driver**.
- Extra runtimes via `containerd_additional_runtimes` ;
`containerd_default_runtime` changes which one is default.

With containerd, use `etcd_deployment_type: host` .

containerd — Registry Mirrors

Mirrors are a **list** under `containerd_registries_mirrors`. There is **no separate insecure key** — an HTTP registry is a mirror with `skip_verify: true`.

```
containerd_registries_mirrors:
- prefix: docker.io
  mirrors:
    - host: https://mirror.gcr.io
      capabilities: ["pull", "resolve"]
      skip_verify: false
  server: https://registry-1.docker.io
- prefix: 172.19.16.11:5000          # private / insecure registry
  mirrors:
    - host: http://172.19.16.11:5000
      capabilities: ["pull", "resolve"]
      skip_verify: true
```

The old `containerd_registries` / `containerd_insecure_registries` variables are **deprecated** — use `containerd_registries_mirrors`.

CRI-O — Enabling & Registries

Set `container_manager: crio` plus a few required keys in `all/all.yml`:

```
container_manager: crio
download_container: false
skip_downloads: false
etcd_deployment_type: host    # or kubeadm
```

```
crio_registries:
- prefix: docker.io
  location: registry-1.docker.io
  mirrors:
  - location: 192.168.100.100:5000
    insecure: true
```

- `crio_insecure_registries` — list of HTTP registries.
- `crio_registry_auth` — per-registry `registry` / `username` / `password`.
- `crio_runtimes` — add runtimes (e.g. user namespaces with `crio_remap_enable`).

gVisor & Kata — Layered on containerd

Both need "a container manager compatible with selecting the RuntimeClass such as **containerd**." Enable them *in addition* to containerd.

```
container_manager: containerd
gvisor_enabled: true           # gVisor — runsc application kernel
kata_containers_enabled: true  # Kata — lightweight micro-VMs (QEMU)
etcd_deployment_type: host

# Kata Pod Overhead (accounts for the VM's resources)
kubelet_cgroup_driver: cgroupfs # or 'systemd' with cgroups v2
kata_containers_qemu_overhead: true
kata_containers_qemu_overhead_fixed_cpu: 10m
kata_containers_qemu_overhead_fixed_memory: 290Mi
```

- **gVisor** — Go application kernel; ships the **runsc** OCI runtime → extra isolation. RuntimeClass name **gvisor** (handler **runsc**).
- **Kata** — each Pod in its own micro-VM. RuntimeClass **kata-qemu** (auto-generated).

Per-Workload RuntimeClass

With sandboxed runtimes enabled, every Pod still uses `runc` unless it **opts in** by name with `runtimeClassName`.

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  runtimeClassName: gvisor      # or: kata-qemu
  containers:
  - name: nginx
    image: nginx:1.14.2
```

runtimeClassName	Result
(omitted)	Default <code>runc</code> — standard container
gvisor	gVisor sandbox (handler <code>runsc</code>)
kata-qemu	Kata micro-VM isolation

5.2 Networking & CNI

kube_network_plugin — the CNI Switch

One variable picks the CNI. Kubespray installs and configures it for you.

```
# group_vars/k8s_cluster/k8s-cluster.yml
kube_network_plugin: calico # default
```

Value	Plugin
calico	Default — mature, NetworkPolicy, BGP
cilium	eBPF datapath, kube-proxy replacement
flannel	Simple VXLAN overlay
kube-ovn · kube-router	OVN/OVS SDN · LVS/IPVS + BGP
multus	Meta-plugin — extra interfaces beside a primary CNI
macvlan · cni · cloud	L2 · generic (you supply config) · provider routing

cni only unpacks plugin binaries (cni_version) into /opt/cni/bin — you must supply /etc/cni/net.d yourself.

Pod & Service CIDRs

Plugin-independent ranges that define the cluster network — pick them **before** deploying and ensure they **do not overlap** existing infra.

Variable	Default	Meaning
kube_service_addresses	10.233.0.0/18	Service ClusterIP range
kube_pods_subnet	10.233.64.0/18	Pod IP range
kube_network_node_prefix	24	Per-node pod subnet carved from the pod CIDR
kube_proxy_mode	ipvs	ipvs , iptables , or nftables

- Each node gets a /24 slice of kube_pods_subnet for its Pods.

Some plugins (Cilium / eBPF) **replace kube-proxy** entirely — the kube_proxy_mode plumbing is then bypassed.

5.3 Calico (the default CNI)

Calico — Datastore & Backend

The most mature, feature-rich option; `kube_network_plugin: calico`. Configured in the sample `k8s-net-calico.yml`.

```
calico_datastore: "kdd"           # Kubernetes datastore (default); or 'etcd'
calico_network_backend: vxlan      # default; 'bird' enables BGP; or 'none'
calico_vxlan_mode: Always         # default — VXLAN is the more mature impl
calico_ipip_mode: Never           # default; needs calico_network_backend: bird
```

- **kdd + > 50 nodes** → enable the **Typha** daemon to offload the API server.
- Encapsulation is set **independently per overlay** (`Always` / `CrossSubnet` / `Never`); VXLAN is on by default, IPIP requires the `bird` backend.

Calico — BGP & Route Reflectors

For routed, **encap-free** fabrics, peer Calico with your datacenter routers.

```
peer_with_router: true           # peer with ToR / border routers
global_as_num: "64512"          # cluster BGP AS number (per-node local_as)
nat_outgoing: true              # SNAT pod traffic leaving the pod CIDR
calico_advertise_cluster_ips: true # advertise Service IPs over BGP
```

- **Route reflectors** — for large fabrics, dedicate nodes to the `calico_rr` inventory group (`cluster_id`, `calico_rr_id`, `calico_group_id`) so nodes peer with reflectors instead of a **full-mesh BGP**.

Calico is also a full **policy engine** (NetworkPolicy + `GlobalNetworkPolicy`) — but *enforcing* policy is a **Lesson 13** topic.

5.4 Cilium, Flannel & Others

Cilium — eBPF Datapath

Modern, eBPF-based plugin; `kube_network_plugin: cilium`.

```
cilium_kube_proxy_replacement: true    # eliminate kube-proxy
cilium_enable_hubble: true            # observability
cilium_hubble_install: true           # relay + UI
cilium_enable_bgp_control_plane: true # modern BGP control plane
cilium_encryption_enabled: true
cilium_encryption_type: wireguard      # or 'ipsec'
```

- **kube-proxy replacement** — on Cilium < v1.16 use `strict`; Kubespray converts `strict` → `true`. Needs the kube-apiserver address for all components.
- **Hubble** gives flow-level observability (DNS, drops, HTTP) and a UI.

Flannel — Simple VXLAN

Minimal overlay; `kube_network_plugin: flannel`.

- Backends: `vxlan` (default), `host-gw`, `wireguard`.
- Writes `/run/flannel/subnet.env` (`FLANNEL_NETWORK`, `FLANNEL_SUBNET`, ...).
- **No NetworkPolicy** — Flannel provides **pod connectivity only**; use Calico/Cilium if you need policy enforcement.

Known issue: a VXLAN-backend bug needs the workaround

```
ethtool --offload flannel.1 rx off tx off
```

 until a fixed version ships.

Kube-OVN · kube-router · Multus · generic

Plugin	What it offers
<code>kube-ovn</code>	OVN/OVS SDN — per-namespace subnets, VPCs, QoS
<code>kube-router</code>	Lightweight — LVS/IPVS services, iptables policy, BGP
<code>multus</code>	Meta-plugin — attach extra interfaces beside a primary CNI
<code>macvlan</code>	Direct L2 attachment
<code>cloud</code>	Let the cloud provider handle routing
<code>cni</code>	Generic — Kubespray drops binaries, you supply <code>/etc/cni/net.d</code>

Multus is layered *on top of* a primary CNI (e.g. Calico) — useful for Pods that need a second NIC (storage, SR-IOV, telco data planes).

How to Pick a CNI

If you need...	Choose
Default, mature, NetworkPolicy, BGP to ToR, large scale	Calico
eBPF datapath, kube-proxy replacement, L7 policy, Hubble	Cilium
Simplest possible overlay, no policy needed	Flannel
OVN-style SDN (per-namespace subnets, VPC)	Kube-OVN
Multiple interfaces per Pod	Multus + a primary CNI
You supply your own CNI config	cni (generic)

Default to **Calico** unless a specific need (eBPF, OVN, minimalism) points elsewhere. The choice is one line in `group_vars` ; the **CIDRs** are the real decision.

Key Takeaways

- `container_manager` = the runtime switch: `containerd` (default) or `crio`
- containerd: `SystemdCgroup` must match the kubelet; mirrors via `containerd_registries_mirrors` (insecure = `skip_verify: true`)
- **gVisor** (`gvisor` / `runsc`) & **Kata** (`kata-qemu`) layer on containerd, chosen **per-Pod** via **RuntimeClass**
- `kube_network_plugin` = the CNI switch (default `calico`)
- CIDRs: services `10.233.0.0/18` , pods `10.233.64.0/18` , node prefix `24` ;
`kube_proxy_mode` = `ipvs` / `iptables` / `nftables`
- **Calico** = VXLAN + BGP/route-reflectors · **Cilium** = eBPF + Hubble ·
Flannel = simple overlay, **no policy**
- This is the **provisioning** choice — **NetworkPolicy & DNS** are **Lesson 13**

End of Lesson 5

Next: Lesson 6 — Deploying the Cluster

```
container_manager: containerd · kube_network_plugin: calico · runtimeClassName: gvisor|kata-qemu ·  
CIDRs 10.233.0.0/18 / 10.233.64.0/18  
</content>
```

Deploying the Cluster (Day-1)

Kubernetes Production · Lesson 6

Bringing up a production cluster with **Kubespray**

`kubespray.io` · `kubernetes-sigs/kubespray`

Learning Objectives

By the end of this lesson you will be able to:

- Run the **Day-1 deploy** with `ansible-playbook ... -b cluster.yml`
- Explain why `-b / --become` is required and what **idempotency** buys you
- Describe what `cluster.yml` does **end-to-end** (preflight → add-ons)
- Add API SANs with `supplementary_addresses_in_ssl_keys`
- Understand that **Kubespray runs kubeadm** under the hood — the bridge
- **Retrieve access** (`kubeconfig_localhost` , `artifacts_dir` , `admin.conf`)
- **Verify** the cluster and use `--limit` for partial runs

This lesson **replaces** the manual `kubeadm` deploy from the CKA course — same cluster, driven declaratively.

6.1 The Deploy Command

One Command Brings Up the Cluster

The whole Day-1 bring-up is a **single, idempotent** `ansible-playbook` run against the inventory built in Lesson 2:

```
ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yml
```

- **-b / --become** = privilege escalation (sudo → root). **Required** — without it the run fails on the first task (cannot install packages, write to `/etc/kubernetes`, restart the kubelet).
- A full first run takes **~20 minutes** (longer on slow networks / many nodes).
- **Idempotent**: re-running **converges** the cluster to the declared state — it does **not** rebuild from scratch, so it is safe to retry.

CKA tip: know this one-liner — and *why* **-b** is mandatory — cold.

The Flags That Matter

```
ansible-playbook -i inventory/mycluster/hosts.yaml -b \  
-u $USERNAME --private-key=~/.ssh/id_rsa cluster.yml
```

Flag	Meaning
-i <inventory>	Inventory with node definitions & groups (<code>hosts.yaml</code>)
-b / --become	become — sudo to root. Required
-u \$USERNAME	SSH login user on the targets
--private-key=...	SSH private key to reach the nodes
-v / -vv / -vvv	Verbose — increase to debug failing tasks
--limit <host group>	Restrict the run to a subset of hosts
--extra-vars "k=v"	Override variables on the command line

The playbook may be written `cluster.yml` or `playbooks/cluster.yml` ; the inventory `hosts.yaml` or `inventory.ini` — same playbook, same flags.

6.2 What `cluster.yml` Does

The Deploy Pipeline (End-to-End)

The single `cluster.yml` playbook orchestrates the whole bring-up, in order:

1. **Preflight** — validate inventory, variables, host reachability; fail fast
2. **bootstrap-os** — OS prep on every node: packages, Python, kernel modules, sysctls, **disable swap**
3. **Install runtime + binaries** — container runtime (containerd) plus `kubeadm` + `kubelet` + `kubectrl`
4. **Run kubeadm** — `kubeadm init` the first control plane, `kubeadm join` the rest (etcd clustered on the `etcd` group)
5. **CNI** — deploy the network plugin (**Calico** by default)
6. **Add-ons** — optional components from `group_vars` (metrics-server, ingress...)

Same outcome as the manual CKA path — just one declarative pass instead of per-node imperative steps.

API SANs: `supplementary_addresses_in_ssl_keys`

To reach the API server through an address **not** already known — an external IP, a load-balancer VIP, or a DNS name — add it to the cert SANs **before** deploying, or `kubectl` from outside fails TLS verification:

```
# group_vars/k8s_cluster/k8s-cluster.yml
supplementary_addresses_in_ssl_keys:
  - 197.140.32.10      # external VIP / load balancer
  - kube.example.com  # DNS name
```

- This is the declarative equivalent of kubeadm's `--apiserver-cert-extra-sans` — **same X.509 SANs**.
- Adding a SAN **after** the cluster is up means **re-issuing certificates** — set it up front.

6.3 Under the Hood: Kubespray Runs kubeadm

Kubespray Drives — kubeadm Is Still the Engine

Kubespray does **not** reinvent the bootstrap. Step 4 of `cluster.yml` calls `kubeadm init` on the first control-plane node and `kubeadm join` on every other node — so the same ordered **kubeadm phases** run *inside* the playbook:

```
preflight → certs → kubeconfig → kubelet-start → control-plane  
→ etcd → upload-config → mark-control-plane → bootstrap-token → addons
```

- Certs, the **control-plane taint**, bootstrap **tokens**, the `/etc/kubernetes` layout — **all identical** to the manual path.
- Your **kubeadm knowledge still applies**, and the **CKA exam may ask kubeadm directly**.

Kubespray is the **driver**; kubeadm is the **engine**. You just drive it declaratively.

Manual kubeadm vs Kubespray

Manual CKA (the old Lesson 2)	Kubespray (this lesson)
Run <code>kubeadm init</code> / <code>kubeadm join</code> by hand	The playbook runs them for you
<code>kubectl apply</code> a CNI manifest	<code>cluster.yml</code> installs the CNI (Calico)
Copy <code>admin.conf</code> yourself	<code>kubeconfig_localhost: true</code> fetches it
Per-node, imperative, error-prone	Declarative, idempotent , repeatable

Same engine, same artifacts — different ergonomics. Learn kubeadm for the exam; use Kubespray in production.

6.4 Retrieving Access (kubeconfig)

Let Kubespray Fetch It

The admin kubeconfig lives at `/etc/kubernetes/admin.conf` on the control plane.

Easiest path — set these **before** the run:

```
# group_vars/k8s_cluster/k8s-cluster.yml
kubeconfig_localhost: true    # copy admin.conf into the artifacts dir
kubectl_localhost: true      # also drop a kubectl binary + kubectl.sh wrapper
```

After deploy, files appear under `artifacts_dir` (default `inventory/mycluster/artifacts/`):

```
cd inventory/mycluster/artifacts
export KUBECONFIG=$PWD/admin.conf
./kubectl.sh get nodes
```

Or Copy `admin.conf` Manually

Take ownership on the controller, copy it, and point the `server:` URL at the **reachable** controller address:

```
ssh $USERNAME@$IP_CONTROLLER_0 \  
  "sudo chown $USERNAME:$USERNAME /etc/kubernetes/admin.conf"  
  
scp $USERNAME@$IP_CONTROLLER_0:/etc/kubernetes/admin.conf ./admin.conf  
# edit the 'server:' field → external/reachable controller IP, then:  
export KUBECONFIG=$PWD/admin.conf
```

If `server:` points at an address **not** in the cert SANs, you get a TLS error — exactly what `supplementary_addresses_in_ssl_keys` prevents.

6.5 Verifying the Cluster

Verify the Cluster

```
kubectl get nodes           # every node should be Ready
kubectl get pods -A         # CNI, CoreDNS, kube-proxy... all Running
kubectl get pods -n kube-system
kubectl top nodes           # works once metrics-server add-on is enabled
```

- A node sits **NotReady** until the **CNI** pods are up — same as the manual kubeadm path.
- `cluster.yml` installs the CNI for you, so after a clean run **all nodes should be Ready**.

CKA tip: if nodes are NotReady, check `kubectl get pods -n kube-system` — the network plugin pods are the usual suspect.

6.6 Partial Runs with `--limit`

Scope a Run with `--limit`

Use `--limit` to target a subset of hosts — retry a single node that failed, or act on one group without touching the rest:

```
# retry just one node after a transient failure
ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yaml \
  --limit node4

# act on a whole group
ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yaml \
  --limit kube_node
```

Because the playbook is **idempotent**, a limited re-run only fixes what drifted on the targeted hosts — the rest of the cluster is untouched.

Key Takeaways

- Day-1 is **one idempotent command**: `ansible-playbook -i inventory/mycluster/hosts.yaml -b cluster.yaml` (~20 min)
- `-b / --become` is required — every task needs root
- `cluster.yaml` runs **preflight** → **bootstrap-os** → **runtime+kubeadm/kubelet/kubectl** → **kubeadm** → **CNI** → **add-ons**
- **Kubespray runs kubeadm** — the kubeadm **phases** happen inside the playbook, so kubeadm knowledge (and CKA questions) still apply
- Get access via `kubeconfig_localhost / kubectl_localhost` (`artifacts_dir`) or by copying `admin.conf` and fixing `server:`
- Add external/LB/DNS endpoints to `supplementary_addresses_in_ssl_keys` before deploying
- Verify with `kubectl get nodes` / `get pods -A`; use `--limit` to retry one node or group

End of Lesson 6

Next: Lesson 7 — High Availability

```
ansible-playbook -i ../hosts.yaml -b cluster.yml · --limit · kubeconfig_localhost: true · kubectl get  
nodes
```

High Availability with Kubespray

Kubernetes Production · Lesson 7

Built from the official documentation

`kubernetes-sigs/kubespray` · ha-mode · kube-vip · etcd · reliability

Learning Objectives

By the end of this lesson you will be able to:

- Explain what an **HA control plane** means in Kubespray
- Choose between **localhost LB**, **external LB** and **kube-vip** for the API
- Configure **kube-vip** in **ARP** or **BGP** mode for a control-plane VIP
- Pick an **etcd deployment type** and **stacked vs external** topology
- Understand and **tune node-failure detection** and failover speed

HA = make the two single-points-of-failure tiers — **etcd** and the **kube-apiserver** — resilient.

7.1 What "HA Control Plane" Means

Two Tiers Made Highly Available

Kubespray makes two critical components HA:

- **etcd cluster** — already **HA-ready with multiple peers**: every client receives the **complete peer list**, so no extra proxy is needed.
- **kube-apiserver** — needs a **reverse-proxy / load-balancer layer** in front of the apiserver instances to become HA.

Production rule of thumb: **≥ 3** control-plane / etcd nodes, **odd** count for etcd quorum (3 → survives 1 failure, 5 → survives 2).

7.2 API Server Load Balancing

Three Ways to Reach the API

Option	What it is
Localhost LB (default)	Per-node nginx reverse proxy on non-master nodes
External LB	One external VIP / domain all nodes point to
kube-vip	A virtual IP managed inside the cluster (ARP or BGP)

- etcd is HA by itself; this layer is **only about the apiserver**.
- Without localhost LB, use the **kube-vip role** or your **own load balancer**.

Workers and **kubelet** / **kube-proxy** must always have a stable apiserver endpoint.

Internal / localhost Load Balancer (Default)

- By default, non-master nodes run a **local nginx reverse proxy** that proxies API-server connections.
- Enabled by `loadbalancer_apiserver_localhost` — defaults to `True`.
- Only **kubelet** and **kube-proxy** on non-master nodes use the local proxy.
- Port `loadbalancer_apiserver_port` — defaults to `kube_apiserver_port`.

Less efficient than a dedicated LB (more health checks), but **practical** when no external LB infrastructure exists — hence the default.

External Load Balancer

Point all nodes at one external VIP + domain name:

```
apiserver_loadbalancer_domain_name: "my-apiserver-lb.example.com"
loadbalancer_apiserver:
  address: <VIP>
  port: 8383
```

- The domain name is added to `/etc/hosts` of every `k8s_cluster` host.
- It is also **wired into the generated self-signed TLS certificates**, so the cert is valid for the LB name.

7.3 kube-vip — Control-Plane VIP

What kube-vip Provides

- A **virtual IP and load balancer** for **both** the control plane (HA cluster) **and** Kubernetes Services of type LoadBalancer.
- **No external hardware or software** required.
- **Control-plane HA** via `kube_vip_controlplane_enabled` + `kube_vip_address`.
- **Service LoadBalancer** (optional) via `kube_vip_services_enabled`.
- Advertises the VIP in **ARP** (layer-2) or **BGP** (routed) mode.

Set `kube_proxy_strict_arp: true` when `kube_proxy_mode` is `ipvs` **and** kube-vip ARP is enabled.

kube-vip — ARP Mode (Layer-2)

```
kube_proxy_strict_arp: true
kube_vip_enabled: true
kube_vip_controlplane_enabled: true
kube_vip_address: 10.42.42.42
loadbalancer_apiserver:
  address: "{{ kube_vip_address }}"
  port: 6443
# kube_vip_interface: ens160
kube_vip_arp_enabled: true
```

- Simplest mode — VIP advertised via **ARP** on the L2 segment.
- `kube_vip_interface` pins the VIP to a specific NIC.

kube-vip — BGP Mode (Routed)

```
kube_vip_bgp_enabled: true
kube_vip_local_as: 65000
kube_vip_bgp_routerid: 192.168.0.2
kube_vip_bgppeers:
- 192.168.0.10:65000::false
- 192.168.0.11:65000::false
```

- Use BGP for **routed / multi-subnet** fabrics instead of L2 ARP.
- Peers as `address:as::password-flag`; pair with `kube_vip_address` for the VIP.

7.4 etcd — Deployment & Topology

etcd Deployment Types

`etcd_deployment_type` selects how etcd runs:

Value	What it does
<code>host</code> (default)	etcd as a systemd service on the host
<code>kubeadm</code>	etcd as a static Pod on control-plane hosts (experimental, new deploys)
<code>docker</code>	etcd in docker containers (container manager = docker)

Members live in the inventory **etcd group**; metrics via `etcd_metrics_port` (e.g. `2381`).

Stacked vs External etcd

- **Stacked** — etcd **co-located on the control-plane nodes** (common Kubespray default; `host` or `kubeadm`).
- **External** — **dedicated etcd nodes**, separate from the control plane.
- For externally load-balanced etcd, override:
`etcd_access_addresses`, `etcd_client_url`, `etcd_cert_alt_names`,
`etcd_cert_alt_ips`.

Always use an **odd number ≥ 3** of etcd members to keep **quorum** through a failure.

7.5 Failure Detection & Reliability

How a Node Failure Is Detected

Parameter	Default	Role
kubelet <code>--node-status-update-frequency</code>	10s	kubelet reports status
ctrl-mgr <code>--node-monitor-period</code>	5s	how often statuses are checked
<code>--node-monitor-grace-period</code>	40s	healthy if updated in this window
<code>nodeStatusUpdateRetry</code>	5	hardcoded retries
<code>--default-not-ready-toleration-seconds</code>	300s	wait before eviction
<code>--default-unreachable-toleration-seconds</code>	300s	wait before eviction

Past the grace period the node is unhealthy → pods evicted; **kube-proxy** drops its endpoints from Service iptables.

Tuning Failover Speed

- **Fast** (~20-50s eviction): `update-frequency=4s`, `monitor-period=2s`, `grace-period=20s`, toleration `30s`.
- **Medium** (~3 min): `update-frequency=20s`, `grace-period=2m`, toleration `60s`.
- **Slow** (~6 min, fewer false positives): `update-frequency=1m`, `grace-period=5m`, toleration `60s`.

Real-world detection is **slower than the theory** (network latency, apiserver delay, etcd commit time) — favour stability in production.

Key Takeaways

- HA = HA **etcd** (≥ 3 peers, odd) + HA **kube-apiserver** (a LB layer in front).
- API LB options: **localhost LB** (default, per-node nginx), **external LB** (`loadbalancer_apiserver` + domain name), or **kube-vip**.
- **kube-vip** gives a control-plane VIP (`kube_vip_controlplane_enabled`) in **ARP** (`kube_vip_arp_enabled`) or **BGP** (`kube_vip_bgp_enabled`) mode.
- etcd runs as `host` / `kubeadm` / `docker`, **stacked** or **external**.
- Failover speed is set by kubelet / controller-manager timers — fast/med/slow.

End of Lesson 7

Next: Lesson 8 — Deploying Applications

`loadbalancer_apiserver` · `kube_vip_*` · `etcd_deployment_type`

Deploying Applications

Kubernetes Production · Lesson 8

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain why **controllers** are preferred over bare Pods
- Use a **Deployment** for rolling updates, rollbacks and scaling
- Choose between **DaemonSet** and **StatefulSet** for the right workload
- Run batch work with **Job** and **CronJob**
- Use **init containers** and native **sidecar containers**
- Autoscale workloads with the **HorizontalPodAutoscaler** (and know VPA / Cluster Autoscaler)

Reference: *Workloads, Autoscaling Workloads* — kubernetes.io/docs

8.1 Pods vs Controllers

Pods vs Controllers

A **Pod** is the smallest deployable unit, but a bare Pod is **fragile** — it is not rescheduled, scaled or rolled forward. **Workload controllers** wrap a Pod template and continuously reconcile the desired state.

- If a bare Pod's node dies, the Pod is **gone** — nothing recreates it
- Controllers add self-healing, scaling and rollout behaviour

Controller	apiVersion	Manages	Workload type
Deployment	apps/v1	ReplicaSet → Pods	Stateless
DaemonSet	apps/v1	One Pod per node	Node agents
StatefulSet	apps/v1	Pods with identity	Stateful
Job / CronJob	batch/v1	Run-to-completion Pods	Batch

When to Use a Bare Pod

Use a standalone Pod **only** for short-lived, throwaway scenarios:

- Quick **debugging** or one-off `kubectl run` testing
- Interactive troubleshooting (`kubectl run -it --rm`)

For anything you care about, use a controller instead:

```
# Throwaway debug Pod — deleted on exit, never recreated
kubectl run tmp --image=busybox:1.28 -it --rm --restart=Never -- sh

# Real workload — managed, self-healing
kubectl create deployment web --image=nginx:1.14.2
```

CKA tip: in production you almost **never** create a naked Pod — a controller gives you healing, scaling and rollouts for free.

8.2 Deployments

What Is a Deployment?

A **Deployment** provides **declarative** updates for Pods and ReplicaSets. You describe the desired state; the Deployment controller drives actual state toward it at a controlled rate.

- You manage a **Deployment** → it owns a **ReplicaSet** → it owns the **Pods**
- The controller adds a `pod-template-hash` label to each ReplicaSet (auto-managed)
- Best fit for **stateless** applications
- Built-in **rolling updates**, **rollback**, **scaling** and **pause/resume**

Never manage the underlying ReplicaSet directly — let the Deployment own it.

A Basic Deployment Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3                # defaults to 1 if omitted
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```

The `selector.matchLabels` **must** match `template.metadata.labels`.

Rolling Updates

A rollout is triggered **if and only if** the Pod template (`.spec.template`) changes — scaling does **not** trigger one. The default strategy is **RollingUpdate**: new Pods come up while old ones drain.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # extra Pods above replicas (default 25%)
      maxUnavailable: 1    # Pods allowed down (default 25%)
```

- **maxSurge** — how many *extra* Pods may exist during the update
- **maxUnavailable** — how many Pods may be *unavailable* during the update
- **Recreate** strategy kills all old Pods first → **downtime**

Driving & Observing a Rollout

```
# Trigger a rollout by changing the image
kubectl set image deployment/nginx-deployment nginx=nginx:1.16.1

# Watch progress until complete
kubectl rollout status deployment/nginx-deployment

# Pause to batch several edits, then resume as one rollout
kubectl rollout pause deployment/nginx-deployment
kubectl rollout resume deployment/nginx-deployment
```

- `Progressing=True, Available=True` → rollout **complete**
- A stuck rollout (bad image, failing readiness) stays **Progressing**

CKA tip: `kubectl edit deployment/<name>` also triggers a rollout when it changes the Pod template.

History, Rollback & Scaling

```
# View revision history
kubectl rollout history deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment --revision=2

# Roll back to the previous (or a specific) revision
kubectl rollout undo deployment/nginx-deployment
kubectl rollout undo deployment/nginx-deployment --to-revision=2

# Scale up or down (no rollout)
kubectl scale deployment/nginx-deployment --replicas=5
```

- `revisionHistoryLimit` (default **10**) caps how many old ReplicaSets are kept
- Old ReplicaSets are retained so you can **roll back**

CKA tip: `--record` is deprecated; revisions are tracked automatically.

8.3 DaemonSets

DaemonSets — One Pod Per Node

A **DaemonSet** ensures that **all (or some) nodes** run a copy of a Pod.

- A node **joins** → the controller automatically schedules the Pod onto it
- A node is **removed** → its Pod is garbage-collected
- Deleting the DaemonSet **cleans up** all the Pods it created
- There is **no replicas** field — count = number of eligible nodes

Typical use cases:

- Cluster **storage** daemon, node **logging** collector (Fluentd), node **monitoring** agent

The controller auto-adds tolerations (**not-ready** , **unreachable** , pressure, **unschedulable**) so Pods run even on problem nodes.

A DaemonSet Manifest

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
  template:
    metadata:
      labels:
        name: fluentd-elasticsearch
    spec:
      tolerations:
        - key: node-role.kubernetes.io/control-plane
          operator: Exists
          effect: NoSchedule
      containers:
        - name: fluentd-elasticsearch
          image: quay.io/fluentd_elasticsearch/fluentd:v5.0.1
```

Use `nodeSelector` / `affinity` to target a subset of nodes.

8.4 StatefulSets

StatefulSets — Stable Identity

A **StatefulSet** manages stateful apps where Pods are **not** interchangeable. Each Pod keeps a **sticky identity** across rescheduling.

Use a StatefulSet when you need:

- **Stable, unique network identifiers** (`web-0` , `web-1` , `web-2`)
- **Stable, persistent storage** that follows each Pod
- **Ordered, graceful** deployment and scaling, plus ordered rolling updates

Behaviour	Order
Create / scale up	0 → 1 → 2 (one at a time, after Ready)
Delete / scale down	2 → 1 → 0 (reverse)

Headless Service + StatefulSet

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  clusterIP: None          # Headless – required for stable DNS
  selector:
    app: nginx
  ports:
    - port: 80
      name: web
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"    # references the headless Service
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: registry.k8s.io/nginx-slim:0.24
```

Stable Storage & DNS

`volumeClaimTemplates` creates **one PVC per Pod**, bound to that Pod's identity:

```
volumeClaimTemplates:
- metadata:
  name: www
  spec:
    accessModes: [ "ReadWriteOnce" ]
    resources:
      requests:
        storage: 1Gi
```

- Each Pod gets a stable DNS name via the headless Service:

`web-0.nginx.default.svc.cluster.local`

- PVCs are **kept** when you scale down or delete the StatefulSet (data safety)

CKA tip: a StatefulSet **requires** a headless Service (`clusterIP: None`); without it, stable network identity does not work.

8.5 Jobs & CronJobs

Jobs and CronJobs

For **batch / run-to-completion** work, not long-running services:

- **Job** — runs Pods until a set number complete **successfully**, then stops
- **CronJob** — creates Jobs on a repeating **schedule** (cron syntax)

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: hello
              image: busybox:1.28
              command: ["sh", "-c", "date; echo Hello from k8s"]
```

Batch Pods use **restartPolicy: OnFailure** or **Never** — **never** Always.

8.6 Init & Sidecar Containers

Init Containers

Init containers run **before** the app containers in a Pod:

- Run **to completion, in order** — each must succeed before the next
- **All** init containers must finish before any app container starts
- On failure the kubelet **restarts** them until they succeed
(unless `restartPolicy: Never` → the Pod fails)
- Do **not** support `livenessProbe`, `readinessProbe`, `startupProbe` or lifecycle hooks

Use cases: wait for a Service, clone a Git repo, render config, run setup scripts kept out of the app image.

Init Container Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  initContainers:
    - name: init-myservice
      image: busybox:1.28
      command: ['sh', '-c', "until nslookup myservice; do echo waiting; sleep 2; done"]
  containers:
    - name: myapp-container
      image: busybox:1.28
      command: ['sh', '-c', 'echo The app is running! && sleep 3600']
```

```
# Inspect an init container's progress and logs
kubectl describe pod myapp-pod
kubectl logs myapp-pod -c init-myservice
```

Native Sidecar Containers (1.29+)

A **native sidecar** is an **init container** with `restartPolicy: Always`.

- **Enabled by default since Kubernetes 1.29** (graduated to stable in 1.33)
- Starts during init, then **keeps running** for the Pod's whole lifetime
- Terminates **after** the main app containers stop (reverse order)
- **Supports probes** — unlike a plain init container
- Started **before** app containers, so it is ready when the app needs it

Difference from a regular init container: an init container **stops** before the app starts; a sidecar **stays up** alongside the app.

Sidecar Manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: alpine:latest
          command: ['sh', '-c', 'while true; do echo hi >> /opt/logs.txt; sleep 1; done']
          volumeMounts:
            - { name: data, mountPath: /opt }
      initContainers:
        - name: logshipper
          image: alpine:latest
          restartPolicy: Always          # makes this a sidecar
          command: ['sh', '-c', 'tail -F /opt/logs.txt']
          volumeMounts:
            - { name: data, mountPath: /opt }
      volumes:
        - name: data
          emptyDir: {}
```

8.7 Workload Autoscaling

From Manual Scaling to Autoscaling

You already scale a Deployment **manually** with `kubectl scale`. A **HorizontalPodAutoscaler (HPA)** does it **automatically** based on observed load.

- HPA adjusts the **replica count** of a Deployment / ReplicaSet / StatefulSet
- Scales on **CPU**, **memory**, or **custom / external** metrics
- **Requires the metrics-server** (or a metrics adapter) to read usage
- Pods **must declare resource requests** — utilization is measured against them
- Does **not** apply to **DaemonSets** (their count tracks nodes)

HPA scales **horizontally** (more Pods). It is itself a control loop that reconciles the current metric toward the desired replica count.

HorizontalPodAutoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

```
# Imperative equivalent + watch
kubectl autoscale deployment php-apache --cpu-percent=50 --min=1 --max=10
kubectl get hpa      # TARGETS = current%/target%, plus REPLICAS
```

$$\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{targetMetric})$$

VPA & Cluster Autoscaler

Two complementary autoscalers — **separate add-ons, not built in:**

- **Vertical Pod Autoscaler (VPA)** — recommends and can set Pod **requests/limits**, then **restarts** Pods to apply (per-Pod sizing)
- **Cluster Autoscaler** — adds/removes **nodes** when Pods can't schedule or nodes sit underused (cloud-provider driven)

Autoscaler	Changes	Scope
HPA	Number of Pods	Workload
VPA	Pod requests/limits	Workload
Cluster Autoscaler	Number of nodes	Cluster

Don't run HPA and VPA on the **same** CPU/memory metric — they fight.

Key Takeaways

- Prefer **controllers** over bare Pods — they self-heal, scale and roll out
- **Deployment** = stateless apps; owns a ReplicaSet; `rollout status/history/undo`
- Tune rollouts with `maxSurge` and `maxUnavailable` (default 25% each)
- **DaemonSet** = one Pod per node (no `replicas`); **StatefulSet** = stable identity + per-Pod storage + headless Service
- **Job / CronJob** = run-to-completion batch work (`restartPolicy` OnFailure/Never)
- **Init containers** run first to completion; a **sidecar** is an init container with `restartPolicy: Always` (1.29+)
- **HPA** autoscales replicas on metrics (needs metrics-server + Pod `requests`); **VPA** tunes requests, **Cluster Autoscaler** adds nodes

End of Lesson 8

Next: Lesson 9 — Managing Storage

`kubectl get deploy,ds,sts` · `kubectl rollout status` · `kubectl scale` · `kubectl autoscale`

Managing Storage

Kubernetes Production · Lesson 9

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Distinguish **ephemeral** from **persistent** volumes and mount them into Pods
- Use **emptyDir**, **projected** sources, and understand **hostPath** caveats
- Explain the **PV / PVC** abstraction and its full **lifecycle**
- Choose the right **access modes** and **reclaim policies**
- Use **StorageClasses** for **dynamic provisioning** and tune `volumeBindingMode`
- Connect storage to **StatefulSet** `volumeClaimTemplates`

References: *Volumes*, *Persistent Volumes*, *Storage Classes* — kubernetes.io/docs/concepts/storage

9.1 Volumes

What Is a Volume?

A **volume** is a directory, possibly containing data, **accessible to the containers in a Pod**. Declared at the **Pod level** and mounted into containers.

- **Ephemeral** volumes have the **lifetime of the Pod** — destroyed when the Pod ceases to exist
- **Persistent** volumes exist **beyond** any individual Pod
- For *any* type, data is **preserved across container restarts** within the Pod

Two fields wire it together:

- `spec.volumes[]` — declares the volume on the Pod
- `containers[].volumeMounts[]` — mounts it at a `mountPath`

A volume **cannot be mounted inside another volume** — use `subPath` instead.

Mounting a Volume Into a Container

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-demo
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: cache
          mountPath: /tmp/cache      # where the container sees it
        - name: cache
          mountPath: /var/log
          readOnly: true
  volumes:
    - name: cache                  # the volume definition
      emptyDir: {}
```

- The **same** volume can mount at **different paths**, even read-only
- Each container independently declares **where** to mount each volume

emptyDir — Scratch Space

An `emptyDir` is created **empty** when the Pod is **assigned to a node** and lives as long as that Pod runs on the node.

- **Survives container crashes**; deleted **permanently** when the Pod is removed
- Great for scratch space, checkpointing, sharing files between containers
- `sizeLimit` caps capacity; `medium: Memory` makes it a **RAM-backed tmpfs**

```
volumes:  
  - name: cache-volume  
    emptyDir:  
      sizeLimit: 500Mi  
      medium: Memory      # tmpfs — counts against the container's memory
```

CKA tip: `medium: Memory` files are RAM-backed and **charged against the Pod's memory limit** — not durable storage.

Projected Volumes

A **projected** volume maps **several sources into one directory**. Supported sources:

- **configMap** — non-sensitive configuration data
- **secret** — sensitive data (keys, tokens, passwords)
- **downwardAPI** — Pod / container metadata as files
- **serviceAccountToken** — short-lived, audience-scoped SA token

```
volumes:  
  - name: all-in-one  
    projected:  
      sources:  
        - configMap: { name: my-config }  
        - secret: { name: my-secret }  
        - downwardAPI:  
            items:  
              - path: "labels"  
                fieldRef: { fieldPath: metadata.labels }  
        - serviceAccountToken:  
            audience: vault  
            expirationSeconds: 3600  
            path: token
```

configMap, secret & downwardAPI

- **configMap** and **secret** volumes are mounted **read-only**; the ConfigMap / Secret must **exist before** the Pod uses it
- **downwardAPI** exposes Pod metadata (labels, annotations, resource limits) as files
- Updates to the source **propagate** to the mounted files automatically...
- ...**except** when mounted via **subPath** — those do **not** receive updates

```
volumes:  
  - name: config-vol  
    configMap:  
      name: log-config  
      items:  
        - key: log_level  
          path: log_level.conf          # appears at mountPath/log_level.conf
```

CKA tip: prefer projected/Secret volumes over env vars for credentials — they can be **rotated** without restarting the Pod.

hostPath — Handle With Care

`hostPath` mounts a file or directory **from the host node** into the Pod.

- `type` validates the target: `Directory`, `DirectoryOrCreate`, `File`, `FileOrCreate`, `Socket`, `CharDevice`, `BlockDevice`, or `""` (no checks)

```
volumes:  
  - name: host-volume  
    hostPath:  
      path: /data  
      type: Directory
```

Security warning: `hostPath` grants direct node filesystem access and enables privilege escalation / container escape. Avoid it; use a **PersistentVolume** instead.

9.2 PersistentVolumes & Claims

The PV / PVC Abstraction

Two API objects decouple **how** storage is provided from **how** it is consumed:

Object	Scope	Who owns it	Purpose
PersistentVolume (PV)	Cluster	Admin / dynamic provisioner	Storage with a lifecycle independent of any Pod
PersistentVolumeClaim (PVC)	Namespace	User / developer	A request for storage (size + access mode)

- A PVC binds to **one** PV in an exclusive **one-to-one** relationship (`claimRef`)
- Pods reference the **PVC**, never the PV directly

Think of it like Pods consuming **node** resources — PVCs consume **PV** resources.

Lifecycle: Provisioning

PVs come into existence in one of two ways:

- **Static** — an administrator pre-creates PVs with real storage details; they sit **Available** until a matching claim arrives
- **Dynamic** — when no static PV matches, the cluster provisions one **on demand** from the PVC's **StorageClass** (needs the `DefaultStorageClass` admission controller for default-class behaviour)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /mnt/data
```

Lifecycle: Binding & Using

Binding

- A control loop matches a PVC to a suitable PV (capacity $\geq$ request, compatible access mode and StorageClass); the user gets **at least** what was requested
- If no PV matches, the PVC stays **Pending** indefinitely

Using

- The Pod mounts the PVC; **Storage Object in Use Protection** finalizers (`pvc-protection` / `pv-protection`) block deletion while in use

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 3Gi
  storageClassName: manual
```

A Pod Consuming a PVC

A Pod references a PVC by name in `spec.volumes[].persistentVolumeClaim.claimName`.

```
apiVersion: v1
kind: Pod
metadata:
  name: task-pod
spec:
  containers:
    - name: task-container
      image: nginx
      volumeMounts:
        - mountPath: /usr/share/nginx/html
          name: task-storage
  volumes:
    - name: task-storage
      persistentVolumeClaim:
        claimName: task-pvc      # the PVC, in the same namespace
```

- PVCs are **namespace-scoped** — the Pod and PVC must share a namespace
- The bound PV may be **larger** than requested; capacity is a minimum

Access Modes

A PV advertises modes; a PVC **requests** the modes it needs.

Mode	Short	Meaning
ReadWriteOnce	RWO	Mounted read-write by a single node (many Pods on it OK)
ReadOnlyMany	ROX	Mounted read-only by many nodes
ReadWriteMany	RWX	Mounted read-write by many nodes
ReadWriteOncePod	RWOP	Mounted read-write by a single Pod (stable v1.29)

- Modes are about **node**-level mounting, not Pod count (except RWOP)
- A volume can only be mounted using **one** access mode at a time

CKA tip: most block storage (cloud disks, RBD) is **RWO**; shared filesystems (NFS, CephFS) provide **RWX**.

Lifecycle: Reclaiming

When a PVC is deleted, the PV's **reclaim policy** decides its fate:

Policy	Behaviour
Retain	PV becomes Released ; data kept, admin must reclaim manually
Delete	PV and the backing external storage are deleted automatically
Recycle	Basic scrub (<code>rm -rf /thevolume/*</code>) — deprecated

PV phases you will observe:

- **Available** → unbound · **Bound** → claimed · **Released** → claim gone, not reclaimed · **Failed** → reclaim error

CKA tip: with **Retain**, a Released PV is **not** automatically reusable — you must delete and recreate it (or clear its `claimRef`).

9.3 StorageClasses & Dynamic Provisioning

What Is a StorageClass?

A **StorageClass** describes a *class* of storage the cluster offers — a "profile" for a QoS tier or admin policy — and enables **dynamic provisioning**.

Key fields:

- **provisioner** (*required*) — the volume plugin / CSI driver to use
- **parameters** — provider-specific options passed to the provisioner
- **reclaimPolicy** — **Delete** (default) or **Retain** for provisioned PVs
- **allowVolumeExpansion** — permit growing provisioned volumes (no shrink)
- **mountOptions** — mount flags applied to provisioned PVs
- **volumeBindingMode** — *when* binding / provisioning happens

```
kind: StorageClass lives under apiVersion: storage.k8s.io/v1 .
```

StorageClass Manifest

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: csi-driver.example-vendor.example
reclaimPolicy: Retain
allowVolumeExpansion: true
mountOptions:
  - discard
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: ssd
```

- A PVC selects a class via `spec.storageClassName: fast-ssd`
- Omit it and the **default** StorageClass (if any) is used

volumeBindingMode

Controls **when** the volume is bound and (for dynamic) provisioned:

Mode	When binding happens
Immediate <i>(default)</i>	As soon as the PVC is created
WaitForFirstConsumer	Delayed until a Pod using the PVC is scheduled

- **Immediate** can provision storage in a zone the Pod's node **cannot reach**
- **WaitForFirstConsumer** provisions respecting the Pod's **resource requests, node selectors, affinity, taints/tolerations** — the topology-aware choice

CKA tip: for zonal cloud disks, prefer **WaitForFirstConsumer** so the disk lands in the same zone as the Pod (don't use **nodeName** with it).

The Default StorageClass

- Mark exactly **one** class default with the annotation
`storageclass.kubernetes.io/is-default-class: "true"`
- A PVC that **omits** `storageClassName` then uses that default automatically
- Setting `storageClassName: ""` (empty string) **disables dynamic provisioning** for that PVC — it can bind only to a matching pre-created PV

```
kubectl get storageclass
kubectl get sc -o jsonpath='{.items[?(@.metadata.annotations.storageclass\.kubernetes\.io/is-default-class=="true")].metadata.name}'
```

Keep only **one** default — if several exist, the **most recently created** wins.

Storage in StatefulSets

StatefulSets give each replica **stable, per-Pod storage** via `volumeClaimTemplates` — a PVC template, not a shared claim.

```
spec:
  serviceName: web
  replicas: 3
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ "ReadWriteOnce" ]
        storageClassName: fast-ssd
        resources:
          requests:
            storage: 1Gi
```

- One PVC is created **per Pod** (e.g. `data-web-0`, `data-web-1`, ...)
- Claims (and their PVs) **persist** across rescheduling and are **not** deleted on scale-down

CKA tip: deleting a StatefulSet leaves its PVCs behind — clean them up manually.

Key Takeaways

- Volumes attach at the **Pod** level (`spec.volumes`) and mount via `volumeMounts`
- **emptyDir** is ephemeral scratch; **projected** combines configMap/secret/downwardAPI/SA-token; **hostPath** is a security risk — avoid it
- **PV** (cluster) + **PVC** (namespace) decouple provisioning from consumption; lifecycle is **provision** → **bind** → **use** → **reclaim**
- Know the access modes: **RWO**, **ROX**, **RWX**, **RWOP**; reclaim policies **Retain** / **Delete**
- **StorageClass** drives **dynamic provisioning**; **WaitForFirstConsumer** is topology-aware; one **default** class per cluster
- **StatefulSets** use **volumeClaimTemplates** for stable per-Pod storage

End of Lesson 9

Next: Lesson 10 — Application Access

```
kubectl get pv · kubectl get pvc · kubectl get sc · kubectl describe pvc <name>
```

Managing Application Access

Kubernetes Production · Lesson 10

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **Service** abstraction, selectors and **EndpointSlices**
- Choose the right **Service type**: ClusterIP, NodePort, LoadBalancer, ExternalName
- Use **headless Services** and **StatefulSet** stable DNS names
- Resolve workloads with **cluster DNS** naming
- Expose HTTP/HTTPS apps with **Ingress** and an **IngressClass**
- Distinguish **Service (L4)** from **Ingress (L7)** and use `kubectl port-forward`
- Route traffic with the **Gateway API** (GatewayClass / Gateway / HTTPRoute)

Reference: *Service, Ingress, Gateway API, DNS for Services and Pods* — kubernetes.io/docs

10.1 The Service Abstraction

Why Services?

Pods are **ephemeral**: they are created and destroyed dynamically, and their **IPs change**.

A **Service** exposes a logical set of Pods behind a single, **stable endpoint**.

- Decouples frontend clients from the backend Pod details
- Tracks the changing set of healthy Pods automatically
- Provides a stable **virtual IP (ClusterIP)** and **DNS name**
- Load-balances connections across the matching Pods

The Service is the **L4** building block of Kubernetes networking — it works on TCP/UDP/SCTP ports, not on HTTP paths.

Selectors & EndpointSlices

A Service with a **selector** continuously matches Pods by their **labels**.

- Matching Pod IP:port pairs are tracked in **EndpointSlices**
- EndpointSlices are populated automatically for selector-based Services
- The legacy **Endpoints** object is now **deprecated** in favour of EndpointSlices
- A Service **without a selector** needs manually managed EndpointSlices
- **kube-proxy** programs the rules that map the virtual IP to those Pods

```
kubectl get endpointslices  
kubectl describe service my-service
```

Defining a Service

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80 # port exposed by the Service
      targetPort: 9376 # port on the Pods (defaults to port)
```

- **port** — the port clients use on the Service's ClusterIP
- **targetPort** — the container port; may reference a **named** port
- Omitting **targetPort** makes it default to the value of **port**

Multi-Port Services

A single Service can expose **several named ports**, each to its own targetPort.

```
spec:
  selector:
    app: web
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 8080
    - name: https
      protocol: TCP
      port: 443
      targetPort: 8443
```

- Each entry **must** be named when more than one port is defined
- Different ports may use different protocols (TCP, UDP, SCTP)

10.2 Service Types

The Four Service Types

Type	Reachable from	Builds on
ClusterIP <i>(default)</i>	Inside the cluster only	—
NodePort	<code><NodeIP>:<port></code> externally	ClusterIP
LoadBalancer	Cloud provider's external LB	NodePort + ClusterIP
ExternalName	CNAME to an external DNS name	— (no proxying)

- Higher types **layer on** the lower ones (LoadBalancer $\supset$ NodePort $\supset$ ClusterIP).
- **kube-proxy** on each node programs the rules that route the virtual IP to Pods.

CKA tip: most workloads stay **ClusterIP**; you expose them externally with a single **Ingress** or LoadBalancer at the edge.

ClusterIP & NodePort

ClusterIP (default)

- Allocates a stable cluster-internal virtual IP
- Only reachable from inside the cluster
- The foundation every other type relies on

NodePort

- Opens the **same static port on every node**, forwarding to the Service
- Default port range: **30000–32767**
- Reachable externally at `<NodeIP>:<NodePort>`; also gets a ClusterIP

```
kubectl expose deployment web --type=NodePort --port=80 --target-port=8080
```

LoadBalancer & ExternalName

LoadBalancer

- Provisions an **external load balancer** via the cloud provider
- Automatically chains the NodePort and ClusterIP underneath
- On bare metal you need an add-on (e.g. MetalLB) to fulfil it

ExternalName

- Maps the Service to an external **DNS name** via a CNAME record
- No selector, no proxying, no EndpointSlices

```
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  type: ExternalName
  externalName: db.prod.example.com
```


10.3 Headless Services & DNS

Headless Services

Set `clusterIP: None` to create a **headless** Service (no virtual IP).

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  clusterIP: None
  selector:
    app: nginx
  ports:
    - port: 80
```

- DNS returns the **individual Pod IPs** (A/AAAA records), not one VIP
- Clients select or round-robin Pods directly — ideal for **StatefulSets**
- Without a selector, you manage the EndpointSlices yourself

Cluster DNS Naming

Every Service gets a DNS name resolved by **CoreDNS**:

```
<service>.<namespace>.svc.cluster.local
```

- Same namespace: just use `<service>` (e.g. `my-svc`)
- Other namespace: `<service>.<namespace>` (e.g. `my-svc.prod`)
- A Pod's `/etc/resolv.conf` adds search domains, so short names expand
- Named ports also publish **SRV** records: `__<port>.__<proto>.<svc>.<ns>.svc...`

A normal Service resolves to its **ClusterIP**; a **headless** Service resolves to the **set of Pod IPs**.

StatefulSet & Pod DNS

A **StatefulSet** + headless Service gives each Pod a **stable** DNS name:

```
<pod-name>.<service>.<namespace>.svc.cluster.local  
web-0.nginx.default.svc.cluster.local
```

- Names **persist** across reschedules — essential for clustered databases
- Pods also get IP-based records: `<ip-dashes>.<namespace>.pod.cluster.local`
 - e.g. `172-17-0-3.default.pod.cluster.local`

```
kubectl run -it --rm dnsutils --image=busybox:1.28 \  
-- nslookup web-0.nginx.default.svc.cluster.local
```

10.4 Ingress (L7 HTTP Routing)

What Is Ingress?

Ingress exposes **HTTP/HTTPS** routes from outside the cluster to Services, providing:

- Name-based **virtual hosting** (route by hostname)
- **Path-based** routing to different backend Services
- **TLS termination** and load balancing at L7

An Ingress resource does **nothing on its own** — you must deploy an **Ingress controller** (nginx, HAProxy, cloud LB) to fulfil it.

- For non-HTTP traffic, use a NodePort or LoadBalancer Service instead.
- The Ingress API is **frozen**; the project now recommends the **Gateway API**.

A Minimal Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```

- An Ingress must define `spec.rules` and/or a `spec.defaultBackend`.

IngressClass & Rules

IngressClass selects which controller handles the resource.

- Set via `spec.ingressClassName`
- If omitted, the cluster's **default** IngressClass is used (if one exists)

Each HTTP rule combines:

- **host** (*optional*) — e.g. `foo.bar.com`; if omitted, matches all hosts
- **path** — the URI to match, e.g. `/app`
- **backend** — the target `service.name` + `service.port.number`

The **defaultBackend** serves any request that matches no rule (required if no rules are specified).

pathType Matters

Every path **must** declare a `pathType`:

pathType	Matching behaviour
Exact	Matches the URL path exactly — <code>/foo</code> $\neq$ <code>/foo/</code>
Prefix	Matches by <code>/</code> -split path prefix — <code>/foo</code> matches <code>/foo/bar</code>
ImplementationSpecific	Delegated to the IngressClass / controller

```
- path: /api
  pathType: Prefix
  backend:
    service:
      name: api-svc
      port:
        number: 8080
```

CKA tip: omitting `pathType` is invalid in `networking.k8s.io/v1`.

10.5 Service vs Ingress & port-forward

Service (L4) vs Ingress (L7)

Aspect	Service	Ingress
Layer	L4 (TCP/UDP/SCTP)	L7 (HTTP/HTTPS)
Routes by	IP + port	Hostname + URL path
Protocols	Any	HTTP/HTTPS only
Needs controller?	No (kube-proxy)	Yes — an ingress controller
Typical use	Internal connectivity, NodePort/LB edge	Web traffic, virtual hosts

- Ingress routes incoming HTTP to **Services**, which forward to **Pods**.
- A single Ingress + controller can front **many** Services on one IP.

kubectl port-forward

For quick, **local** access to a Pod or Service — no Service exposure needed.

```
# Forward local port 8080 to a Pod's port 80
kubectl port-forward pod/web-0 8080:80

# Forward to a Service (picks a backing Pod)
kubectl port-forward service/my-service 8080:80
```

- Traffic tunnels through the **API server** to the target
- Great for **debugging** and reaching ClusterIP-only workloads
- It runs only while the command is active — **not** a production exposure

CKA tip: use **port-forward** to test a Pod before wiring up a Service or Ingress.

10.6 Gateway API

Gateway API — Ingress's Successor

The **Gateway API** (gateway.networking.k8s.io) is the next-generation, role-oriented standard for managing traffic into the cluster. The **Ingress API is frozen** — new routing features now land here.

- More **expressive** than Ingress: header/method matching, traffic splitting, cross-namespace routing
- **Role-oriented** resources separate infrastructure from application concerns
- Handles **HTTP, HTTPS, TLS, TCP and gRPC** — not just HTTP/S
- Shipped as **CRDs** (installed separately) plus a controller that implements them

Like Ingress, a Gateway does nothing without a **controller** (an implementation such as NGINX, Istio, Contour or Cilium).

The Resource Model

Three core kinds, each owned by a different role:

Kind	Owned by	Purpose
GatewayClass	Infra provider	The implementation/controller (like an IngressClass)
Gateway	Cluster operator	A traffic entry point: listeners, ports, protocols
HTTPRoute	App developer	How requests map to backend Services

- A **Gateway** references a **GatewayClass**; **Routes** attach to a Gateway via `parentRefs`.
- Route kinds: **HTTPRoute**, **GRPCRoute**, **TLSRoute**, **TCPRoute**, **UDPRoute**.

Gateway + HTTPRoute

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
spec:
  gatewayClassName: example-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
  ---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: example-route
spec:
  parentRefs:
  - name: example-gateway
  hostnames: ["www.example.com"]
  rules:
  - matches:
    - path: { type: PathPrefix, value: / }
    backendRefs:
    - name: example-svc
      port: 80
```


Ingress vs Gateway API

Aspect	Ingress	Gateway API
API group	networking.k8s.io	gateway.networking.k8s.io
Status	Frozen (no new features)	Active — where features land
Model	One object + annotations	Role-oriented: Class / Gateway / Route
Protocols	HTTP/HTTPS	HTTP, gRPC, TLS, TCP, UDP
Expressiveness	Host + path	Headers, methods, weights, cross-NS
Install	Built-in API + controller	CRDs + controller

CKA tip: know the three kinds and that routes attach via `parentRefs` ; the API is delivered as **CRDs** you install first.

Key Takeaways

- A **Service** gives Pods a stable VIP + DNS name; selectors populate **EndpointSlices** (Endpoints is deprecated)
- **Types**: ClusterIP (internal), NodePort (`30000-32767`), LoadBalancer (cloud LB), ExternalName (CNAME)
- **Headless** (`clusterIP: None`) returns Pod IPs; StatefulSet Pods get stable `<pod>.<svc>.<ns>.svc.cluster.local` names
- DNS pattern: `<svc>.<ns>.svc.cluster.local`, resolved by CoreDNS
- **Ingress** = L7 HTTP routing; needs an **ingress controller** + a `pathType`
- **Service is L4, Ingress is L7**; `kubectl port-forward` is for local access
- The **Gateway API** (`gateway.networking.k8s.io`) is Ingress's role-oriented successor — **GatewayClass** → **Gateway** → **HTTPRoute**, installed as CRDs

End of Lesson 10

Next: Lesson 11 — Configuration & Quotas

`kubectl expose` · `kubectl get svc,endpointslices` · `kubectl get ingress` · `kubectl port-forward`

Managing Clusters

Kubernetes Production · Lesson 11

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Use **Namespaces** to isolate resources and switch context
- Create and consume a **ConfigMap** (env, envFrom, volume, subPath)
- Create and consume a **Secret**, and explain why base64 ≠ encryption
- Constrain a namespace with a **ResourceQuota**
- Inject defaults and per-container bounds with a **LimitRange**

All five objects are `apiVersion: v1` — kubernetes.io/docs/concepts

11.1 Namespaces

What Is a Namespace?

A **Namespace** isolates groups of resources within a **single cluster**.

- Resource **names must be unique within a namespace**, not across them
- Applies to **namespace-scoped** objects (Deployments, Services, Pods...)
- Cluster-wide objects (Nodes, PersistentVolumes, StorageClass) are **not** namespaced
- Use them for **many users across teams**; for a few users, prefer **labels**

Production tip: consider **not** using the `default` namespace.

The Four Initial Namespaces

Namespace	Purpose
default	Use the cluster without first creating a namespace
kube-system	Objects created by the Kubernetes system
kube-public	Readable by all clients; public cluster-wide data
kube-node-lease	Holds Lease objects for node heartbeats

- Avoid the kube- prefix — reserved for system namespaces
- Control plane sets an immutable label `kubernetes.io/metadata.name`

Namespaced vs Cluster-Scoped

```
# List namespaces
kubectl get namespace

# Create a namespace
kubectl create namespace dev

# Which resources live IN a namespace?
kubectl api-resources --namespaced=true

# Which resources are cluster-wide?
kubectl api-resources --namespaced=false
```

CKA tip: `--namespaced=false` quickly tells you whether an object (e.g. a Node or PV) can be placed in a namespace.

Switching Namespace Context

```
# One-off: target a namespace for a single command
kubectl get pods --namespace=dev
kubectl run nginx --image=nginx -n dev

# Persistent: change the default namespace for your context
kubectl config set-context --current --namespace=dev

# Verify the active namespace
kubectl config view --minify | grep namespace:
```

- Service DNS: `<service-name>.<namespace-name>.svc.cluster.local`
- Same namespace → short `<service-name>`; cross-namespace → FQDN

11.2 ConfigMaps

What Is a ConfigMap?

A **ConfigMap** (`apiVersion: v1`) stores **non-confidential** data as key-value pairs, decoupling config from the container image.

- Consumed as **env vars**, **command args**, or **files in a volume**
- **Not** secret and **not** encrypted — for confidential data use a Secret
- Size limit **1 MiB**; `data` = UTF-8 strings, `binaryData` = base64
- `immutable: true` (since v1.19) prevents updates and improves performance

ConfigMap and the Pod that uses it must be in the **same namespace**.

Creating a ConfigMap

```
# From literal key-value pairs
kubectl create configmap game-demo \
  --from-literal=player_initial_lives=3

# From a file (key = filename) or a whole directory
kubectl create configmap app-config --from-file=game.properties
kubectl create configmap app-config --from-file=config-dir/
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  player_initial_lives: "3"
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
```

Consume as Env Vars

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-demo-pod
spec:
  containers:
    - name: demo
      image: alpine
      env:
        - name: PLAYER_INITIAL_LIVES      # single key
          valueFrom:
            configMapKeyRef:
              name: game-demo
              key: player_initial_lives
      envFrom:
        - configMapRef:                  # all keys at once
            name: game-demo
```

Env vars are **not** auto-updated when the ConfigMap changes — restart the Pod.

Consume as a Mounted Volume

```
spec:
  containers:
    - name: demo
      image: redis
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
          readOnly: true
  volumes:
    - name: config-volume
      configMap:
        name: game-demo          # each key becomes a file under /etc/config
```

- Omit `items` → every key is mounted as a file named after the key
- **Volume-mounted** ConfigMaps are eventually updated automatically

Mounting a Single Key with subPath

```
volumeMounts:
- name: config
  mountPath: /etc/config/game.properties
  subPath: game.properties          # mount ONE key as a single file
  readOnly: true
volumes:
- name: config
  configMap:
    name: game-demo
```

- `subPath` injects one key **without hiding** other files in the directory
- Trade-off: a `subPath` mount does **not** receive automatic updates

11.3 Secrets

What Is a Secret?

A **Secret** (`apiVersion: v1`) stores a **small amount of sensitive data** — passwords, tokens, keys — out of Pod specs and images.

- `data` holds **base64-encoded** values; `stringData` takes plaintext (auto-encoded)
- Size limit **1 MiB**; supports `immutable: true`
- **base64 is encoding, NOT encryption** — anyone with API/etcd access can read it

Hardening: enable **Encryption at Rest** (`EncryptionConfiguration`), apply least-privilege **RBAC**, and restrict Secret access to specific containers.

Secret Types

Type	Usage
Opaque	Arbitrary user-defined data (default)
kubernetes.io/service-account-token	ServiceAccount token
kubernetes.io/dockerconfigjson	Private registry pull credentials
kubernetes.io/basic-auth	Basic-auth username/password
kubernetes.io/ssh-auth	SSH credentials
kubernetes.io/tls	TLS cert + key (tls.crt , tls.key)
bootstrap.kubernetes.io/token	Bootstrap token data

Creating a Secret

```
# Generic secret from literals
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=s3cr3t

# TLS secret from cert + key files
kubectl create secret tls tls-secret \
  --cert=path/to/cert.crt --key=path/to/key.key
```

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
stringData:      # plaintext, encoded for you on create
  username: admin
  password: s3cr3t
```

Consuming a Secret

```
spec:
  containers:
  - name: app
    image: redis
    env:
    - name: PASSWORD
      valueFrom:
        secretKeyRef:      # one key as an env var
          name: mysecret
          key: password
    envFrom:
    - secretRef:           # all keys as env vars
      name: mysecret
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: mysecret  # keys become files under /etc/secrets
```

Inspecting a Secret

```
# View the (base64-encoded) Secret
kubectl get secret mysecret -o yaml

# Decode a single value
kubectl get secret mysecret \
  -o jsonpath='{.data.username}' | base64 -d
```

- Mounted Secret files appear at `/etc/secrets/<key>`
- Volume-mounted Secrets refresh automatically; env vars require a Pod restart

CKA tip: `base64 -d` to decode — the exam often hands you a Secret and asks for the plaintext value.

11.4 ResourceQuota

What Is a ResourceQuota?

A **ResourceQuota** (`apiVersion: v1`) limits **aggregate** resource consumption **per namespace** so one team can't monopolize the cluster.

- A cluster admin creates **at least one per namespace**
- The control plane **rejects requests with HTTP 403** when a quota is exceeded
- Constrains **compute** (cpu/memory), **storage**, and **object counts**

When a compute quota is set, **every new Pod must declare requests/limits** for that resource — otherwise its creation is rejected.

Quota Dimensions

Category	Example fields
Compute	requests.cpu , requests.memory , limits.cpu , limits.memory
Storage	requests.storage , persistentvolumeclaims
Object count	pods , services , configmaps , secrets , count/<resource>

```
kubectl create quota my-quota \
  --hard=requests.cpu=1,requests.memory=1Gi,limits.cpu=2,limits.memory=2Gi

kubectl describe quota my-quota -n myteam      # shows Used vs Hard
```

ResourceQuota Manifest

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: myteam
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    persistentvolumeclaims: "10"
    pods: "100"
```

- `kubectl describe quota` shows a **Used vs Hard** table per resource
- The 101st Pod (or one exceeding `requests.cpu`) is **rejected with 403**

11.5 LimitRange

What Is a LimitRange?

A **LimitRange** (`apiVersion: v1`) constrains resource allocations for **individual** Pods, Containers, and PVCs in a namespace.

- Sets **min / max** compute per Pod/Container (and storage per PVC)
- **Auto-injects** `default` (limit) and `defaultRequest` when a container omits them
- Enforces `maxLimitRequestRatio` between limit and request
- Applied by the **LimitRange admission controller** at Pod admission time

Complementary to ResourceQuota (namespace totals) — use them **together**.

LimitRange Manifest

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-mem-constraints
  namespace: myteam
spec:
  limits:
    - type: Container
      default:                # default LIMIT if unset
        cpu: 500m
        memory: 512Mi
      defaultRequest:        # default REQUEST if unset
        cpu: 100m
        memory: 128Mi
      min:                   # smallest allowed per container
        cpu: 100m
      max:                   # largest allowed per container
        cpu: "1"
```

How LimitRange + ResourceQuota Combine

1. A user creates a Pod with **no** `resources` set
2. **LimitRange** injects `requests` / `limits` from its defaults at admission
3. **ResourceQuota** sums the injected requests against the namespace **hard** total
4. If the namespace would go **over quota** → request **rejected with 403**
5. If a container's request falls outside LimitRange `min` / `max` → also **403**

CKA tip: pairing them lets bare Pods get sane defaults *and* still be counted against the namespace quota — neither works as well alone.

Key Takeaways

- **Namespaces** isolate namespaced objects; switch with `kubectl config set-context --current --namespace=`
- **ConfigMap** = non-secret config; consume via `env`, `envFrom`, volume, `subPath`
- **Secret** = base64 (NOT encryption); enable **encryption at rest** + RBAC
- **ResourceQuota** caps a namespace's totals; over-quota requests get **403**
- **LimitRange** sets per-container defaults/bounds and **auto-injects** requests/limits

End of Lesson 11

Next: Lesson 12 — Scheduling

```
kubectl create ns · kubectl create configmap · kubectl create secret generic · kubectl describe quota
```


Managing Scheduling

Kubernetes Production · Lesson 12

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain how the **scheduler** picks a node (filtering + scoring)
- Constrain Pods with **nodeSelector**, labels and **nodeName**
- Use **node affinity** and **inter-pod affinity / anti-affinity**
- Apply **taints & tolerations** to repel or admit Pods
- Set **resource requests & limits** and read **QoS classes**
- Spread Pods evenly with **topology spread constraints**

Reference: *Scheduling, Preemption and Eviction* — kubernetes.io/docs/concepts/scheduling-eviction

12.1 How the Scheduler Works

How the Scheduler Picks a Node

`kube-scheduler` watches for Pods with **no assigned node** and binds each to a suitable one in **two steps**:

Step	What it does
Filtering	Keeps only nodes where it is <i>feasible</i> to run the Pod (resources, taints, affinity) — the <i>feasible</i> set
Scoring	Ranks the feasible nodes and picks the highest score (ties broken at random)

- If the feasible set is **empty**, the Pod stays **Pending**.
- The chosen node is **bound** to the Pod; its kubelet then starts the containers.

CKA tip: `kubectl describe pod` → the **Events** section tells you *why* a Pod is unschedulable (e.g. `0/3 nodes are available: insufficient cpu`).

Ways to Constrain a Pod to Nodes

Kubernetes offers several mechanisms, from simplest to most expressive:

- **nodeSelector** — match a Pod to nodes that carry **all** given labels
- **Node affinity / anti-affinity** — richer rules with operators and weights
- **Inter-pod affinity / anti-affinity** — place Pods relative to **other Pods**
- **nodeName** — bypass the scheduler and pin to one named node
- **Topology spread constraints** — distribute Pods across failure domains

All label-based methods rely on **node labels** — many are populated automatically (e.g. `topology.kubernetes.io/zone` , `kubernetes.io/hostname`).

12.2 nodeSelector & nodeName

Node Labels & nodeSelector

`nodeSelector` is the **simplest** form of node selection: the Pod runs only on nodes that carry **all** the listed labels.

```
# Label a node so Pods can target it
kubectl label nodes node1 disktype=ssd
kubectl get nodes --show-labels
```

```
spec:
  nodeSelector:
    disktype: ssd
```

Labels with the `node-restriction.kubernetes.io/` prefix **cannot** be set by the kubelet — use them (with the `NodeRestriction` admission plugin) for secure node isolation.

nodeName — Direct Assignment

`nodeName` is the most direct selection method — it **bypasses the scheduler** and the kubelet on that node runs the Pod.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  nodeName: node1
  containers:
  - name: nginx
    image: nginx
```

Limitations — the Pod **fails** if:

- the named node **does not exist** or is **unreachable**
- the node **lacks the resources** to run the Pod
- node names are not stable in cloud autoscaling environments

Prefer affinity/nodeSelector over `nodeName` for anything but quick tests.

12.3 Node Affinity

Node Affinity — Two Rule Types

Node affinity is like `nodeSelector` but far more expressive — and it adds **soft** rules. Two flavours:

Rule	Meaning
<code>requiredDuringSchedulingIgnoredDuringExecution</code>	Hard — Pod is <i>not</i> scheduled unless the rule is met
<code>preferredDuringSchedulingIgnoredDuringExecution</code>	Soft — scheduler <i>prefers</i> matching nodes, schedules anyway if none

- **IgnoredDuringExecution**: if labels change *after* scheduling, the running Pod is **not** evicted.
- If both `nodeSelector` **and** `nodeAffinity` are set, **both** must match.

Node Affinity — Example

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values:
                  - antarctica-east1
                  - antarctica-west1
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 1
            preference:
              matchExpressions:
                - key: another-node-label-key
                  operator: In
                  values:
                    - another-node-label-value
```

Operators & Matching Logic

Operator	Matches when the node label...
<code>In</code>	value is in the given list
<code>NotIn</code>	value is not in the list (<i>anti-affinity</i>)
<code>Exists</code>	key exists (no <code>values</code> needed)
<code>DoesNotExist</code>	key is absent (<i>anti-affinity</i>)
<code>Gt</code> / <code>Lt</code>	value is greater / less than (integer comparison)

- Multiple `nodeSelectorTerms` are **OR-ed** (any term may match).
- Multiple `matchExpressions` within a term are **AND-ed** (all must match).
- Soft rules carry a **weight** (**1–100**); the scheduler sums the weights of all satisfied preferences and favours the highest-scoring node.

12.4 Inter-Pod Affinity & Anti-Affinity

Inter-Pod Affinity & Anti-Affinity

These rules place a Pod **relative to other Pods** already running, based on their **labels** — not node labels.

- **podAffinity** — co-locate Pods (e.g. app next to its cache for low latency)
- **podAntiAffinity** — spread Pods apart (e.g. one replica per node for HA)
- Same hard/soft variants: `requiredDuringScheduling...` and `preferred...`
- The **topologyKey** defines the domain that "together" / "apart" means:
`kubernetes.io/hostname` = per node, `topology.kubernetes.io/zone` = per zone

Inter-pod affinity examines labels across many Pods and can **slow scheduling** on large clusters — not recommended beyond a few hundred nodes.

Pod Anti-Affinity — Example

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - web-store
            topologyKey: kubernetes.io/hostname
  containers:
    - name: web-app
      image: nginx
```

- The `labelSelector` picks the Pods to avoid (here `app=web-store`).
- `topologyKey: kubernetes.io/hostname` → at most **one** such Pod per node.

12.5 Taints & Tolerations

Taints & Tolerations

Taints let a node **repel** Pods; **tolerations** let a Pod be admitted despite a taint. They are the **opposite** of affinity.

```
# Add a taint
kubectl taint nodes node1 key1=value1:NoSchedule
# Remove it (trailing minus)
kubectl taint nodes node1 key1=value1:NoSchedule-
```

Effect	Behaviour for non-tolerating Pods
NoSchedule	Not scheduled here; running Pods stay
PreferNoSchedule	Scheduler tries to avoid the node (soft)
NoExecute	Evicted if running, and not scheduled

Tolerations in the Pod Spec

A toleration matches a taint when **key** and **effect** match and the operator is **Equal** (values equal) or **Exists** (any value).

```
tolerations:  
- key: "key1"  
  operator: "Equal"  
  value: "value1"  
  effect: "NoSchedule"  
- key: "key1"  
  operator: "Exists"  
  effect: "NoExecute"  
  tolerationSeconds: 3600
```

- **tolerationSeconds** (NoExecute only): stay bound this long, then evict.
- Empty **key** + **Exists** tolerates **everything**; empty **effect** matches all effects for that key.

The Control-Plane Taint

By default `kubeadm` taints control-plane nodes so **user workloads stay off**:

```
node-role.kubernetes.io/control-plane:NoSchedule
```

- To run a Pod there, add a matching toleration — system add-ons (CoreDNS, kube-proxy) already do.
- On a single-node lab, **remove** the taint to schedule normal Pods:

```
kubectl taint nodes <node> \
node-role.kubernetes.io/control-plane:NoSchedule-
```

Kubernetes also adds **node-condition taints** automatically, e.g.

`node.kubernetes.io/not-ready` and `node.kubernetes.io/unreachable` (NoExecute).

12.6 Resources & QoS

Resource Requests & Limits

Each container can declare **requests** (what it needs) and **limits** (its ceiling).

```
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        cpu: "250m"           # 0.25 core – used for scheduling
        memory: "256Mi"
      limits:
        cpu: "500m"           # throttled above this
        memory: "512Mi"       # OOM-killed if exceeded
```

- **CPU**: cores; **1** = one core, **100m** = 0.1 core (**m** = millicpu).
- **Memory**: bytes; binary suffixes **Ki, Mi, Gi...**, decimal **k, M, G...**.

How Requests Drive Scheduling

- The scheduler places a Pod only on a node where the **sum of requests** of all Pods still **fits the node's allocatable** capacity — it looks at *requests*, not actual live usage.
- A container **may** use more than its request (if the node has spare), but **never** more than its limit.

At runtime the kubelet enforces limits:

- **CPU over limit** → throttled (slowed, not killed).
- **Memory over limit** → container is **OOM-killed**.

CKA tip: a Pod stuck **Pending** with **Insufficient cpu/memory** means **no node has enough unreserved requests** — lower the request or add capacity.

Quality of Service (QoS) Classes

Kubernetes derives a QoS class from a Pod's requests/limits; it drives the **eviction order** under node memory pressure.

QoS class	Condition	Evicted
Guaranteed	Every container has CPU & memory requests = limits (both set, > 0)	Last
Burstable	Not Guaranteed, but at least one request or limit is set	Second
BestEffort	No requests or limits set on any container	First

- View it with `kubectl get pod <name> -o jsonpath='{.status.qosClass}'`.
- Only Pods **exceeding their requests** are prime eviction candidates.

CKA tip: to guarantee a critical Pod, set `requests == limits` for **every** container, for **both** CPU and memory.

12.7 Topology Spread Constraints

Topology Spread Constraints

Spread matching Pods evenly across **failure domains** (zones, nodes) for HA and balanced utilisation.

- **maxSkew** — max allowed difference in Pod count between domains (must be > 0)
- **topologyKey** — node label defining a domain (e.g. `.../zone`, `.../hostname`)
- **whenUnsatisfiable** — `DoNotSchedule` (hard) or `ScheduleAnyway` (soft, best-effort)
- **labelSelector** — which Pods are counted toward the skew

Skew = (Pods in a domain) – (global minimum across eligible domains).

The scheduler keeps it $\leq$ `maxSkew`.

Topology Spread — Example

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: web
  containers:
    - name: web
      image: nginx
```

- Keeps `app=web` Pods within **1** of each other across zones.
- `DoNotSchedule` → leaves the Pod **Pending** rather than break the spread.

Key Takeaways

- The scheduler **filters** then **scores** nodes; unschedulable Pods stay Pending
- **nodeSelector** = simple label match; **nodeName** bypasses the scheduler
- **Node affinity** adds operators (`In/NotIn/Exists...`) and soft `weight` rules
- **Inter-pod (anti-)affinity** places Pods via labels + a **topologyKey**
- **Taints** repel Pods; **tolerations** admit them (`NoSchedule/PreferNoSchedule/NoExecute`)
- **Requests** drive scheduling; **limits** cap usage; QoS = **Guaranteed/Burstable/BestEffort**
- **Topology spread** balances Pods across domains via `maxSkew` + `whenUnsatisfiable`

End of Lesson 12

Next: Lesson 13 — Networking & NetworkPolicy

```
kubectl taint · kubectl label nodes · kubectl get pod -o jsonpath='{.status.qosClass}'
```

</content>

Kubernetes Networking

Kubernetes Production · Lesson 13

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **Kubernetes network model** and its fundamental requirements
- Describe the role of **CNI plugins** (Calico, Cilium, Flannel)
- Explain how **kube-proxy** implements **Services**
- Resolve names with **cluster DNS** (CoreDNS) and its naming scheme
- Write **NetworkPolicy** manifests to isolate and allow traffic

Reference: *Cluster Networking* — kubernetes.io/docs/concepts/cluster-administration/networking

13.1 The Kubernetes Network Model

Four Networking Concerns

Kubernetes networking addresses **four distinct concerns**:

- **Container-to-container** — solved by Pods sharing `localhost`
- **Pod-to-Pod** — the **primary focus** of the network model
- **Pod-to-Service** — stable virtual IPs, covered by Services
- **External-to-Service** — exposing workloads, also covered by Services

Instead of dynamic port allocation, **every Pod gets its own cluster-wide IP address** — you never link Pods or map container ports to host ports.

Fundamental Requirements

The model imposes requirements that hold **without NAT**:

- Every **Pod** gets its own unique, cluster-wide **IP address**
- Pods on a node can communicate with **all Pods on all nodes** without NAT
- **Agents on a node** (kubelet, system daemons) can reach all Pods on that node

This lets Pods be treated much like **VMs or physical hosts** for port allocation, naming, and service discovery.

CKA tip: no source NAT is applied to Pod-to-Pod traffic — a Pod's own IP is the same address other Pods use to reach it.

Non-Overlapping IP Ranges

Clusters must allocate **non-overlapping** IP addresses for three spaces:

Range	Configured by
Pod IPs	Network plugin (CNI)
Service IPs (cluster IPs)	kube-apiserver
Node IPs	kubelet / cloud-controller-manager

- Supports **IPv4-only**, **IPv6-only**, or **dual-stack** clusters
- All components must agree on the configured **primary IP family**
- Overlapping ranges break routing and Service resolution

13.2 CNI Plugins

The Role of CNI

The network model is **implemented by the container runtime** on each node, which uses **Container Network Interface (CNI)** plugins.

- Vanilla Kubernetes ships **no default CNI** — you choose and install one
- The plugin assigns Pod IPs and wires up Pod-to-Pod connectivity
- Capabilities range from **basic** add/remove of interfaces to **advanced IPAM**, multi-plugin chaining, policy, and encryption

Without a working CNI installed, Pods stay in **Pending / ContainerCreating** and CoreDNS Pods never become Ready.

Choosing a CNI → see Lesson 5

Which CNI to run and how to configure it (Calico / Cilium / Flannel / Kube-OVN..., CIDRs, encapsulation) is a **provisioning** decision — covered in **Lesson 5 — Container Runtimes & CNI**.

Here, the only plugin trait that matters is **NetworkPolicy** support:

- **NetworkPolicy is enforced by the CNI** — **Calico, Cilium, Antrea** enforce it
- **Flannel** alone does **not** — policy objects are created but **ignored**

CKA tip: exam clusters come with a CNI installed; you focus on the **model**, **DNS** and **NetworkPolicy** — choosing/installing the CNI is Lesson 5.

13.3 kube-proxy & Services

kube-proxy and Services

A **Service** gives a stable virtual IP (**cluster IP**) and name in front of a changing set of Pods.

- **kube-proxy** runs on each node and watches Services and EndpointSlices
- It programs **network rules** (iptables or IPVS) on the node
- Traffic to a Service's cluster IP is **load-balanced** to backend Pod IPs
- It is **optional** — some CNIs (e.g. Cilium) can replace kube-proxy

Pod IPs are ephemeral; **always reach Pods through a Service**, never by a hard-coded Pod IP.

Service Types (Quick Recap)

Type	Reachable from	Use
ClusterIP	Inside the cluster only	Default; internal apps
NodePort	<NodeIP>:<port>	Simple external access
LoadBalancer	External LB IP	Cloud-provider exposure
ExternalName	DNS CNAME	Alias to an external host

- Pod-to-Service traffic flows through kube-proxy's rules to a backend Pod
- External-to-Service traffic typically enters via NodePort or LoadBalancer

13.4 Cluster DNS (CoreDNS)

Cluster DNS with CoreDNS

CoreDNS runs as a cluster add-on (in `kube-system`) and gives Services and Pods resolvable names.

- The kubelet points every Pod's `/etc/resolv.conf` at the DNS Service
- Default **cluster domain**: `cluster.local`
- Services get **A/AAAA** records; named ports get **SRV** records
- Resolution is namespace-aware via a DNS **search path**

CKA tip: if name resolution fails, check the **CoreDNS Pods** and the `kube-dns` Service in `kube-system`.

The Naming Scheme

A **Service** is resolvable as:

```
my-svc.my-namespace.svc.cluster-domain.example
```

- Same namespace: just `my-svc`
- Cross-namespace: `my-svc.my-namespace`
- Fully qualified: `my-svc.my-namespace.svc.cluster.local`

A **Pod** by IP is resolvable as:

```
172-17-0-3.my-namespace.pod.cluster.local
```

SRV for a named port: `_port-name._port-protocol.my-svc.my-namespace.svc.cluster.local`

DNS Search Path & Lookup

The kubelet writes a search path so short names expand automatically:

```
search <namespace>.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

Verify resolution from a Pod:

```
# Resolve a Service by its FQDN
kubectl run dns-test --rm -it --image=busybox:1.28 --restart=Never -- \
  nslookup nginx.web.svc.cluster.local

# Short name (same namespace) – search path completes it
kubectl exec -it dns-test -- nslookup nginx
```

13.5 NetworkPolicy

NetworkPolicy Basics

NetworkPolicy controls traffic at **L3/L4** (IP / port) for TCP, UDP, SCTP between Pods and other network entities.

- It is a **namespaced** resource (`networking.k8s.io/v1`)
- Requires a **CNI that supports NetworkPolicy** (Calico, Cilium, Antrea...)
- By **default** Pods are non-isolated — **all** ingress and egress is allowed
- A Pod becomes **isolated** for a direction once a policy selects it
- Policies are **additive** — the result is the **union** of matching rules

With Flannel alone, NetworkPolicy objects are **created but not enforced**.

Anatomy of a Policy

Key fields in `spec`:

- **podSelector** — which Pods the policy applies to (empty `{}` = all in ns)
- **policyTypes** — `Ingress`, `Egress`, or both
- **ingress.from / egress.to** — allowed peers via:
 - **podSelector** — Pods in the same namespace
 - **namespaceSelector** — Pods in selected namespaces
 - **ipBlock** — `cidr` with optional `except`
- **ports** — `protocol` + `port` the rule allows

`podSelector` **and** `namespaceSelector` in **one** entry = logical **AND**
(specific Pods in specific namespaces); separate entries = **OR**.

Default-Deny All Ingress

An empty `podSelector` selects **every** Pod in the namespace; with no `ingress` rules, **all** inbound traffic is dropped:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: default
spec:
  podSelector: {}
  policyTypes:
    - Ingress
```

A common pattern: apply **default-deny**, then add explicit **allow** policies.

Allow From a Specific Pod

Permit ingress to `role: db` Pods only from `role: frontend` Pods on TCP `6379`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: frontend
      ports:
        - protocol: TCP
          port: 6379
```

Allow From a Namespace

Permit ingress from **all** Pods in namespaces labeled `project: myproject`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-namespace
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          project: myproject
  ports:
  - protocol: TCP
    port: 6379
```

Restrict Egress

Allow `role: db` Pods to send traffic only to a CIDR block on TCP `5978`:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-cidr
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/24
    ports:
    - protocol: TCP
      port: 5978
```

For a connection to succeed, the **source's egress** *and* the **destination's ingress** must both allow it.

Key Takeaways

- The network model: **every Pod gets a unique cluster-wide IP**; Pod-to-Pod works **without NAT**
- **CNI plugins** (Calico, Cilium, Flannel) implement the network — vanilla K8s ships none by default
- **kube-proxy** programs node rules so **Services** load-balance to Pods
- **CoreDNS** resolves `my-svc.<ns>.svc.cluster.local`; a search path expands short names
- **NetworkPolicy** is namespaced, default-deny-friendly, and **requires a supporting CNI** to be enforced

End of Lesson 13

Next: Lesson 14 — RBAC (Security)

```
kubectl get networkpolicy · kubectl get svc -n kube-system · kubectl exec -it pod -- nslookup <svc>
```

RBAC for Administrators

Kubernetes Production · Lesson 14

Grant (and revoke) the right access — to humans and to apps

kubernetes.io/docs

What an admin actually does with RBAC

This lesson is **task-oriented** — the 4 things you do for real:

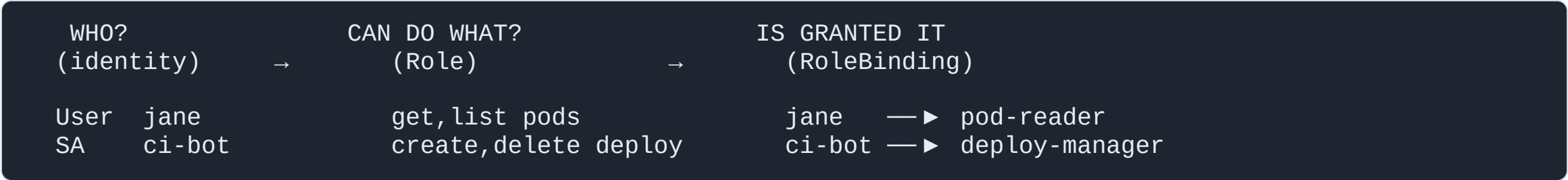
-  **Onboard a human** — give a teammate access to their namespace
-  **Grant an app / CI pipeline access** — via a ServiceAccount
-  **Audit** — answer *"who can do what?"*
-  **Troubleshoot** an `Error ... is forbidden`

A single mental model serves all four. We lay it down first, then apply it.

14.1 The mental model (the only one to remember)

Identity → Role → Binding

An identity can do **nothing** by default. Three building blocks, in this order:



Block	Role	You create it with
Identity	<i>who</i> makes the request	User (certificate) or ServiceAccount
Role	<i>which actions on which resources</i>	<code>kubectl create role</code>
RoleBinding	glues the identity to the Role	<code>kubectl create rolebinding</code>

No RoleBinding = no access. **RBAC has no "deny" rule**: whatever is not granted is refused.

14.2 Identities: human vs app

Two kinds of identity

	 User (human)	 ServiceAccount (app)
For	you, a dev, an external client	a Pod, a CI pipeline, an operator
Object in the cluster	none	a real object (<code>kubectl get sa</code>)
Created by	an external authority (certificate, OIDC)	Kubernetes
Authenticates with	a certificate in the kubeconfig	a token (mounted in the Pod)
Name seen by the API	<code>jane</code>	<code>system:serviceaccount:<ns>:<name></code>

Test question: **"does it run inside a Pod?"** Yes → ServiceAccount.
No (a person) → User.

Creating a User = signing a certificate

There is **no** `kubectl create user`. A human is a **certificate** signed by the cluster; the **CN becomes their name**:

```
openssl genrsa -out jane.key 2048
openssl req -new -key jane.key -out jane.csr -subj "/CN=jane/O=dev-team"
#                                     | O = group
#                                     | CN = user name
```

Submit the CSR to the API, **approve** it, retrieve the signed certificate:

```
kubectl certificate approve jane
kubectl get csr jane -o jsonpath='{.status.certificate}' | base64 -d > jane.crt
```

Full CSR + kubeconfig steps are in the **lab**. Keep the idea:
human = certificate, managed outside Kubernetes.



Creating a ServiceAccount = an object

For an app, no ceremony — one command:

```
kubectl create serviceaccount ci-bot -n dev
```

- The token is **mounted automatically** into Pods that use it:

```
spec:  
  serviceAccountName: ci-bot      # the Pod now acts as ci-bot
```

- From outside (a CI runner), mint a token on demand:

```
kubectl create token ci-bot -n dev --duration=1h
```

Each namespace's **default** SA has **almost no rights** — by design.

The symmetry that simplifies everything

Onboarding a human or an app is **the same recipe**. Only **step 1** changes:

Step	 User jane	 SA ci-bot
1. Identity	a certificate (CN=jane)	<code>kubectl create serviceaccount ci-bot</code>
2. Role	<code>kubectl create role ...</code>	<code>kubectl create role ...</code> <i>(same)</i>
3. Binding	<code>--user=jane</code>	<code>--serviceaccount=dev:ci-bot</code>
4. Verify	<code>--as=jane</code>	<code>--as=system:serviceaccount:dev:ci-bot</code>

Understand **one** of them → you understand both. The RBAC mechanism makes **no** distinction between a human and a robot.

14.3 The Role: what is allowed

Writing a Role

A Role is a list of **rules**: *which verbs on which resources*.

```
kubectl create role pod-reader \
  --verb=get,list,watch --resource=pods -n dev
```

is equivalent to:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
- apiGroups: [""]          # "" = the "core" group: pods, services, configmaps...
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

- Common **verbs**: `get`, `list`, `watch` (read) · `create`, `update`, `patch`, `delete` (write).
- **apiGroups**: `""` for core, `apps` for Deployments, `rbac.authorization.k8s.io` ...

Don't reinvent the wheel: the built-in roles

Kubernetes ships 4 ready-made **ClusterRoles** — reach for them before writing your own:

Built-in ClusterRole	Grants
<code>view</code>	read-only (except Secrets)
<code>edit</code>	create/modify/delete common resources
<code>admin</code>	<code>edit</code> + manage RBAC within the namespace
<code>cluster-admin</code>	everything , everywhere — reserve for <i>break-glass</i>

```
# "jane can edit everything in dev" – without writing a single Role
kubectl create rolebinding jane-edit --clusterrole=edit --user=jane -n dev
```

90% of needs = `view` or `edit` bound in a namespace.

Role vs ClusterRole: just the scope

Same syntax, the only difference is **where** it applies:

In ONE namespace	Across the WHOLE cluster
Role + RoleBinding	ClusterRole + ClusterRoleBinding
"edit in <code>dev</code> "	"read Pods everywhere"
namespaced resources	+ cluster resources (nodes, PV)

Powerful trick: a **RoleBinding** can point at a **ClusterRole**. You write the permission **once** (ClusterRole) and "scope" it to a specific namespace via the RoleBinding.

14.4 The RoleBinding: grant the access

Bind the identity to the Role

The RoleBinding links **subjects** (who) → **roleRef** (which Role):

```
kubectl create rolebinding jane-can-read \
  --role=pod-reader --user=jane -n dev
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: jane-can-read, namespace: dev }
subjects:
- kind: User                # ← or ServiceAccount, or Group
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

roleRef is **immutable**: to change the target Role, delete and recreate the binding.

14.5 Audit & troubleshoot

"Who can do what?" — kubectl auth can-i

The admin's #1 tool. It answers **yes / no**:

```
# Can I?  
kubectl auth can-i create deployments -n dev  
  
# Can SOMEONE ELSE? (--as = impersonation, admin only)  
kubectl auth can-i list pods --as=jane -n dev  
kubectl auth can-i delete pods \  
    --as=system:serviceaccount:dev:ci-bot -n dev  
  
# EVERYTHING this subject is allowed to do  
kubectl auth can-i --list --as=jane -n dev
```

--as makes you "become" someone for a single command — perfect to **verify an access before claiming it works**.

Troubleshooting a "Forbidden"

The error message **already contains** the whole diagnosis:

```
Error from server (Forbidden): pods is forbidden:
User "system:serviceaccount:dev:ci-bot" cannot list resource
"pods" in API group "" in the namespace "dev"
```

Read it as an RBAC checklist:

The message says...	You check...
User "..."	the right identity ? (expected SA?)
cannot **list**	the verb is missing from the Role
resource "pods" in API group ""	the right resource + apiGroup
namespace "dev"	is the binding in this namespace ?

Reproduce it safely: `kubectl auth can-i list pods --as=... -n dev` → `no`
confirms the gap; add the verb/resource to the Role → `yes` .

Least privilege in 4 reflexes

- Prefer a **Role** in one namespace over a cluster-wide grant.
- **Precise verbs and resources** — avoid `verbs: ["*"]` and `resources: ["*"]`.
- An **app = its own dedicated ServiceAccount** (never the `default` SA).
- Reuse `view` / `edit` before inventing a broad role.

 RBAC stops you from **escalating your own privileges**: you can only grant rights you already hold. Keep `cluster-admin` for emergencies.

Key Takeaways

- **Identity** → **Role** → **RoleBinding**. No binding, no access; there is no "deny" rule.
- **User = certificate** (human, outside the cluster) · **ServiceAccount = object** (app, inside a Pod). Same RBAC recipe afterwards — **only step 1 differs**.
- A **Role** lists `verbs` × `resources`; reuse the built-ins `view` / `edit` / `admin`.
- **Role/RoleBinding** = one namespace; **ClusterRole/ClusterRoleBinding** = the whole cluster.
- Audit and debug with `kubectl auth can-i [--as] [--list]`; the `Forbidden` message **is** your checklist.

End of Lesson 14

Next: Lesson 15 — CRDs & Operators

```
kubectl create role · kubectl create rolebinding · kubectl auth can-i --as
```

Lab: onboard `jane` (User) + `ci-bot` (ServiceAccount), audit, fix a `Forbidden`.

CRDs & Operators

Kubernetes Production · Lesson 15

Built from the official Kubernetes documentation

`kubernetes.io/docs`

Learning Objectives

By the end of this lesson you will be able to:

- Explain that the Kubernetes API is **declarative** and **extensible** (**CRI / CNI / CSI**)
- Define a **custom resource** and when to use one vs a ConfigMap
- **Read** a **CustomResourceDefinition** and use its custom resources (`get` / `explain`)
- Describe the **controller control loop** and the **Operator pattern**
- **Operate** an operator: install it, drive it via CRs, read its `.status`
- Manage the CR lifecycle: **ownerReferences / GC** and **finalizers** (stuck deletions)
- **Troubleshoot** a misbehaving operator (controller logs, webhooks, RBAC)

Reference: *Extending Kubernetes; Custom Resources; Operator pattern* — kubernetes.io/docs

15.1 Extending Kubernetes

A Declarative, Extensible API

Kubernetes is built around a **declarative API**: you describe **desired state** in objects, and **controllers** continuously reconcile reality to match it.

The platform is designed to be **extended** without forking Kubernetes:

- **Client extensions** — `kubectl` / credential plugins
- **API-access** — authentication, authorization, **admission webhooks**
- **API additions** — **CRDs** and the **aggregation layer**
- **Infrastructure** — **CRI**, **CNI**, **CSI**, device plugins
- **Scheduling** — custom schedulers / extenders
- **Controllers** — watch state and reconcile

This lesson focuses on the **API-additions** point: **custom resources**.

The Extension Points

Extension point	What it customises
kubectl / credential plugins	Client subcommands, auth
Authn / Authz	Who calls the API, and what they may do
Admission webhooks	Block, mutate, or validate requests
CRDs / aggregation layer	Add new API kinds
CRI	Container runtime interface
CNI	Pod network interface
CSI	Container storage interface
Scheduling / controllers	Placement and reconciliation

The **webhook** and **controller** patterns recur across many of these points.

Infrastructure Interfaces: CRI / CNI / CSI

The CKA expects you to **recognise** the three infrastructure interfaces — stable APIs that let a cluster swap implementations:

- **CRI — Container Runtime Interface**: how the **kubelet** talks to the container runtime (e.g. **containerd**, **CRI-O**)
- **CNI — Container Network Interface**: pluggable **Pod networking** (e.g. **Calico**, **Cilium**, **Flannel**)
- **CSI — Container Storage Interface**: pluggable **storage backends** mounted into Pods

Each is an **interface** behind which vendors provide plugins — the control plane stays the same while runtime / network / storage are swapped.

15.2 Custom Resources

What Is a Custom Resource?

A **custom resource** is *"an extension of the Kubernetes API that is not necessarily available in a default Kubernetes installation."*

- It is an **endpoint in the Kubernetes API** that stores a collection of objects
- It can **appear and disappear dynamically** in a running cluster
- You use it with `kubectl` **just like built-in resources** (Pods, Deployments)

On its own, a custom resource only **stores structured data**. Combined with a **custom controller**, it becomes a true **declarative API** that *acts*.

- Caution: don't use a custom resource as **data storage** for application or end-user data — that couples your app too tightly to the API server.

ConfigMap vs Custom Resource

Use a ConfigMap when...	Use a Custom Resource when...
There is an existing config-file format (e.g. <code>mysql.cnf</code>)	You want top-level <code>kubectl get <kind></code> support
A program in a Pod reads the file to configure itself	You want API conventions: <code>.spec</code> , <code>.status</code> , <code>.metadata</code>
Consumers prefer file/env access	You want automation that watches and reconciles
Rolling updates come from the Deployment	The object abstracts a collection of controlled resources

Reach for a **custom resource** when the object is a first-class **API** that something should act on — not just a config blob.

15.3 Two Ways to Add a Custom API

CRD vs API Aggregation

	CustomResourceDefinition	API Aggregation
Programming	None — declarative YAML	Required — build an apiserver
What runs	The existing kube-apiserver	A subordinate apiserver behind a proxy
Flexibility	Less (generic impl.)	More — storage, version conversion
Typical use	The common path	Complex APIs needing fine control

"Kubernetes provides these two options ... so that neither ease of use nor flexibility is compromised."

- **CRD is the no-code, common path** — and the focus of the CKA.

15.4 CustomResourceDefinition

The CRD — the essentials

A **CustomResourceDefinition** registers a **new kind**; the apiserver then serves it like any built-in.

- `apiVersion: apiextensions.k8s.io/v1` · `kind: CustomResourceDefinition`
- `metadata.name` **must** be `<plural>.<group>` (`crontabs.stable.example.com`)
- `spec.scope`: `Namespaced` or `Cluster` — but **the CRD object is always cluster-scoped**
- `spec.versions[]`: each is `served` (reachable) + **exactly one** `storage: true`
- `spec.names`: `plural` / `singular` / `kind` (CamelCase) / `shortNames`
- `spec.versions[].schema.openAPIV3Schema`: a **structural schema** (mandatory in v1)

As an admin you mostly **receive** CRDs (installed by an operator) — you rarely hand-write the schema. Knowing the fields is enough to **read and debug** one.

A CRD by example (CronTab)

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: crontabs.stable.example.com      # <plural>.<group>
spec:
  group: stable.example.com
  scope: Namespaced
  names: { plural: crontabs, singular: crontab, kind: CronTab, shortNames: ["ct"] }
  versions:
    - name: v1
      served: true
      storage: true                      # exactly one storage version
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties: { cronSpec: {type: string}, image: {type: string} }

```

Optional enrichments: **additionalPrinterColumns** (extra `kubectl get` columns) and **subresources** — `status` (controller writes status) and `scale` (lets `kubectl scale` / the **HPA** drive the CR).

Create & discover custom resources

A custom resource is just YAML with `<group>/<version>` + the `kind`:

```
apiVersion: stable.example.com/v1
kind: CronTab
metadata: { name: my-cron }
spec: { cronSpec: "* /5 * * * *", image: my-cron-image }
```

The **admin's discovery workflow** on an unfamiliar cluster:

```
kubectl get crd                # what extensions are installed?
kubectl api-resources | grep stable # which kinds does this group add?
kubectl explain crontab.spec    # what fields does the kind accept?
kubectl get crontabs -A         # the actual objects, all namespaces
```

CKA flow: **get crd** → **api-resources** → **explain** — read any custom API you land on without prior knowledge.

15.5 Controllers & Operators

The Controller Control Loop

Every Kubernetes controller runs a **control loop**:

1. **Observe** — read the object's `.spec` (desired state)
2. **Diff** — compare desired state with **actual** cluster state
3. **Act** — make changes to reconcile, then update `.status`



This loop is the **core pattern** of Kubernetes — built-ins and custom controllers work the same way.

The Operator Pattern

Operators are *"software extensions to Kubernetes that make use of custom resources to manage applications and their components" and "follow Kubernetes principles, notably the control loop."*

Operator = CRD(s) + a custom controller

- The **CRD** defines the API (e.g. a `SampleDB` kind)
- The **controller** runs the loop and **encodes operational knowledge** — how a human operator would deploy, back up, upgrade and recover the app
- The controller usually runs **outside the control plane**, as a **Deployment**

You declare *what you want* (a custom resource); the operator makes reality match and keeps it healthy.

What Operators Automate

Operators capture the expertise of a human operator. From the docs, they can:

- **Deploy** an application on demand
- **Take and restore backups** of application state
- **Handle upgrades** — including schema and config changes
- **Publish services** for apps that don't speak the Kubernetes API
- **Simulate failure** to test resilience
- Manage **leader election** for distributed apps

Real examples: **Prometheus Operator**, **etcd** / database operators (PostgreSQL, MySQL), **cert-manager**.

Installing an Operator

*"The most common way to deploy an operator is to add the **CRD** and its **Controller** to the cluster."* Via **manifests**, a **Helm chart** (see Lesson 19), or **OLM / OperatorHub** — installation brings in:

- the **CRDs** (the new kinds)
- the **controller Deployment** (runs the reconcile loop, usually in its own namespace)
- the **RBAC** it needs (ServiceAccount + ClusterRole/Role bindings)

OLM (Operator Lifecycle Manager) + **OperatorHub.io** are how operators are catalogued, installed and upgraded in production — *a package manager for operators.*

Operating an operator, day-to-day

You don't touch the objects it manages — you **edit the custom resource** and let it reconcile:

```
kubectl get sampledb -A           # the instances it manages
kubectl describe sampledb/example -n prod # spec + status + Events
kubectl edit sampledb/example -n prod    # change desired state → it reconciles
kubectl get sampledb/example -o jsonpath='{.status}' # what the controller reports
```

 **Golden rule:** change the **CR**, never the Deployments/Services/PVCs the operator created — the control loop will just revert them to match the CR.

ownerReferences & garbage collection

An operator stamps **ownerReferences** on everything it creates, pointing back to the CR — this wires up **cascade deletion**:

```
metadata:
  ownerReferences:
  - apiVersion: demo.example.com/v1
    kind: Widget
    name: w1
    uid: <widget-uid>           # the owner's UID
```

- Delete the **CR** → Kubernetes **garbage-collects** every owned object automatically.
- Cascade modes: **background** (default), **foreground** (`--cascade=foreground`), **orphan** (`--cascade=orphan` , keep the children).

This is why deleting one custom resource can tear down a whole app — the operator **owns** the pieces.

Finalizers & stuck deletions (the classic incident)

A **finalizer** is a string that **blocks deletion** until a controller runs cleanup (drain, deregister, delete cloud resources) and removes it:

```
$ kubectl delete widget w2          # hangs — the object goes to "Terminating"
$ kubectl get widget w2 -o jsonpath='{.metadata.deletionTimestamp} {.metadata.finalizers}'
2026-...Z ["demo.example.com/cleanup"] # marked for deletion, but still there
```

⚠ If the operator is **down or uninstalled**, nothing removes the finalizer → the CR (or its **namespace**) is **stuck Terminating forever**. Last-resort unblock:

```
kubectl patch widget w2 --type=merge -p '{"metadata":{"finalizers":null}}'
```

Prefer fixing the controller so cleanup actually runs; strip the finalizer **only** when the operator is gone for good.

Troubleshooting operators

Symptom	Where to look
CR ignored, empty <code>.status</code>	the controller Deployment — Running? <code>kubectl logs</code> it
Every CR create/edit fails	an admission / conversion webhook is down (operator offline)
CR stuck <code>Terminating</code>	a finalizer + a dead controller (previous slide)
<code>Forbidden</code> in controller logs	the operator's RBAC lacks a verb/resource (Lesson 14)

```
kubectl -n <op-ns> logs deploy/<operator-controller> --tail=50
kubectl describe <kind>/<name> # Events explain most reconcile failures
kubectl get crd <name> -o yaml | grep -A5 conditions
```

Reconcile problems are **almost always** in the controller's logs or the CR's **Events** — start there, not in the managed objects.

Key Takeaways

- The Kubernetes API is **declarative** and **extensible**; **CRI / CNI / CSI** are the infrastructure interfaces (runtime / network / storage)
- A **custom resource** extends the API (used with `kubectl` like built-ins); alone it stores data — a **controller** makes it act. CRD = `apiextensions.k8s.io/v1`, named `<plural>.<group>`, **cluster-scoped**, **one** `storage` version
- **Operator = CRD(s) + controller** running **observe** → **diff** → **act**; installing brings **CRDs + Deployment + RBAC** (manifests / Helm / **OLM**)
- **Operate by editing the CR**, never the managed objects — they get reverted
- Operators wire **ownerReferences** (cascade **GC**) and **finalizers** (cleanup); a dead controller + finalizer = **stuck** `Terminating` → patch finalizers as last resort
- **Debug** via the controller's **logs** and the CR's **Events**

End of Lesson 15

Next: Lesson 16 — Node Maintenance & Scaling

```
kubectl get crd · kubectl api-resources · kubectl explain <kind> · kubectl apply -f crd.yaml
```

Node Maintenance & Scaling

Kubernetes Production · Lesson 16

Universal `kubectl` maintenance + Kubespray Day-2 scaling

`kubernetes.io/docs` · `kubespray.io`

Learning Objectives

By the end of this lesson you will be able to:

- Take a node out of service with **cordon / drain / uncordon**
- Use drain flags `--ignore-daemonsets`, `--delete-emptydir-data`, `--force`
- Protect availability during eviction with a **PodDisruptionBudget**
- Explain **graceful termination** and `terminationGracePeriodSeconds`
- Add a worker on Kubespray with `scale.yml` (+ `--limit`)
- Change cluster membership: `remove-node.yml`, control-plane & etcd changes

Framing: `drain` is *how you safely evict*; Kubespray's `scale.yml` / `remove-node.yml` are *how you change cluster membership*.

16.1 Cordon, Drain, Uncordon

Cordon, Drain, Uncordon

The three verbs of node maintenance — **universal**, work on any cluster.

Command	What it does
<code>kubectl cordon <node></code>	Marks the node unschedulable (<code>node.kubernetes.io/unschedulable</code> taint). No new Pods land; existing Pods stay.
<code>kubectl drain <node></code>	Safely evicts eligible Pods with graceful termination, respecting PDBs . Auto-cordons the node first.
<code>kubectl uncordon <node></code>	Re-enables scheduling after maintenance.

- You rarely call `cordon` alone — `drain` cordons for you.
- Once `drain` returns **without errors**, it is safe to power down or delete the node.

Always **uncordon** when maintenance is finished, or the node stays empty.

Draining a Node

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

Flag	Effect
<code>--ignore-daemonsets</code>	Proceed despite DaemonSet Pods. Required when any exist — they're re-created instantly and tolerate the taint, so they can't be evicted. Without it, drain fails .
<code>--delete-emptydir-data</code>	Allow eviction of Pods using local <code>emptyDir</code> storage (that data is lost).
<code>--force</code>	Evict/delete bare Pods not managed by a controller. They won't be rescheduled — use with care.

`--force` does **not** bypass graceful termination — `terminationGracePeriodSeconds` still applies to every Pod.

What Drain Evicts and Respects

- **Evicted** — regular app Pods and Pods owned by **Deployments, ReplicaSets, StatefulSets, Jobs**. Their controllers reschedule replacements on other nodes.
- **Ignored (not moved):**
 - **DaemonSet** Pods (with `--ignore-daemonsets`).
 - **Mirror / static Pods** — bound to the node via `nodeName`.
 - Any Pod with `nodeName` **set directly** — it keeps running on the drained node.

```
# Verify the node is empty of evictable Pods afterwards
kubectl get pods -A -o wide --field-selector spec.nodeName=<node-name>
```

CKA tip: if drain **hangs**, a **PodDisruptionBudget** is blocking it — drain *waits* rather than break the budget (next section).

16.2 Graceful Termination

Graceful Termination

When a Pod is evicted or deleted, the kubelet shuts it down in **stages**:

1. Send **SIGTERM** to the container's main process.
2. Wait up to **terminationGracePeriodSeconds** (default **30s**).
3. If still running, send **SIGKILL** (forceful kill).

```
spec:
  terminationGracePeriodSeconds: 60
  containers:
  - name: app
    image: nginx
    lifecycle:
      preStop:
        exec:
          command: ["sh", "-c", "nginx -s quit; sleep 5"]
```

- The container should **catch SIGTERM** to finish in-flight work, flush, close.
- A **preStop** hook runs **before** SIGTERM — useful to drain LB connections.

16.3 PodDisruptionBudget

PodDisruptionBudget (PDB)

A **PDB** caps how many Pods of a set may be down at once; drain's **Eviction API** honors it.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2           # or: maxUnavailable: 1
  selector:
    matchLabels:
      app: web
```

- Set **minAvailable** or **maxUnavailable** (number or percentage).
- An eviction that would breach the budget is **blocked**; drain **waits**.
- Guards **voluntary** disruptions only (drain, Eviction API) — **not** a node crash, kernel panic or hardware failure.

PDB During Drain & Concurrent Drains

- During `drain`, evictions go through the **Eviction API** (`/eviction` subresource), which checks the PDB before removing each Pod.
- Drain **waits** for evicted Pods to come back healthy elsewhere before evicting the next — keeping the budget satisfied throughout.
- **Concurrent drains** of several nodes still respect each PDB **collectively**:

A StatefulSet of **3** replicas with `minAvailable: 2` lets only **1** Pod be evicted at a time — across **all** running drains combined.

- The Eviction API gives this PDB-respecting behavior **programmatically**, without `kubect1 drain`.

16.4 Kubespray — Adding a Worker

Adding a Worker — `scale.yml`

On Kubespray you change membership by editing the **inventory** and running the **matching playbook** — idempotent, touches only what changed.

```
# 0. Refresh facts for ALL nodes first (avoids stale-facts bugs with --limit)
ansible-playbook -i inventory/mycluster/hosts.yaml facts.yml

# 1. Add the new host to the inventory, then:
ansible-playbook -i inventory/mycluster/hosts.yaml scale.yml --limit=node4
```

- `scale.yml` is the **additive worker path** — joins new workers without re-running the whole cluster.
- `--limit=NODE_NAME` scopes the run to the new node only.
- **Facts caveat:** run `facts.yml` without `--limit` first, or the limited run works from **stale facts** about the rest of the cluster.

You **cannot** use `scale.yml` for a **control-plane** node (next section).

16.5 Kubespray — Control-Plane Changes

Adding a Control-Plane Node — `cluster.yml`

A control-plane node is **not** added with `scale.yml`.

```
# Append the new host to the END of kube_control_plane, then:  
ansible-playbook -i inventory/mycluster/hosts.yaml cluster.yml
```

- **Always append** new control-plane nodes to the **end** of the `kube_control_plane` group. Adding one in the **first** position is **not supported** and the playbook **fails**.
- After the run, **restart `nginx-proxy` on every host** so it reloads config:

```
crictl ps | grep nginx-proxy | awk '{print $1}' | xargs crictl stop
```

`cluster.yml` re-issues apiserver/cert config across the control plane, wiring the new member into certs, kubeconfig and the apiserver SAN/endpoint list.

etcd Members & the First-Node Caveat

- **Keep etcd odd** (3, 5, ...) for quorum — adding etcd is a **replace** or a **scale-up by two**. Add via `cluster.yml` + `upgrade-cluster.yml` with `--limit=etcd,kube_control_plane -e ignore_assert_errors=yes`, then add the new member to `--etcd-servers=...` in each `kube-apiserver.yml`.
- **First-node caveat** — you **cannot directly remove the first** node in `kube_control_plane` / etcd:

1. **Reorder** the inventory so the first entry moves back (e.g. `node-2`, `node-3`, `node-1`).
2. Run `cluster.yml` / `upgrade-cluster.yml` to apply the new ordering.
3. *Then* remove the old first node with `remove-node.yml`.

16.6 Kubespray — Removing a Node

remove-node.yml — Cordon, Drain, Reset

The single entry point for removing **any** node type — worker, control-plane or etcd.

```
# Node still in the inventory; drains + resets it:
ansible-playbook -i inventory/mycluster/hosts.yaml remove-node.yml \
  -e node=node4

# Offline / unreachable node — skip the on-node reset:
ansible-playbook -i inventory/mycluster/hosts.yaml remove-node.yml \
  -e node=node4 -e reset_nodes=false -e allow_ungraceful_removal=true
```

- By default it **cordons** → **drains** → **resets** (wipes the K8s install/config).
- **-e node=NODE_NAME** is mandatory; comma-separate multiple (**node4,node5**).
- Run it **while the node is still in the inventory**, *then* delete it from the inventory.

Kubespray's playbooks **wrap the same kubectl drain** you run by hand — safe eviction underpins every membership change.

Key Takeaways

- **cordon / drain / uncordon** = take a node out and back; always **uncordon** after
- `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data` is the canonical command
- `--ignore-daemonsets` required if any DaemonSet Pods exist; `--force` only for bare Pods
- **Graceful termination** = SIGTERM → `terminationGracePeriodSeconds` (30s) → SIGKILL
- A **PDB** blocks evictions that breach `minAvailable` / `maxUnavailable`; drain **waits**
- Kubespray worker → `scale.yml` (+ `--limit`, prior `facts.yml`); control-plane → `cluster.yml`
- Any removal → `remove-node.yml -e node=NODE_NAME` (drains + resets); append CP nodes to the **end**

End of Lesson 16

Next: Lesson 17 — Upgrades

```
kubectl drain --ignore-daemonsets --delete-emptydir-data · kubectl uncordon · scale.yml --limit ·  
remove-node.yml -e node=
```


Upgrades

Kubernetes Production · Lesson 17

Graceful Kubespray upgrades + universal version-skew rules

kubespray.io · kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Run a **graceful, rolling upgrade** with `upgrade-cluster.yml` (cordon → drain → uncordon)
- Follow the workflow: **checkout matching tag** → **bump kube_version** → **run** → **verify**
- Apply the **version-skew rule** — one minor at a time, control plane before workers
- Control the roll-out with `serial`, `--limit`, and the `upgrade_node_*` vars
- Recognise the **unsafe** `cluster.yml -e upgrade_cluster_setup=true` path
- Explain the **bridge**: Kubespray runs `kubeadm upgrade` under the hood

Reference: *Upgrading Kubernetes in Kubespray* — kubespray.io · and
Upgrading kubeadm clusters — kubernetes.io/docs

17.1 Graceful Upgrades with `upgrade-cluster.yml`

The Graceful Upgrade Playbook

`upgrade-cluster.yml` (note the **exact hyphenated name**) is the **graceful, serial** upgrade path for a running Kubespray cluster.

- Unlike `cluster.yml`, it **cordons** → **drains** → **uncordons** each node during a cluster upgrade, so workloads are evacuated **one batch at a time** instead of all at once.
- Requires **at least one** `kube_control_plane` **already deployed** — it upgrades an existing cluster, it does **not** bootstrap one.
- Driven by **one declarative knob**: `kube_version`, the target Kubernetes version.

`cluster.yml` re-applies state but does **not** drain — only
`upgrade-cluster.yml` evacuates workloads safely batch by batch.

Cordon / Drain / Uncordon — One Batch at a Time

For every node it touches, the playbook performs the safe maintenance cycle:

Step	What happens
Cordon	Mark the node unschedulable — no new Pods land
Drain	Evict workloads gracefully, respecting PodDisruptionBudgets
Upgrade	Bump packages, run <code>kubeadm upgrade</code> , restart the kubelet
Uncordon	Re-enable scheduling once the node is back on the new version

- The default batch size is **20%** of available nodes; tune it with `serial`.
- This is the **same cordon/drain you learned in CKA node maintenance** — Kubespray just orchestrates it across the whole fleet.

17.2 The Required Workflow

Workflow: tag → kube_version → run → verify

The upgrade is always four ordered steps:

1. **Check out the matching Kubespray release tag** — one tag at a time.
2. **Bump kube_version** in `group_vars/k8s_cluster/k8s-cluster.yml` (or pass `-e kube_version=vX.Y.Z`).
3. **Run** the graceful playbook.
4. **Verify** between steps.

```
ansible-playbook upgrade-cluster.yml -b \  
-i inventory/sample/inventory.ini \  
-e kube_version=v1.31.4
```

```
kubectl version          # control plane on the new version?  
kubectl get nodes        # every node Ready on the new build?
```

Check Out the Tag, Bump the Version

The **Kubespray tag and the Kubernetes version are coupled** — each tag targets a specific K8s range, so move **one tag at a time**.

```
# 1. Match the Kubespray release tag to your target step
git checkout v2.24.0
```

```
# 2. group_vars/k8s_cluster/k8s-cluster.yml
kube_version: v1.31.4
```

Variable **formats and locations change between Kubespray releases** — diff the upstream sample inventory for the new tag **before** running, or the upgrade can fail on drifted vars.

17.3 Version-Skew: Never Skip a Minor

One Minor at a Time, Control Plane First

This rule is **identical** in the CKA kubeadm course and the Kubespray docs — it is a property of **Kubernetes/kubeadm**, not of any one tool.

- **Upgrade one minor version at a time; never skip a minor.** Patch jumps within a minor are fine.
- **Order: control plane first, then workers** — primary control plane → additional control planes → worker nodes.

Kubespray tags	Kubernetes minors
v2.22 → v2.23 → v2.24 ✓	1.30 → 1.31 → 1.32 ✓
v2.22 → v2.24 ✗ (skips a minor)	1.30 → 1.32 ✗ (skips a minor)

"Upgrading from an older release straight to the latest is unsupported and likely to break something." Kubespray respects kubeadm's version-skew rules — because it **is** kubeadm underneath.

17.4 Controlling the Roll-out

serial · --limit · upgrade_node_*

Control	Effect
(default)	Upgrade in batches of 20% of available nodes
-e serial=1	Upgrade N nodes at a time (here, one — safest)
--limit "node4:node6"	Restrict to specific nodes only
upgrade_node_confirm: true	Manual approval before each node
upgrade_node_pause_seconds: 60	Automatic pause before each node
upgrade_node_post_upgrade_confirm: true	Manual approval after each node
upgrade_node_post_upgrade_pause_seconds: 60	Automatic pause after each node

The `upgrade_node_*` confirm/pause vars let you gate each node with a human checkpoint or a fixed delay — invaluable for cautious production roll-outs.

serial=1 and --limit Examples

```
# One node at a time (safest, slowest)
ansible-playbook upgrade-cluster.yml -b \
  -i inventory/sample/inventory.ini \
  -e kube_version=v1.31.4 -e "serial=1"
```

```
# Refresh facts first, then upgrade only named nodes
ansible-playbook playbooks/facts.yml -b \
  -i inventory/sample/inventory.ini
ansible-playbook upgrade-cluster.yml -b \
  -i inventory/sample/inventory.ini \
  -e kube_version=v1.31.4 --limit "node4:node6:node7:node12"
```

Always run `playbooks/facts.yml` before a `--limit` run so Ansible has **fresh facts** for the whole cluster and does not act on stale data.

17.5 Graceful vs. Unsafe

The Unsafe Non-Graceful Path

There is a faster but **disruptive** alternative — know it so you avoid it.

```
# UNSAFE: bypasses graceful cordon/drain
ansible-playbook cluster.yml -b \
  -i inventory/sample/inventory.ini \
  -e kube_version=v1.31.4 \
  -e upgrade_cluster_setup=true
```

Path	Behaviour
<code>upgrade-cluster.yml</code> (recommended)	Cordon → drain → uncordon, one batch at a time
<code>cluster.yml -e upgrade_cluster_setup=true</code>	Migrates control plane immediately, no drain → workload disruption

Use the unsafe path **only** when you can tolerate disruption. For every production version bump, use `upgrade-cluster.yml`.

17.6 Under the Hood — the kubeadm Bridge

Kubespray Runs `kubeadm upgrade` Per Node

Per node, the graceful playbook does **exactly** what the manual CKA procedure does — same kubeadm mechanics, driven declaratively:

1. **cordon + drain** the node (respecting PodDisruptionBudgets)
2. upgrade the kubeadm/kubelet packages
3. run `kubeadm upgrade apply vX.Y.Z` on the first control plane, `kubeadm upgrade node` everywhere else
4. **restart the kubelet**, then **uncordon**

So the CKA *"kubeadm upgrade"* knowledge **maps directly here** — one minor at a time, control plane first, drain during the per-node upgrade, certs auto-renewed on `kubeadm upgrade`. Kubespray just runs it across the fleet from one `kube_version` change.

Key Takeaways

- `upgrade-cluster.yml` = graceful, serial upgrade — cordon → drain → uncordon, one batch at a time
- Workflow: checkout matching tag → bump `kube_version` → run → verify
- **Never skip a minor; control plane before workers** — `v2.22 → v2.23 → v2.24`, `1.30 → 1.31 → 1.32`
- Tune the roll-out with `serial` (default 20%), `--limit`, and the `upgrade_node_*` confirm/pause vars
- Run `playbooks/facts.yml` before any `--limit` run; diff the inventory between tags
- `cluster.yml -e upgrade_cluster_setup=true` is the **unsafe** non-graceful path — no drain
- Bridge: Kubespray runs `kubeadm upgrade` per node — same kubeadm mechanics, declarative

End of Lesson 17

Next: Lesson 18 — Backup, Recovery & Reset

```
git checkout v2.24.0 · upgrade-cluster.yml -e kube_version=vX.Y.Z · -e serial=1 · --limit
```

Backup, Recovery & Reset

Kubernetes Production · Lesson 18

The manual etcd skill (CKA) + Kubespray automation

`kubernetes.io/docs` · `kubespray.io`

Learning Objectives

By the end of this lesson you will be able to:

- Explain why **etcd is the source of truth** and why a **snapshot is your backup**.
- Take an etcd **snapshot** with `etcdctl` (v3 API + TLS certs) and verify it with `snapshot status`.
- **Restore** a snapshot into a **new data directory**, moving static-pod manifests aside to stop etcd/apiserver during the restore.
- Recognize Kubespray's `host` etcd deployment (certs in `/etc/ssl/etcd/ssl/`, client `2379`, metrics `2381`).
- Recover a broken control plane with `recover-control-plane.yml` and tear a cluster down with `reset.yml`.
- Enable **certificate auto-renewal** and **encryption of Secrets at rest**.

18.1 Why Back Up etcd?

etcd is the Source of Truth

- **etcd is the backing store for ALL cluster data** — Deployments, Secrets, ConfigMaps, RBAC, every object the API server serves.
- Access to etcd is **equivalent to root on the cluster**. Lose etcd → **lose the cluster state**.
- The backup primitive is a **point-in-time snapshot**; the restore primitive writes it into a **fresh, empty data directory** as a new etcd member.
- Two layers we cover:
 - **Manual `etcdctl`** — the universal, CKA-exam skill (any kubeadm/host cluster).
 - **Kubespray playbooks** — automate the same workflow at cluster scale.

Treat snapshots as **disaster-recovery backups**: take them on a schedule and **copy them off the node**.

18.2 Manual etcd Backup with etcdctl

Taking a Snapshot

Use `etcdctl` with the **v3 API** and TLS client certs. On a **kubeadm** cluster the certs are under `/etc/kubernetes/pki/etcd/`.

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /opt/snapshot.db
```

Flag	Meaning
<code>--endpoints</code>	etcd client endpoint(s) — client port 2379
<code>--cacert</code>	CA cert to verify the server
<code>--cert</code> / <code>--key</code>	client certificate and key (mutual TLS)

Verifying the Snapshot

```
ETCDCTL_API=3 etcdctl snapshot status /opt/snapshot.db --write-out=table
```

- Reports **hash**, **revision**, **total keys**, and **db size** — confirming the snapshot is **readable** and not obviously truncated.
- **Important:** `snapshot status` is a **sanity check, not a full integrity guarantee**. It shows the file parses and prints its metadata — it does **not** prove the backup will restore cleanly.
- Still, **always run it right after a save** — it catches the obvious failures.

18.3 Manual etcd Restore

Restore Prerequisites

1. **Stop all etcd members** and **all kube-apiserver instances** for the duration of the restore.
2. Restore must go into a **NEW, empty data directory** — **never** overwrite the live one in place.

Move static-pod manifests aside. On a static-pod control plane, moving `etcd.yaml` (and `kube-apiserver.yaml`) **out** of `/etc/kubernetes/manifests/` stops those static Pods cleanly; moving them **back** restarts them. That is how you bounce etcd/apiserver around a restore.

Restore to a New Data Dir

```
ETCDCTL_API=3 etcdctl snapshot restore /opt/snapshot.db \  
--data-dir=/var/lib/etcd-restore
```

- Writes the snapshot's data into the **new** directory as a **fresh etcd member**.
- The live data directory is left **untouched** — if something goes wrong you can start over from the snapshot.

Point etcd at the Restored Data

1. Edit the etcd **static Pod manifest** `/etc/kubernetes/manifests/etcd.yaml`.
2. Update the **hostPath** volume for the etcd data dir to `/var/lib/etcd-restore` (and the `--data-dir` flag to match).
3. Move the manifests **back** → the **kubelet recreates** etcd against the restored data.
4. **Restart the kube-apiservers** and confirm cluster state with `kubectl get nodes` / `kubectl get pods -A`.

18.4 Kubespray etcd Deployment

Kubespary Runs etcd on the Host

- Kubespray's default is `etcd_deployment_type: host` — etcd runs as a **systemd service** on the `etcd` group nodes.
 - Other options: `kubeadm` (static pod) or `docker`.
- Client TLS material lives under `/etc/ssl/etcd/ssl/` — **not** `/etc/kubernetes/pki/etcd/`.
- Client port `2379`; metrics on `etcd_metrics_port: 2381`. Backups use the **client** port.

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd_snapshot \  
--endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/ssl/etcd/ssl/ca.pem \  
--cert=/etc/ssl/etcd/ssl/member-<node>.pem \  
--key=/etc/ssl/etcd/ssl/member-<node>-key.pem
```


18.5 Recovering a Broken Control Plane

Recovery Runbook

Use `recover-control-plane.yml` for hardware failure, failure during patch/upgrade, **etcd corruption**, or any degraded control plane.

Prerequisite: you need **at least one functional node** to recover.

- Back up what you can; provision new nodes for the broken ones.
- Put broken etcd nodes in the `broken_etcd` group (set `etcd_member_name`).
- Put broken control-plane nodes in `broken_kube_control_plane`.
- List the **surviving** nodes **first** in the `etcd` / `kube_control_plane` groups; add the **new** nodes **below** them.

Run recover-control-plane.yml

Scope the run to the control-plane/etcd groups and raise the etcd retry count (the right value is hard to predict — go larger if needed):

```
ansible-playbook -i inventory/mycluster/hosts.yaml \
  recover-control-plane.yml \
  --limit etcd,kube_control_plane \
  -e etcd_retries=10
```

- `--limit etcd,kube_control_plane` targets exactly those groups — the broken member plus a healthy node, so the playbook rebuilds from a working quorum.

Recover from Lost Quorum

The playbook **detects whether etcd quorum is intact**. If quorum is **lost**, it takes a snapshot from the **first node in the etcd group** and restores from it.

Pass a path to restore a **specific** snapshot:

```
ansible-playbook -i inventory/mycluster/hosts.yaml \
  recover-control-plane.yml \
  --limit etcd,kube_control_plane \
  -e etcd_retries=10 \
  -e etcd_snapshot=/tmp/etcd_snapshot
```

Caveats (docs): only tested with **small etcd DBs**; **may cause disruptions**; **no guarantees**. Practice on a test cluster first.

18.6 Tearing Down with reset.yml

reset.yml — Destructive Inverse of cluster.yml

`reset.yml` removes Kubernetes from the target nodes — the **inverse of `cluster.yml`**. Useful for redeploying from scratch or decommissioning.

```
ansible-playbook -i inventory/mycluster/hosts.yaml reset.yml \
  -e reset_confirmation=yes
```

- It is **destructive**, so Kubespray **requires an explicit confirmation**.
- `-e reset_confirmation=yes` skips the interactive prompt — only when you are certain.

It wipes K8s components, **etcd data**, CNI and config on the nodes. **Back up etcd first.**

18.7 & 18.8 Certificates & Encryption at Rest

Certificate Auto-Renewal

kubeadm issues **1-year** control-plane certs by default; Kubespray can **auto-renew** them so they never lapse silently.

- `auto_renew_certificates: false` (default) → set `true` to auto-renew on a schedule (default: **first Monday of each month**).
- Configurable via `auto_renew_certificates_systemd_calendar`, e.g. `"Mon * - * -1,2,3,4,5,6,7 03:00:00"` — installs a **systemd timer/service** on the control plane.
- Re-running `cluster.yml` / `upgrade-cluster.yml` also refreshes certs via the kubeadm flow; auto-renewal is the hands-off option between runs.

Set `auto_renew_certificates: true` on long-lived clusters so an expired apiserver/etcd cert never takes the control plane down.

Encrypting Secret Data at Rest

By default Secrets are stored **base64-encoded, not encrypted**, in etcd.
Kubespray enables encryption via a kube-apiserver `EncryptionConfiguration`.

- `kube_encrypt_secret_data: false` (default) → `true` generates a key and wires an `EncryptionConfiguration` into the apiserver.
- **5 providers:** `identity`, `aesgcm`, `aescbc`, `secretbox` (default), `kms`.

Provider	Why / why not (default = <code>secretbox</code>)
<code>identity</code>	no encryption
<code>aesgcm</code>	must be rotated every 200k writes
<code>aescbc</code>	not recommended (padding-oracle risk)
<code>secretbox</code>	default — Poly1305 MAC + XSalsa20
<code>kms</code>	officially recommended, but needs an external KMS

Enabling it only encrypts **newly written** Secrets until you re-write existing ones.

Key Takeaways

- **etcd is the source of truth** → **a snapshot is your backup**. Take them on a schedule, **copy off-node**, verify with `snapshot status` (a sanity check).
- **Manual backup:** `ETCDCTL_API=3 etcdctl snapshot save` with the four flags (`--endpoints --cacert --cert --key`); kubeadm certs in `/etc/kubernetes/pki/etcd/`.
- **Manual restore:** new `--data-dir`, **move static-pod manifests aside**, then repoint the etcd manifest at the restored dir.
- **Kubespray etcd is host** (systemd) — certs in `/etc/ssl/etcd/ssl/`, client `2379`, metrics `2381`.
- **recover-control-plane.yml** — `broken_etcd` / `broken_kube_control_plane` groups, `--limit etcd,kube_control_plane -e etcd_retries=...`, restore via `-e etcd_snapshot=...` on lost quorum.
- **reset.yml -e reset_confirmation=yes** is the destructive inverse of `cluster.yml`; `auto_renew_certificates` + `kube_encrypt_secret_data` are the preventive toggles.

End of Lesson 18

Next: Lesson 19 — Add-ons & Packaging

Cheat-sheet:

```
etcdctl snapshot save /opt/snap.db --endpoints ... --cacert ... --cert ... --key ... .
```

```
etcdctl snapshot status /opt/snap.db --write-out=table .
```

```
etcdctl snapshot restore /opt/snap.db --data-dir=/var/lib/etcd-restore .
```

```
recover-control-plane.yml --limit etcd,kube_control_plane -e etcd_retries=10 -e etcd_snapshot=... .
```

```
reset.yml -e reset_confirmation=yes .
```

```
auto_renew_certificates: true · kube_encrypt_secret_data: true ((secretbox))
```

Add-ons & Packaging

Kubernetes Production · Lesson 19

Cluster add-ons via Kubespray + Helm & Kustomize

kubespray.io · helm.sh · kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Separate **platform add-ons** (Kubespray) from **application packaging** (you)
- Toggle cluster add-ons in `addons.yml` and reconcile with `cluster.yml`
- Configure **cert-manager**, the **registry** and **MetalLB** (L2 / BGP)
- Explain why **ingress-nginx / dashboard** were removed → install via **Helm**
- Package apps with **Helm** (Chart / Repo / Release / Values, no Tiller)
- Customise manifests with **Kustomize** (bases, overlays, generators, patches)

Reference: *Kubespray add-ons · Using Helm · Kustomization* —
kubespray.io · helm.sh · kubernetes.io/docs

19.1 Two Layers: Add-ons vs Packaging

Two Layers of "Extra Stuff" on a Cluster

There are **two distinct layers** — at **different times, different tools**:

Layer	What	Who / When	Tool
Platform add-ons	metrics-server, MetalLB, cert-manager, registry...	Kubespray, at deploy time	<code>addons.yml</code> → <code>cluster.yml</code>
Application workloads	<i>your</i> apps + third-party components	You, at run time	Helm / Kustomize

- **Kubespray** `addons.yml` installs *platform* add-ons **as part of the cluster** — re-run `cluster.yml` and Ansible converges the desired state.
- **Helm / Kustomize** are how *you* deploy and customise *application* workloads on the **running** cluster's API server.

Resolving the overlap: `helm_enabled: true` does **not** deploy your apps — it only makes the **Helm tooling available**. Two layers, two times, no overlap.

19.2 Cluster Add-ons via Kubespray

The `addons.yml` File

Add-ons are **optional cluster services** Kubespray deploys as part of `cluster.yml`. Each is an **on/off boolean** in one inventory file:

```
inventory/sample/group_vars/k8s_cluster/addons.yml
```

```
# 1. edit the toggle → 2. re-run the playbook → 3. add-on appears
ansible-playbook -i inventory/mycluster/hosts.yaml cluster.yml
# or scope it: --tags metallb
```

- **Provision-at-deploy-time:** you do **not** `helm install` these — `cluster.yml` reconciles them.
- **Declarative & idempotent** — same model as the rest of Kubespray: the file describes desired state, the playbook converges to it.

Common Toggles

All booleans, in `group_vars/k8s_cluster/addons.yml`:

Toggle	Add-on
<code>helm_enabled</code>	Helm client on control-plane nodes (charts post-deploy)
<code>metrics_server_enabled</code>	metrics-server — <code>kubectl top</code> + HPA
<code>cert_manager_enabled</code>	cert-manager — certificate controller
<code>ingress_nginx_enabled</code>	ingress-nginx — removed from recent releases
<code>metallb_enabled</code>	MetalLB — <code>type: LoadBalancer</code> on bare metal
<code>local_path_provisioner_enabled</code>	dynamic local-disk PVs (Rancher)
<code>registry_enabled</code>	in-cluster private Docker registry
<code>dashboard_enabled</code>	Kubernetes Dashboard — also removed

```
# addons.yml
metrics_server_enabled: true
helm_enabled: true
cert_manager_enabled: true
```

cert-manager (`cert_manager_enabled`)

A **native Kubernetes certificate controller** — issues from **Let's Encrypt**, **Vault**, **Venafi**, **signing key pairs**, or **self-signed**, and **renews before expiry**.

- **Secures Ingress** via the **ingress-shim**: annotate the Ingress and the shim **auto-creates the Certificate**; certs **rotate ~every 3 months**, stored as **Secrets**.

```
metadata:
  annotations:
    cert-manager.io/cluster-issuer: ca-issuer
```

- **Issuers**: an **ACME Issuer** = an account at an ACME CA (e.g. Let's Encrypt); ACME certs are **free + trusted by default**, validated via **HTTP01 / DNS01**.
- **Trust an internal CA**: set `cert_manager_trusted_internal_ca` (a PEM block) in `addons.yml`; wire your root CA (built with **cfssl**) as a **ClusterIssuer**.

In-Cluster Registry (`registry_enabled`)

A **private Docker registry as a Pod** — store proprietary images with **no external dependency**. Vars: `registry_namespace`, `registry_storage_class`, `registry_disk_size`. Runs in `kube-system`.

- **Storage: persistent** via a PV/PVC, or **temporary** via `emptyDir`.
- **Exposure:** a **Service on port 5000** + a **DaemonSet proxy on every node** binding `hostPort: 5000` → each node reaches it at `localhost`.
- **Why the proxy:** the registry has **no SSL or auth** (Docker would reject it as **"insecure"**). Running the proxy on `localhost` makes Docker treat it as **secure**.

```
# reference images via the localhost proxy
docker push localhost:5000/user/container
```

MetalLB — Bare-Metal LoadBalancer (`metallb_enabled`)

Lets you create `type: LoadBalancer` Services on bare metal (no cloud).

```
# addons.yml
metallb_enabled: true
metallb_speaker_enabled: true
metallb_namespace: metallb-system
kube_proxy_strict_arp: true          # PREREQUISITE
```

- **Strict ARP** stops MetalLB answering unwanted ARP queries from the `kube-ipvs0` interface — **required**.
- **Address pools** (`metallb_config.address_pools`): `ip_range` as **CIDR** (`192.0.3.0/24`) or **dash form** (`192.0.1.0-192.0.1.254`), `auto_assign` (default `true`), `avoid_buggy_ips: true` (skip `.0` / `.255`).

Calico ≥ 3.18: set `metallb_speaker_enabled: false` and use `calico_advertise_service_loadbalancer_ips` (Calico does the BGP advertising).

MetalLB — L2 vs BGP (address pools)

```
metallb_config:
  address_pools:
    primary:
      ip_range:
        - 192.0.3.0/24
      auto_assign: true
      avoid_buggy_ips: true
  layer2:                                # L2 (default): one node answers via ARP
    - primary
  layer3:                                # BGP / Layer3: announce to routers
    metallb_peers:
      peer1:
        peer_address: 192.0.2.1
        peer_asn: 64512
        my_asn: 64513
        # optional: peer_port (179), hold_time (120s),
        # password (TCP-MD5), router_id, communities, address_pool
```

- **Layer2** — simple ARP, one node owns the VIP. **BGP** — advertise service IPs to upstream routers via `metallb_peers`.

Local Storage + Removed Add-ons → Helm

Local storage toggles:

- `local_path_provisioner_enabled` — Rancher provisioner, **dynamic** local-disk PVs from a node path (simple default for labs).
- `local_volume_provisioner_enabled` — **static** (`kubernetes.io/no-provisioner`): one PV per mount under `host_dir`; **you pre-create the mounts** and persist them via `/etc/fstab`.

Removed add-ons → install with Helm:

- In **recent Kubespray versions**, `ingress_nginx_enabled` and `dashboard_enabled` **were removed** from the built-in set.
- Recommended replacement: **install them with Helm after deploy** — which is exactly why `helm_enabled: true` exists.

19.3 Helm — Packaging Your Workloads

Helm — Three Concepts + Helm 3

Helm is the **package manager for Kubernetes** — bundle, version, template, upgrade and roll back sets of resources.

Concept	Meaning
Chart	A Helm package — all resources to run an app (like apt/yum)
Repository	Where charts are shared (indexed HTTP server or OCI registry)
Release	A running instance of a chart — one chart → many releases

- **Values** parameterize a chart's templates.
- **Helm 3 is client-only — no Tiller.** The client renders templates **locally** and submits manifests **directly to the API server** using your **kubeconfig + RBAC**. Release state is stored as **Secrets** in the release's namespace.

Chart Structure & Chart.yaml

```
mychart/  
├── Chart.yaml      # REQUIRED – chart metadata  
├── values.yaml     # default configuration values  
├── charts/         # subcharts (dependencies)  
├── crds/           # CRDs installed before the rest of the chart  
└── templates/      # Go-templated Kubernetes manifests
```

```
# Chart.yaml  
apiVersion: v2      # v2 = Helm 3  
name: mychart  
version: 1.2.3      # CHART version – SemVer 2 (required)  
appVersion: "1.16.0" # version of the packaged APP (informational)  
type: application
```

- **version** = the chart's own version; **appVersion** = the app's version — **independent**. Defaults live in `values.yaml`; manifests in `templates/`.

Helm Workflow

```
# Repositories
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update           # refresh cached index
helm search repo nginx     # search added repos

# Install / upgrade / uninstall
helm install my-release bitnami/nginx
helm upgrade --install my-release bitnami/nginx # idempotent deploy
helm uninstall my-release

# Inspect & roll back
helm list -A               # releases in all namespaces
helm history my-release    # revision history
helm rollback my-release 1 # revert to revision 1
helm template my-release bitnami/nginx # render, do NOT install
```

helm upgrade --install is the idempotent CI/CD deploy verb;
helm template renders manifests **without** touching the cluster.

Overriding Values & Precedence

```
# YAML file(s) of overrides — later -f wins
helm install my-release bitnami/nginx -f base.yaml -f prod.yaml

# Inline overrides — rightmost --set wins
helm upgrade --install my-release bitnami/nginx --set image.tag=1.25
```

Value precedence (lowest → highest):

1. chart's default `values.yaml`
2. parent chart values (for subcharts)
3. user `-f` / `--values` files (later file overrides earlier)
4. `--set` flags — **highest priority, override everything**

CKA: `--set > -f > values.yaml`. Useful flags: `-n` / `--namespace`, `--create-namespace`, `--dry-run`, `--debug`, `--version`.

19.4 Kustomize — Customising Manifests

Kustomize — Template-Free, in kubectl

- **Template-free, declarative** customization of raw Kubernetes YAML — no Go templating, no DSL. You keep **valid manifests** and layer changes on top.
- **Built into `kubectl` since v1.14** — `kubectl apply -k` and `kubectl kustomize` need **no extra binary**.
- Driven by one `kustomization.yaml` listing **resources** plus **generators** (create objects) and **transformers** (mutate objects).

Field	Category	Purpose
<code>resources</code>	compose	Manifest files / dirs / bases to include
<code>configMapGenerator</code> / <code>secretGenerator</code>	generate	Build from literals / files
<code>namespace</code>	transform	Set namespace on all resources
<code>namePrefix</code> / <code>nameSuffix</code>	transform	Prefix/suffix all names
<code>labels</code>	transform	Add labels (and selectors) — replaces <code>commonLabels</code>
<code>images</code>	transform	Override <code>newName</code> / <code>newTag</code> / digest
<code>patches</code>	patch	Modify specific fields of specific resources

Generators & Transformers

```
# kustomization.yaml
resources:
  - deployment.yaml
  - service.yaml

namespace: my-namespace
namePrefix: dev-
labels:                                     # use labels: (NOT deprecated commonLabels)
  - pairs:
      app: bingo

configMapGenerator:
  - name: app-config
    literals:
      - LOG_LEVEL=debug

images:
  - name: nginx
    newTag: 1.25-alpine
```

Generators append a **content hash** to the name by default (a config change rolls the Pods); disable with `generatorOptions.disableNameSuffixHash: true`.

Bases & Overlays

A **base** holds shared manifests; an **overlay** references it and patches it per environment — keeping the base DRY.

```
# overlays/prod/kustomization.yaml
resources:
  - ../../base          # reference the base
namespace: production
namePrefix: prod-
patches:
  - path: replica-patch.yaml    # strategic-merge patch
images:
  - name: myapp
    newTag: v1.2.0
```

```
kubectl kustomize overlays/prod    # render to stdout (preview, no apply)
kubectl apply -k overlays/prod     # build AND apply
kubectl delete -k overlays/prod    # delete the built resources
```


19.5 Helm vs Kustomize

Helm vs Kustomize — When to Use Which

Aspect	Helm	Kustomize
Approach	Package manager + templating	Template-free overlays/patches
Unit	Versioned chart , tracked release	Plain YAML + <code>kustomization.yaml</code>
Tooling	Separate <code>helm</code> binary	Built into <code>kubectl</code> (<code>-k</code>)
Customization	<code>values.yaml</code> , <code>--set</code> , <code>-f</code>	bases + overlays, generators, patches
Lifecycle	install / upgrade / rollback / history	<code>apply -k</code> / <code>delete -k</code> (no rollback)
Best for	Third-party cluster components & complex apps	Tweaking your own manifests per env

- Use **Helm** for off-the-shelf components (ingress controllers, monitoring) and apps that benefit from **templating + rollbacks**.
- Use **Kustomize** to maintain **your own** manifests across dev/staging/prod.
- They **compose** — post-render a Helm chart with Kustomize, or install dependencies with Helm alongside Kustomize-managed manifests.

Resolving the Overlap

The two layers map onto **time** and **ownership**:

- **addons.yml (Kubespray)** → *platform* add-ons, installed **as part of the cluster**, at **deploy time**, via **cluster.yml** (Ansible converges state).
- **Helm / Kustomize (you)** → *application* workloads, deployed at **run time** against the **running** API server.
- **helm_enabled: true** does **not** deploy your apps — it just makes the **Helm tooling available** on the control plane.

The removed add-ons (**ingress-nginx**, **dashboard**) are the bridge: they *used* to be platform toggles, now you install them as **application** charts with Helm — same cluster, different layer.

Key Takeaways

- **Two layers:** Kubespray `addons.yml` = *platform* add-ons; Helm/Kustomize = *your* apps
- Add-ons are **booleans in** `addons.yml`, reconciled by `cluster.yml` (idempotent)
- **MetalLB** = `type: LoadBalancer` on bare metal — **L2 or BGP**; set `kube_proxy_strict_arp: true`
- **ingress-nginx / dashboard removed** → install via **Helm**; that's what `helm_enabled` is for
- **Helm 3 = no Tiller**, client-only; state in **Secrets**; `upgrade --install` is idempotent
- **Value precedence:** `--set` > `-f` > `values.yml`
- **Kustomize** = `kubectl apply -k`, base + overlays, `labels:` (not `commonLabels`)

End of Lesson 19

Next: Lesson 20 — Logging, Monitoring & Troubleshooting

`addons.yml` · `ansible-playbook cluster.yml` · `helm upgrade --install` · `helm rollback` · `kubectl apply -k`

Logging, Monitoring & Troubleshooting

Kubernetes Production · Lesson 20

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **container logging architecture** and read logs with `kubectl logs`
- Investigate cluster **events** and use `kubectl describe` for root-cause analysis
- Deploy the **resource metrics pipeline** and read `kubectl top`
- Use `kubectl debug` **ephemeral containers** for distroless images
- Diagnose common **Pod failure states** and **node-level** problems

Reference: *Troubleshooting Applications* — kubernetes.io/docs/tasks/debug/debug-application

20.1 Container Logging Architecture

How Container Logs Work

Kubernetes captures logs from each container in a running Pod that writes to **standard output** and **standard error**.

- The **container runtime** handles and redirects `stdout` / `stderr`
- Its integration with the kubelet is the standardized **CRI logging format**
- On Linux, node-level container logs live in `/var/log/pods` (default)
- `kubectl logs` reads these node-level files **through the Kubernetes API**

Write application logs to `stdout` / `stderr` — *not* to files inside the container — so Kubernetes can collect them automatically.

Node-Level Logging & Rotation

The **kubelet** manages container logs at the node level:

- Keeps **one** terminated container (and its logs) after a restart
- Evicts **all** of a Pod's containers and logs when the **Pod is evicted**
- Rotates logs via kubelet config: `containerLogMaxSize` (default **10Mi**), `containerLogMaxFiles` (default **5**)

Caveat: `kubectl logs` returns only the **latest** log file. A Pod writing 40 MiB under 10 MiB rotation exposes at most **10 MiB** via the API.

Cluster-Level Logging

Kubernetes provides **no** native storage for logs — add a backend so logs survive crashes, evictions, and dead nodes.

Architecture	How it works
Node logging agent	DaemonSet tails <code>/var/log/pods</code> on every node
Sidecar agent	Logging container ships logs alongside the app
Direct exposure	App pushes logs straight to a backend

- The node-agent pattern (Fluentd / Fluent Bit) is the most common.

20.2 Reading Logs

kubectl logs Essentials

```
# Logs of a Pod's (single) container
kubectl logs my-pod

# A specific container in a multi-container Pod
kubectl logs my-pod -c app

# Logs from the PREVIOUS (crashed) container instance
kubectl logs my-pod --previous

# Stream/follow logs in real time
kubectl logs -f my-pod
```

`--previous` (`-p`) is the key to debugging a **CrashLoopBackOff** — it shows why the *last* container died before the restart.

Filtering Logs by Time & Label

```
# Only the last 15 minutes (also: --since-time=RFC3339)
kubectl logs my-pod --since=15m

# Last N lines, with timestamps
kubectl logs my-pod --tail=100 --timestamps

# Aggregate logs from all Pods matching a label selector
kubectl logs -l app=nginx --max-log-requests=10

# All containers of the Pod at once
kubectl logs my-pod --all-containers=true
```

- `-l` is invaluable for tailing a **Deployment's** Pods together.

20.3 Events & Describe

Cluster Events

Events record **what the control plane did** to your objects (scheduling, pulls, restarts, probe failures).

```
# Events in the current namespace, newest last
kubectl get events --sort-by=.metadata.creationTimestamp

# Events across all namespaces
kubectl get events -A

# Only events for one object
kubectl get events --field-selector involvedObject.name=my-pod
```

Events are **ephemeral** — they expire. Investigate failures promptly.

kubectl describe for Root Cause

`kubectl describe` is the single best command for *why* a Pod is unhealthy.

```
kubectl describe pod my-pod
```

It surfaces, in one view:

- **Container state**: Waiting (reason) / Running / Terminated (exit code)
- **Restart count** and last termination reason (e.g. `OOMKilled`)
- Resource requests/limits and Pod **Conditions**
- The **Events** block — usually names the exact failure

Workflow: `get pods` → `describe pod` → `logs --previous`.

20.4 The Resource Metrics Pipeline

Metrics Pipeline Components

Resource usage flows from container to CLI through four stages:

Stage	Role
cAdvisor	In-kubelet daemon; per-container CPU/memory from cgroups/CRI
kubelet	Exposes node/Pod metrics at <code>/metrics/resource</code>
metrics-server	Cluster addon; polls kubelets, serves the Metrics API
Metrics API	<code>metrics.k8s.io/v1beta1</code> ; feeds <code>top</code> , HPA, VPA, scheduler

- **metrics-server is not deployed by default** — you must install it.

kubectl top

```
# Per-node CPU & memory usage
kubectl top nodes

# Per-Pod usage (current namespace)
kubectl top pods

# Per-container breakdown, across all namespaces
kubectl top pods --containers -A
```

- `kubectl top` shows CPU in **millicores** (`m`); the raw Metrics API uses **nanocores** (`n`). Memory in **bytes** (`Ki`).
- Each value is a **window** (~30s) average — not a historical graph.

If `kubectl top` errors with *"Metrics API not available"*, metrics-server is missing or unhealthy — check `kubectl get pods -n kube-system`.

20.5 Debugging Running Pods

Exec & Ephemeral Debug Containers

```
# Get an interactive shell (image must contain a shell)
kubectl exec -it my-pod -- /bin/sh

# Attach an ephemeral debug container to a running Pod
kubectl debug -it my-pod --image=busybox --target=app
```

- `kubectl exec` works **only** if the image ships a shell.
- `kubectl debug` injects an **ephemeral container** without restarting the Pod — perfect for **distroless** images with no shell, `ps`, or package manager.
- `--target` shares the process namespace with the named container.

Debug by Copying the Pod

When you cannot disrupt the live Pod, debug a **copy**:

```
# Copy the Pod, add a debug container, share process namespace
kubectl debug my-pod -it --copy-to=my-pod-debug --share-processes

# Copy but override the command (e.g. crashloop on startup)
kubectl debug my-pod -it --copy-to=my-pod-debug -- /bin/sh

# Copy and swap a container's image
kubectl debug my-pod --copy-to=my-pod-debug --set-image=app=ubuntu
```

Ideal for a **CrashLoopBackOff** where the original command exits instantly — change the command to keep the container alive.

20.6 Troubleshooting Pod States

Common Pod Failure States

Symptom	Likely cause	First fix
Pending	No node fits; quota/taints/hostPort	<code>describe pod</code> → FailedScheduling; free resources
ImagePullBackOff	Bad image name/tag, auth, registry	Fix image; add <code>imagePullSecrets</code>
CrashLoopBackOff	App/command exits or crashes	<code>logs --previous</code> ; fix command/config
OOMKilled	Exceeded memory limit	Raise <code>limits.memory</code> or fix the leak
CreateContainerConfigError	Missing ConfigMap/Secret	Create the referenced object

Troubleshooting is **~30% of the CKA exam** — make this table reflex.

Pending & ImagePullBackOff

Pending — the scheduler could not place the Pod:

```
kubectl describe pod my-pod | grep -A5 Events
# FailedScheduling: Node didn't have enough resource: CPU / taints / selector
```

ImagePullBackOff — the kubelet cannot pull the image:

- Check spelling and **tag** of **image:** (**:latest** may not exist)
- Private registry → ensure a valid **imagePullSecrets**
- Confirm the node has network access to the registry

CrashLoopBackOff & OOMKilled

CrashLoopBackOff — container starts then exits repeatedly:

```
kubectl logs my-pod --previous    # why the last run died
kubectl describe pod my-pod       # exit code, restart count
```

- Common causes: wrong **command/args**, missing config, failing probe.

OOMKilled — kernel killed the container for exceeding memory:

- `describe` shows `Last State: Terminated, Reason: OOMKilled`
- Increase `resources.limits.memory` or reduce the app footprint.

20.7 Node Troubleshooting

Inspecting a Node

Start from the API view, then go to the host:

```
kubectl get nodes
kubectl describe node my-node    # Conditions, capacity, allocatable
```

Watch the **node conditions**:

- **Ready=True** — healthy and accepting Pods
- **MemoryPressure** / **DiskPressure** / **PIDPressure** — resource exhaustion
- **Ready=Unknown** — kubelet stopped reporting (network/kubelet down)

A **NotReady** node with no events usually means the **kubelet** is down.

On the Node: kubelet & Runtime

The kubelet and container runtime are **not** Pods — inspect them with systemd:

```
# kubelet logs and status
journalctl -u kubelet -f
systemctl status kubelet

# containerd logs
journalctl -u containerd

# What the runtime actually sees
crictl ps -a
crictl pods
```

- For static control-plane Pods, also check `/etc/kubernetes/manifests`.

Key Takeaways

- App logs go to **stdout/stderr**; `kubectl logs` reads node files at `/var/log/pods` — use `--previous`, `-f`, `--since`, `-l`
- `describe` + `events --sort-by` pinpoint **why** an object failed
- Deploy **metrics-server** (not default) to enable `kubectl top nodes/pods`
- `kubectl debug` ephemeral containers debug **distroless** Pods
- Memorize the **Pending / ImagePull / CrashLoop / OOMKilled** table
- Node down → `journalctl -u kubelet` and `crictl` on the host

End of Lesson 20

Next: Lesson 21 — Hardening & Offline

`kubectl logs --previous` · `kubectl top pods` · `kubectl debug` · `journalctl -u kubelet`

Hardening & Large / Offline Deployments

Kubernetes Production · Lesson 21

Built from the official documentation

`kubernetes-sigs/kubespray` · `docs/operations`

Learning Objectives

By the end of this lesson you will be able to:

- **Harden** a cluster via `group_vars` to CIS / **kube-bench** expectations
- **Harden workloads** with `securityContext` (non-root, drop capabilities, read-only FS) and enforce it with **Pod Security Admission**
- Turn off **anonymous-auth**, enable **audit logging** and **encryption at rest**
- Tune Kubespray for **hundreds of nodes** (forks, download-once, events etcd)
- Reserve node CPU/memory with **cgroups** (`kube_reserved` / `system_reserved`)
- Install in a fully **air-gapped / offline** environment via mirrors

All four topics are driven from `group_vars` — no code changes needed.

21.1 Security Hardening

Hardening — Where & Prerequisites

The hardening guide is a single block you add to `group_vars` (e.g. `group_vars/k8s_cluster/k8s-cluster.yml`).

- Requires **Kubernetes ≥ v1.23.6** for all features.
- Use a **recent Kubespray** — some of these vars were added recently.
- Ensure **no other vars override** the hardening settings.
- After install, **verify with kube-bench** (CIS Benchmark).

One opinionated `group_vars` block flips the cluster from "default" to "CIS-aligned". Review each value against your policy.

apiserver — AuthN / AuthZ & TLS

- `authorization_modes: ['Node', 'RBAC']`
- `remove_anonymous_access: true` — anonymous-auth **off** (safe with RBAC)
- `kube_apiserver_service_account_lookup: true`
- `kube_apiserver_request_timeout: 120s`
- `tls_min_version: VersionTLS12`
- `tls_cipher_suites:` strong ECDHE/GCM + ChaCha20 suites only

Loopback binds + RBAC + strong TLS are the foundation; the next slides add encryption, admission control and audit.

apiserver — Encryption at Rest

```
kube_encrypt_secret_data: true
kube_encryption_resources: [secrets]
kube_encryption_algorithm: "secretbox"

kube_profiling: false # disable /debug/pprof
```

- Secrets are encrypted **in etcd** rather than stored in plaintext.
- **secretbox** is the chosen encryption provider.
- Profiling endpoints are **disabled** (CIS check).

Admission Plugins & PodSecurity

```
kube_apiserver_enable_admission_plugins:  
- EventRateLimit  
- AlwaysPullImages  
- NodeRestriction  
- ResourceQuota  
- PodSecurity          # + ServiceAccount, NamespaceLifecycle, LimitRanger...  
kube_apiserver_admission_control_config_file: true  
  
kube_pod_security_use_default: true  
kube_pod_security_default_enforce: restricted # kube-system exempted
```

EventRateLimit + the admission config file throttle event floods;
PSA enforces the **restricted** profile by default.

Audit Logging

```
kubernetes_audit: true
audit_log_path: "/var/log/kube-apiserver-log.json"
audit_log_maxage: 30
audit_log_maxbackups: 10
audit_log_maxsize: 100
```

- Records every API request to a JSON log on the control plane.
- Rotation: keep **30 days**, **10 backups**, **100 MB** per file.

Controller-Manager / Scheduler / etcd

- `kube_controller_manager_bind_address: 127.0.0.1` — loopback only
- `kube_scheduler_bind_address: 127.0.0.1` — loopback only
- `kube_controller_terminated_pod_gc_threshold: 50`
- `kube_controller_feature_gates: ["RotateKubeletServerCertificate=true"]`
- `etcd_deployment_type: host` — **etcd isolated** outside the cluster

Binding control-plane components to loopback removes them from the network attack surface.

kubelet Hardening

```
kubelet_authorization_mode_webhook: true
kubelet_authentication_token_webhook: true
kubelet_rotate_certificates: true
kubelet_rotate_server_certificates: true
kube_read_only_port: 0 # disable read-only port
kubelet_protect_kernel_defaults: true
kubelet_streaming_connection_idle_timeout: "5m"
kubelet_seccomp_default: true
kubelet_systemd_hardening: true
```

Also: `kubelet_make_iptables_util_chains`, `kubelet_event_record_qps: 1`,
`kube_owner: root`, `kube_cert_group: root`.

kube-bench Alignment

These `group_vars` map directly onto CIS Kubernetes Benchmark checks:

- anonymous-auth **off**, kubelet **read-only port 0**
- **audit logging** enabled with rotation
- **encryption at rest** for secrets
- **restricted** Pod Security profile
- control-plane components on **loopback**, **profiling off**

Workflow: apply hardening → re-run `cluster.yml` → run **kube-bench** → remediate any remaining FAIL items.

21.2 Workload Hardening — securityContext

Cluster hardening vs workload hardening

21.1 locked down the **cluster** (apiserver, kubelet, PSA `restricted`).

`securityContext` locks down the **workload** — how each container **process runs on the node**:

- which **UID/GID** the process uses (`runAsUser` , `runAsNonRoot`)
- which **Linux capabilities** it keeps (`capabilities.drop/add`)
- whether it can **escalate** (`allowPrivilegeEscalation`) or **write its FS** (`readOnlyRootFilesystem`)

Orthogonal to **RBAC** (who may call the API). The apiserver's `restricted` PSA (21.1) *rejects* Pods that aren't hardened — `securityContext` is how a Pod *passes*.

Two levels + process identity

Pod-level = defaults for all containers; container-level **overrides** it.

```
spec:
  securityContext:           # Pod level
    runAsNonRoot: true      # refuse to start as root (UID 0)
    runAsUser: 1000
    fsGroup: 2000           # kubelet chowns mounted volumes to this GID
    seccompProfile: { type: RuntimeDefault }
```

Field	Level	Effect
runAsUser / runAsGroup	Pod + container	process UID / GID
runAsNonRoot	Pod + container	kubelet blocks start if it would run as root
fsGroup	Pod only	group owner of mounted volumes
capabilities	container only	add / drop Linux capabilities

Pitfall: runAsUser does **not** chown volumes — that's fsGroup's job.

Hardened container — the recipe

```
securityContext:                # container level
  allowPrivilegeEscalation: false # sets no_new_privs
  readOnlyRootFilesystem: true   # immutable "/" — mount emptyDir for writes
  capabilities:
    drop: ["ALL"]                # drop everything...
    add:  ["NET_BIND_SERVICE"]   # ...add back only what is needed
```

- `drop: ["ALL"]` then add the minimum = huge attack-surface reduction.
-  `privileged: true` and `CAP_SYS_ADMIN` **force** `allowPrivilegeEscalation` back to true — you cannot half-harden a privileged container.

securityContext ↔ Pod Security Admission

- **securityContext** = what the **developer declares** (self-imposed).
- **PSA** = what the **admin enforces** per namespace (from 21.1):

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted
```

- A non-hardened Pod is then **rejected** with the exact violations; a hardened Pod is **admitted**.
- Verify at runtime: `kubectl exec <pod> -- id` (non-root), `touch /` fails (read-only FS).

Together: **securityContext** hardens the Pod; **PSA guarantees** no un-hardened Pod is admitted.

21.3 Large Deployments

Scaling Ansible

```
# 200 nodes: parallelise and extend timeouts
ansible-playbook -i inventory/mycluster/hosts.yaml \
  cluster.yml --forks=50 --timeout=600
```

```
download_run_once: true    # download images/binaries once, push to nodes
retry_stagger: 60         # spread retries so they don't hammer the CP
```

`download_run_once` is the single biggest scale win — nodes stop each fetching from the internet.

Scaling the Cluster

- `etcd_events_cluster_setup: true` — store **events in a dedicated etcd**, so event churn doesn't overload the main etcd.
- Add `calico_rr` route-reflector nodes for faster recovery with Calico/Canal.
- Lengthen reliability timeouts so transient pressure doesn't evict nodes:
 - `kube_controller_node_monitor_grace_period`
 - `kube_apiserver_pod_eviction_not_ready_timeout_seconds`
 - `kube_apiserver_pod_eviction_unreachable_timeout_seconds`

Separating the events store is the key etcd protection at scale.

Resource Sizing & CoreDNS

- Pattern: `foo_cpu_limit` / `foo_cpu_requests`, `foo_memory_limit` / `foo_memory_requests` — **limits must be $\geq$ requests**.
- Memory in `M`; CPU drops the `m` suffix.
- **CoreDNS:** `dns_replicas`, `dns_cpu_limit` / `dns_cpu_requests`, `dns_memory_limit` / `dns_memory_requests`.
- Size the network for the node/pod count:
`kube_network_node_prefix`, `kube_service_addresses`, `kube_pods_subnet`.

Scale CoreDNS replicas and resources with the cluster — DNS is a common bottleneck.

21.4 cgroups — Node Reservation

Node Allocatable

$\text{Allocatable} = \text{Capacity} - \text{kube_reserved} - \text{system_reserved} - \text{eviction}$

- `kubelet_enforce_node_allocatable` selects what is enforced:
 - `"pods"`
 - `"pods, kube-reserved"`
 - `"pods, kube-reserved, system-reserved"`
- **kube-reserved** = kubelet + container engine.
- **system-reserved** = OS daemons (sshd, udev...).

Reserving resources keeps the **kubelet responsive** under node pressure.

kube_reserved / system_reserved

```
kube_reserved: true
kube_reserved_cgroups_for_service_slice: kube.slice
kube_reserved_cgroups: "/{{ kube_reserved_cgroups_for_service_slice }}"
kube_memory_reserved: 256Mi
kube_cpu_reserved: 100m

system_reserved: true
system_reserved_cgroups_for_service_slice: system.slice
system_memory_reserved: 512Mi
system_cpu_reserved: 500m
```

Optional: `*_ephemeral_storage_reserved`, `*_pid_reserved` for disk/PID caps.

21.5 Offline / Air-Gapped

Offline — Hosts & Mirrors

Mirror everything Kubespray would pull, then point `group_vars` at it.

Reference: `inventory/sample/group_vars/all/offline.yml`.

- `registry_host` / `registry_addr` — internal container image registry
- `files_repo` — HTTP server for binaries / zips
- `yum_repo` / `debian_repo` / `ubuntu_repo` — internal OS package mirrors
- Helper: `contrib/offline/generate_list.sh` lists everything to mirror

Three classes of artifact: **images**, **binaries**, **OS packages** — each gets its own mirror.

Image & Binary Repo Overrides

```
kube_image_repo: "{{ registry_host }}"
gcr_image_repo: "{{ registry_host }}"
quay_image_repo: "{{ registry_host }}"
github_image_repo: "{{ registry_host }}"

github_url: "{{ files_repo }}/github.com"
dl_k8s_io_url: "{{ files_repo }}/dl.k8s.io"
storage_googleapis_url: "{{ files_repo }}/storage.googleapis.com"
get_helm_url: "{{ files_repo }}/get.helm.sh"
```

All upstream image registries and download URLs are redirected to the mirror.

OS Packages & Containerd Mirror

```
docker_rh_repo_base_url: "{{ yum_repo }}/docker-ce/$releasever/$basearch"
containerd_ubuntu_repo_base_url: "{{ ubuntu_repo }}/containerd"

containerd_registries_mirrors:
  - prefix: "{{ registry_addr }}"
    mirrors:
      - host: "{{ registry_host }}"
        capabilities: ["pull", "resolve"]
        skip_verify: true
```

Auth via `files_repo_user/pass` and `registry_user/pass` (use Ansible Vault).

Key Takeaways

- **Hardening** = one `group_vars` block: anonymous-auth off, audit on, encryption at rest, restricted PSA, kubelet locked down — then **kube-bench**.
- **Workload hardening** = `securityContext` (`runAsNonRoot`, drop `ALL` caps, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`); **PSA** enforces it per namespace (`pod-security.kubernetes.io/enforce=restricted`).
- **Scale** = `download_run_once`, more **forks/timeout**, a **separate events etcd**, longer node-monitor/eviction timeouts.
- **cgroups** reserve CPU/mem/PIDs for kubelet (`kube_reserved`) and the OS (`system_reserved`) so workloads can't starve the node.
- **Offline** = redirect images (`*_image_repo`), binaries (`*_url`) and OS packages (`*_repo`) to internal mirrors via `offline.yml`.

End of Lesson 21

Next: Lesson 22 — Cloud & OpenStack

`kube-bench` · `download_run_once` · `kube_reserved` · `offline.yml`

Cloud Integration & OpenStack

Formation · Lesson 13 (Capstone)

Built from the official documentation

`kubernetes-sigs/kubespray` · cloud_controllers · CSI

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **external cloud-provider** model and why in-tree is gone
- Enable OpenStack with `cloud_provider: external` + `external_cloud_provider`
- Supply **OpenStack credentials** (openstack-rc or application credentials)
- Configure **Octavia/LBaaS** so Services get a floating IP
- Deploy **Cinder CSI** and a StorageClass for dynamic volumes
- **Verify** the integration: pods, a PVC, a LoadBalancer

Capstone: wire a Kubespray cluster into a real cloud — primarily OpenStack.

22.1 The External Cloud Provider Model

In-Tree Is Removed — Use the External CCM

Since Kubernetes **v1.31** (Kubespray **v2.27**), Kubernetes no longer supports `cloud_provider` **except the external cloud provider**.

Concept	In-tree (removed)	External (current)
CCM	inside controller-manager	separate <code>cloud-controller-manager</code> pod
Enable	<code>cloud_provider: openstack</code>	<code>external</code> + <code>external_cloud_provider</code>
Storage	in-tree volume plugin	Cinder CSI (<code>cinder.csi.openstack.org</code>)
LBaaS	Neutron LBaaS	Octavia (default)

Storage moved to **out-of-tree CSI**; the CCM and Cinder CSI are configured separately and can even run independently.

22.2 Enabling OpenStack

Turn On the External OpenStack Provider

Set two variables in `group_vars/all/all.yml`, then re-run `cluster.yml`:

```
cloud_provider: external
external_cloud_provider: openstack
```

- Kubespray configures the provider **to use Octavia by default** (not the old Neutron LBaaS).
- The same playbook deploys the **cloud-controller-manager** and (when enabled) **Cinder CSI** pods.
- vSphere uses the identical pattern with `external_cloud_provider: vsphere`.

Inventory **hostnames must match** the OpenStack instance names — otherwise Cinder will not work as expected.

OpenStack Credentials

Source your `openstack-rc` before running the playbook:

```
# exports OS_AUTH_URL, OS_USERNAME, OS_PASSWORD, ...  
source path/to/your/openstack-rc
```

Or use Keystone **application credentials** (precedence over username/password; **all three required**) in `group_vars/all/openstack.yml`:

```
external_openstack_application_credential_name:  
external_openstack_application_credential_id:  
external_openstack_application_credential_secret:
```

Multi-NIC VMs: override `external_openstack_network_internal_networks` / `_public_networks` and `external_openstack_metadata_search_order`.

22.3 Octavia / LBaaS

LBaaS Variables (group_vars/all/openstack.yml)

```
external_openstack_lbaas_enabled: true
external_openstack_lbaas_floating_network_id: "Neutron network ID for floating IP"
external_openstack_lbaas_subnet_id: "Neutron subnet ID to create LBaaS VIP"
external_openstack_lbaas_provider: amphora          # or ovn
external_openstack_lbaas_method: ROUND_ROBIN
external_openstack_lbaas_manage_security_groups: false
external_openstack_lbaas_create_monitor: false
```

- `..._floating_network_id` gives a `LoadBalancer` Service its **floating IP**.
- `..._provider` selects the Octavia driver: **amphora** (default) or **ovn**.

Migrating? Replace the old `openstack_lbaas_subnet_id` with the new `external_openstack_lbaas_subnet_id` — the new provider uses **Octavia**.

22.4 Cinder CSI

Enable Cinder CSI

In `group_vars/all/openstack.yml` (after sourcing openstack-rc):

```
cinder_csi_enabled: true
cinder_csi_controller_replicas: 1
cinder_csi_ignore_volume_az: true    # schedule on any node AZ
cinder_topology: true                # optional topology support
```

- Cinder CSI does **not** require enabling the OpenStack CCM — both can coexist (their provisioners are named differently).
- To also deploy the provisioner, set `persistent_volumes_enabled: true` in `group_vars/k8s_cluster/k8s_cluster.yml`.
- Snapshotter & sidecars are pinned via `cinder_csi_snapshotter_image_tag`, `..._attacher`, `..._provisioner`, `..._resizer`, `livenessprobe`.

Only **Cinder v3** is supported by the CSI driver.

Cinder StorageClass

```
storage_classes:
- name: "cinder-csi"
  provisioner: "cinder.csi.openstack.org"
  mount_options:
    - "discard"
  parameters:
    type: "thin"
    availability: "nova"
    reclaim_policy: "Delete"
    volume_binding_mode: "WaitForFirstConsumer"
```

- Default class is `cinder-csi`, real provisioner `cinder.csi.openstack.org`.
- `WaitForFirstConsumer` binds the volume only once a Pod is scheduled.

22.5 Other Clouds

vSphere / AWS / Azure / GCP

vSphere — same model, in `group_vars/all/vsphere.yml`:

```
cloud_provider: "external"
external_cloud_provider: "vsphere"
external_vsphere_vcenter_ip: "myvcenter.domain.com"
external_vsphere_vcenter_port: "443"
external_vsphere_insecure: "true"
external_vsphere_user: "administrator@vsphere.local"
external_vsphere_password: "K8s_admin"
external_vsphere_datacenter: "DATACENTER_name"
vsphere_csi_enabled: true
```

- Requires vSphere **6.7 U3+**, VM hardware **v15+**, **UUID activated** per VM.
- **AWS / GCP** legacy flag `-e cloud_provider=[aws|gce]`; in-tree removed
≥ v1.31 → use the external CCM + matching CSI per cloud (Azure, OCI, Huawei).

22.6 Verification

Check Pods, PVC and LoadBalancer

```
# cloud-controller + cinder-csi pods Running
kubectl -n kube-system get pods | grep cinder
# csi-cinder-controllerplugin-... 5/5 Running
# csi-cinder-nodeplugin-... 2/2 Running

kubectl get storageclass      # cinder-csi cinder.csi.openstack.org

# apply nginx.yaml (PVC + Pod) → volume provisioned & bound
kubectl get pvc              # csi-pvc-cinderplugin Bound 1Gi cinder-csi
kubectl exec -it nginx -- df -h | grep /var/lib/www/html # /dev/vdb mounted

kubectl get svc              # LoadBalancer EXTERNAL-IP = Octavia floating IP
```

Result: a **PVC provisions a Cinder volume** and a **LoadBalancer Service gets a floating IP** — the integration is working end to end.

L3 CNl on OpenStack (calico / kube-router)

OpenStack drops packets from IPs it does not know (anti-spoofing). L3 CNIs route with **pod IPs directly**, so you must allow them on the Neutron ports:

```
openstack port set <PORT_ID> \  
  --allowed-address ip-address=10.233.0.0/18 \  
  --allowed-address ip-address=10.233.64.0/18
```

- `10.233.0.0/18` = `kube_service_addresses`, `10.233.64.0/18` = `kube_pods_subnet`.
- Set on **every** instance port (sweep with `openstack port list | xargs`).

Skip this and pod traffic silently fails on calico / kube-router.

Key Takeaways

- **External CCM + CSI** is the only supported model: `cloud_provider: external`
 - `external_cloud_provider: openstack`.
- Auth = **source openstack-rc** or **application credentials** (all three);
inventory hostnames must match instance names.
- **Cinder CSI** (`cinder_csi_enabled: true`) → dynamic volumes; **Octavia** → `LoadBalancer` Services with a floating IP.
- **vSphere / AWS / Azure / GCP** follow the same external-CCM + CSI pattern.

Formation Complete

You can now deploy & operate Kubespray clusters — including on OpenStack

`cloud_provider: external` · `cinder_csi_enabled` · Octavia LoadBalancers

Exam Preparation & Practice

Kubernetes Production · Lesson 23

Strategy, speed, and practice scenarios for exam day

Learning Objectives

By the end of this lesson you will be able to:

- Describe the exam logistics and rules
- Prioritise study time by domain weight
- Work fast with imperative `kubectl` and `--dry-run`
- Apply a repeatable per-task and time-management strategy
- Run two end-to-end practice scenarios

Practice on a real cluster until these patterns are automatic.

23.1 Exam Logistics

Exam Day Logistics

Item	Detail
Format	Performance-based tasks in a browser terminal
Duration	2 hours
Passing	66%
Version	Kubernetes v1.35
Proctoring	Remote, ID required, clean desk
Allowed docs	kubernetes.io/docs (+ subdomains), nothing else
Retake	One free retake within 12 months

You get **multiple clusters** — each task tells you which `kubectl context` to use. Switching context first, every time, prevents costly mistakes.

Always Switch Context First

Each question specifies a cluster. Begin **every** task with:

```
kubectl config use-context <context-from-the-question>
```

And work in the right namespace:

```
kubectl config set-context --current --namespace=<ns>
```

A correct answer on the **wrong cluster** scores **zero**. Make context-switch your reflex before reading the rest of the task.

23.2 Study Strategy

Prioritise by Domain Weight

Domain	Weight	Focus
Troubleshooting	30%	logs, events, describe, fix broken Pods/nodes
Cluster Architecture	25%	kubeadm, etcd backup/restore, RBAC, upgrades
Services & Networking	20%	Services, Ingress, NetworkPolicy, DNS
Workloads & Scheduling	15%	Deployments, taints/affinity, resources
Storage	10%	PV, PVC, StorageClass

Spend the most practice time on **Troubleshooting** and **Architecture** — together they are **55%** of the score.

New 2025 Topics

The refreshed CKA adds real-world tooling. Be comfortable with:

- **Helm** — install/upgrade charts (`helm install`, `helm upgrade`)
- **Kustomize** — `kubectl apply -k`, overlays and patches
- **Gateway API** — the successor to Ingress (`GatewayClass`, `Gateway`, `HTTPRoute`)
- **CRDs & Operators** — `kubectl get crd`, custom resources

All now covered in this course: **Helm & Kustomize (L12)**, **CRDs & Operators (L13)**, **Gateway API (L5)**, **autoscaling (L3)**, **HA control plane (L2)**.

23.3 Working Fast

Speed Setup

Configure your shell in the first minute:

```
alias k=kubectl
export do='--dry-run=client -o yaml'      # generate manifests fast
export now='--force --grace-period=0'    # delete immediately
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
```

kubectl autocompletion + a short alias save real minutes across 15+ tasks.

Generate, Don't Hand-Write YAML

Start from generated manifests, then edit:

```
# Deployment skeleton
k create deploy web --image=nginx:1.27 --replicas=3 $do > web.yaml

# Pod skeleton
k run tmp --image=busybox $do --command -- sleep 3600 > pod.yaml

# Service / expose
k expose deploy web --port=80 --target-port=8080 $do > svc.yaml

# RBAC
k create role r --verb=get,list --resource=pods $do
k create rolebinding rb --role=r --serviceaccount=ns:sa $do
```

Imperative-then-edit is far faster than writing YAML from scratch.

Look It Up Fast

```
kubectl explain pod.spec.containers.resources    # field schema, offline
kubectl explain pvc.spec --recursive             # full tree

kubectl api-resources                           # kind ↔ shortname ↔ apiVersion
kubectl get pod -o yaml | less                  # see every field of a live object
```

In the docs, search for the task name (e.g. "network policy") and **copy the example YAML** straight from the page — then adapt it.

Bookmark-by-memory: most tasks map to one docs example you can paste & tweak.

Time Management

- **~8 minutes per task** average — but weights vary, watch the % shown.
- **Triage first**: do quick, high-value tasks; **flag** hard ones and return.
- Never get stuck — partial credit counts; move on and come back.
- **Verify** each answer (`kubectl get` , `describe` , `logs`) before leaving it.
- Keep ~10 minutes at the end to re-check flagged tasks.

Don't polish. A working, verified answer beats a perfect one you never finish.

23.4 Task Patterns Checklist

Patterns You Must Know Cold

- Create/scale/rollback a **Deployment**; update its image
- **Expose** with ClusterIP / NodePort; create an **Ingress**
- Create **PV / PVC**, bind to a Pod; pick a **StorageClass**
- **ServiceAccount + Role + RoleBinding**; verify with `auth can-i`
- **Taints / tolerations / nodeSelector / affinity**
- **etcd snapshot save / restore**
- **drain / cordon / uncordon** a node for maintenance
- **NetworkPolicy** default-deny + selective allow
- **Troubleshoot** a failing Pod or NotReady node

Troubleshooting Reflex

```
kubectl get pod -A | grep -vE 'Running|Completed'    # what's broken?
kubectl describe pod <p>                            # Events at the bottom = root cause
kubectl logs <p> [-p] [-c c]                        # app errors / previous crash
kubectl get events --sort-by=.lastTimestamp
kubectl top pod / nodes                             # resource pressure
```

Symptom	Likely cause
ImagePullBackOff	wrong image/tag or registry auth
CrashLoopBackOff	app exits — check logs / command
Pending	no node fits (resources, taint, selector, PVC)
OOMKilled	memory limit too low

23.5 Practice Scenarios

Practice Exam #1 (excerpt)

In the provided cluster/namespace:

1. Create Deployment `web` (`nginx:1.27`, 3 replicas, label `tier=frontend`); expose it ClusterIP :80.
2. Add a **NetworkPolicy** allowing ingress to `web` only from Pods labelled `role=tester`.
3. Create a **200Mi PVC**, mount it in a Pod, prove data persists across Pod restarts.
4. Schedule a Pod onto a `tier=gold` node that **tolerates** a `batch` taint.
5. Fix a broken Deployment so it becomes `Available`.

Full solution + verification: `_labs/Lesson_12_13_Practice_Exam.md`.

Practice Exam #2 (excerpt)

1. Build an `auditor` **ServiceAccount** that can `get/list/watch` in two namespaces but **nothing** in `default`; verify with `auth can-i`.
2. Take an **etcd snapshot** and confirm it with `snapshot status`.
3. **Cordon + drain** a worker for maintenance; confirm Pods moved; uncordon.
4. Deploy an app and route `Host: echo.local` to it through an **Ingress**.

Full solution + verification: `_labs/Lesson_12_13_Practice_Exam.md`.

A Worked Example: etcd Backup

A frequent, high-value task — practise until automatic:

```
ETCDCTL_API=3 etcdctl \  
  --endpoints=https://127.0.0.1:2379 \  
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \  
  --cert=/etc/kubernetes/pki/etcd/server.crt \  
  --key=/etc/kubernetes/pki/etcd/server.key \  
  snapshot save /opt/snapshot.db  
  
etcdctl snapshot status /opt/snapshot.db -w table  # verify
```

The cert paths are standard on kubeadm clusters — memorise them.

Final Checklist

The day before

- Re-run both practice exams from memory, timed.
- Re-do any task you hesitated on.

On exam day

- Set aliases + completion in minute one.
- Switch **context** before every task; set the namespace.
- Triage, flag, verify, move on.
- Keep the docs open and search by task name.

Confidence comes from reps. If you can do the labs blind, you are ready.

Key Takeaways

- **2 hours, 66%, v1.35**, performance-based, open `kubernetes.io/docs`
- **Switch context first** on every task — wrong cluster = zero
- Generate YAML with `--dry-run=client -o yaml`, then edit
- **Triage and verify**; never get stuck on one task
- Weight your prep toward **Troubleshooting + Architecture (55%)**

Good Luck!

You've got this.

Practice the labs · Trust your reflexes · Verify every answer